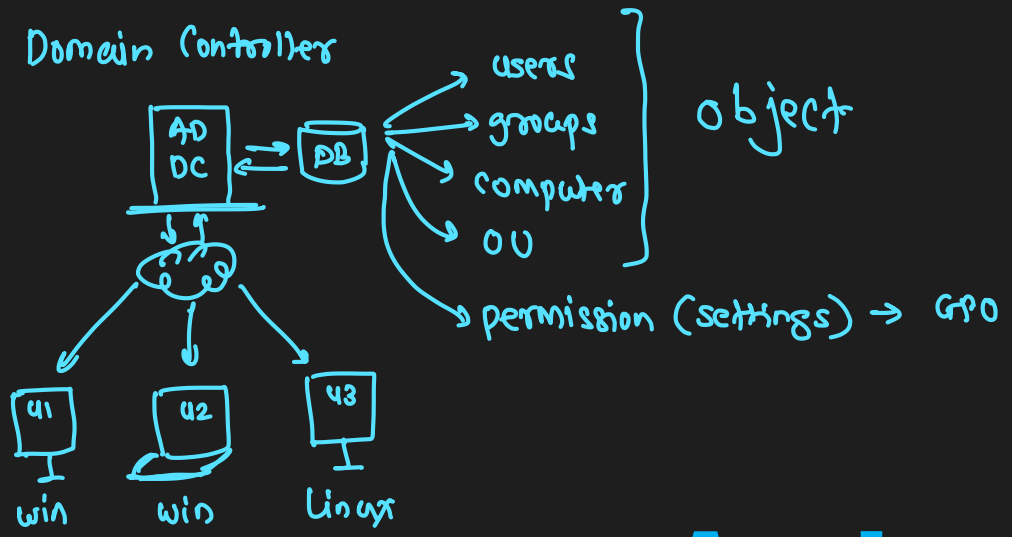


local users
created on local machine

Active Directory

core feature of windows server

Introduction

- Active Directory (AD) is a set of directory services developed by Microsoft, first released as part of Windows 2000 Server
- It was designed to help organizations manage on-premises, domain-based networks by giving administrators a centralized way to manage users, computers, and resources across the entire organization
- The most widely deployed component is **Active Directory Domain Services (AD DS)**
- Think of a directory as the organisation's phone book plus security guard
- It answers four questions for every request on the network:
  - **Identification** – who are you? (a user or computer object)
  - **Authentication** – prove it (password, smart card, certificate, Kerberos ticket)
  - **Authorisation** – what may you access? (group membership checked against permissions)
  - **Management** – what settings should you get? (Group Policy applied from the directory)

centralized database

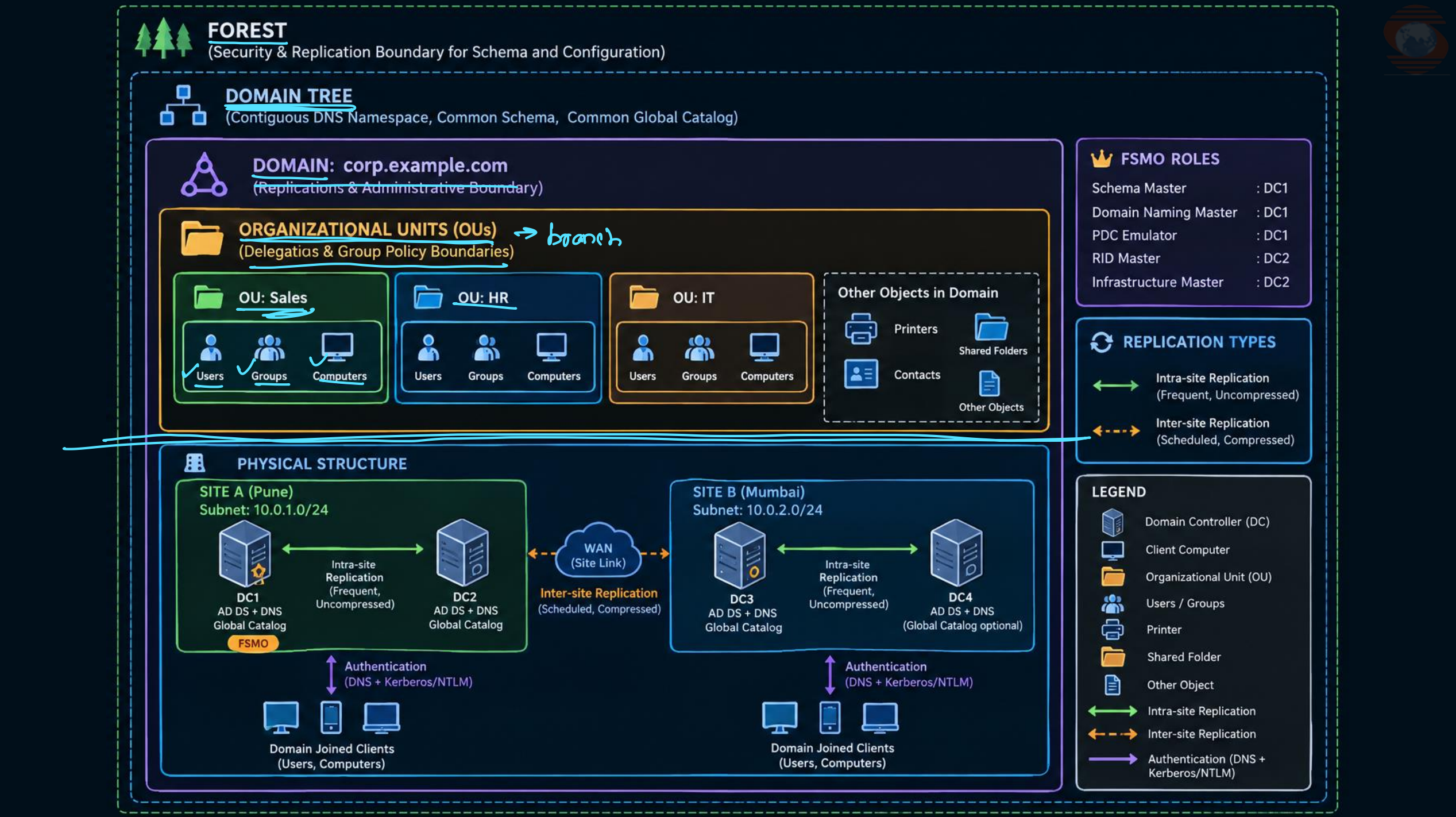
(GPO)

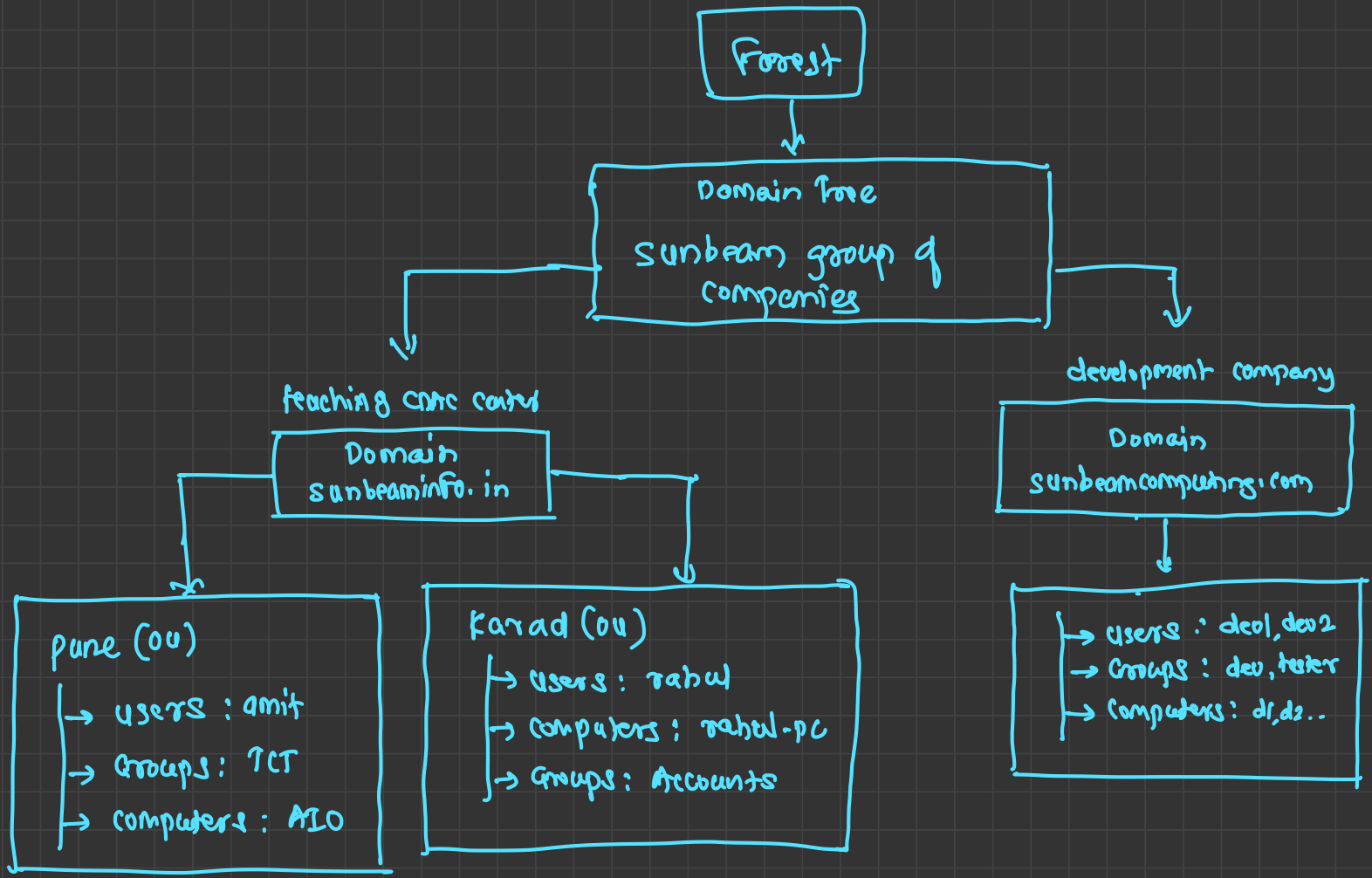
to check if user exists in the AD database

authenticated user will be checked for permission

History of Active Directory

1993–1996	Windows NT 3.1 – NT 4.0	NT domains: a flat SAM database, one writable Primary Domain Controller (PDC) and read-only Backup DCs (BDCs), NetBIOS/WINS naming, NTLM authentication, practical limit of about 40,000 objects per domain
2000	Windows 2000 Server	**Active Directory born**: hierarchical, DNS-based, LDAP access, Kerberos v5, multi-master replication, forests/trees/OUs, Group Policy. Design inspired by the X.500 directory standard
2003	Windows Server 2003	Forest trusts, domain rename, install-from-media, universal group membership caching, ADAM (later AD LDS)
2008	Windows Server 2008	Renamed **AD DS**; Read-Only Domain Controller (RODC), fine-grained password policies, restartable AD DS service, Server Core DCs, DFS-R replication of SYSVOL
2009	Windows Server 2008 R2	AD Recycle Bin, Active Directory Administrative Center (ADAC), ActiveDirectory PowerShell module, Managed Service Accounts, offline domain join
2012	Windows Server 2012	Virtualisation-safe DCs and DC cloning, group Managed Service Accounts (gMSA), Dynamic Access Control, adprep built into the promotion wizard
2013	Windows Server 2012 R2	Protected Users group, authentication policies and silos
2016	Windows Server 2016	Privileged Access Management (time-bound group membership), Azure AD (now Entra ID) integration. **Last new functional level for eight years**
2018–2021	Windows Server 2019 / 2022	No new functional level; security hardening only
2023	Updates to 2019/2022	Windows LAPS built into the OS; Azure AD renamed Microsoft Entra ID
2024	**Windows Server 2025** (GA 1 Nov 2024)	New **Windows Server 2025 functional level**, optional 32k database pages, LDAP encryption by default, TLS 1.3 for LDAP, NUMA support, delegated MSAs (dMSA) – see section 6


FOREST
(Security & Replication Boundary for Schema and Configuration)
DOMAIN TREE
(Contiguous DNS Namespace, Common Schema, Common Global Catalog)
DOMAIN: corp.example.com
(Replications & Administrative Boundary)
ORGANIZATIONAL UNITS (OUs)
(Delegatias & Group Policy Boundaries)
branch
OU: Sales
Users
Groups
Computers
OU: HR
Users
Groups
Computers
OU: IT
Users
Groups
Computers
Other Objects in Domain
Printers
Shared Folders
Contacts
Other Objects
FSMO ROLES
Schema Master : DC1
Domain Naming Master : DC1
PDC Emulator : DC1
RID Master : DC2
Infrastructure Master : DC2
REPLICATION TYPES
Intra-site Replication
(Frequent, Uncompressed)
Inter-site Replication
(Scheduled, Compressed)
PHYSICAL STRUCTURE
SITE A (Pune)
Subnet: 10.0.1.0/24
DC1
AD DS + DNS
Global Catalog
FSMO
Intra-site
Replication
(Frequent,
Uncompressed)
DC2
AD DS + DNS
Global Catalog
WAN
(Site Link)
Inter-site Replication
(Scheduled, Compressed)
SITE B (Mumbai)
Subnet: 10.0.2.0/24
DC3
AD DS + DNS
Global Catalog
Intra-site
Replication
(Frequent,
Uncompressed)
DC4
AD DS + DNS
(Global Catalog optional)
Authentication
(DNS + Kerberos/NTLM)
Domain Joined Clients
(Users, Computers)
Authentication
(DNS + Kerberos/NTLM)
Domain Joined Clients
(Users, Computers)
LEGEND
Domain Controller (DC)
Client Computer
Organizational Unit (OU)
Users / Groups
Printer
Shared Folder
Other Object
Intra-site Replication
Inter-site Replication
Authentication (DNS + Kerberos/NTLM)

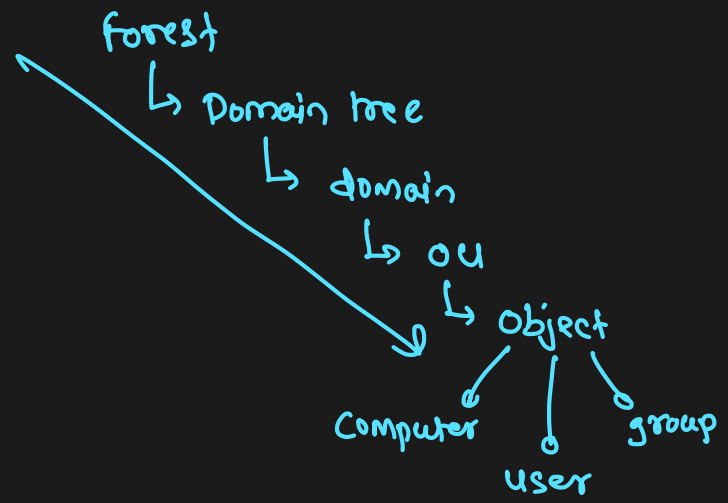

Forest
Domain tree
sunbeam group of companies
teaching care center
Domain
sunbeaminfo.in
development company
Domain
sunbeamcomputing.com
pune (ou)
→ users : amit
→ Groups : TCT
→ computers : AIO
Karad (ou)
→ users : rahul
→ computers : rahul-pc
→ Groups : Accounts
→ users : dev1, dev2
→ Groups : dev, tester
→ Computers : d1, d2..

Logical Structure

Logical Structure

plan / template

- The logical structure is the "blueprint" of Active Directory
- It defines how objects are organized, named, and related to one another
- It is independent of the physical network topology
- It contains
  - Objects
  - Organizational Units (OU)
  - Domain
  - Domain Tree
  - Forest
  - Global Catalog
  - Trust Relationship



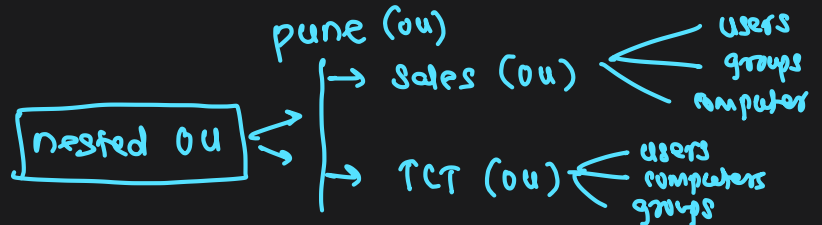
- sunbaem.local
  - Domain Controllers
  - Users / Computers)
  - _Admin
  - Pune (ou)
    - Staff
    - Workstations
    - Servers
  - Karad
    - Staff
    - Workstations
    - Servers
  - Groups
    - Role (Global)
    - Resource (Domain Local)
  - Service Accounts

Objects

- An **object** is the fundamental unit of storage in Active Directory
- Every entity in the directory—a user, a computer, a printer, a group, a shared folder, or an application—is represented as an object
- Each object has
  - **Attributes:** Characteristics that describe the object (e.g., a user object has attributes for first name, last name, email, phone number, and department)
  - **A globally unique identifier (GUID):** A 128-bit number assigned at creation that never changes, even if the object is renamed or moved
  - **A Security Identifier (SID):** Used for security and access control
- Objects are defined by the schema, which specifies what types of objects can exist and what attributes they can have
  - → class
- The schema is shared forest-wide, ensuring consistency across all domains

Organizational Units (OUs)

- An **Organizational Unit (OU)** is a container within a domain that is used to organize objects into a logical administrative hierarchy
- OUs serve two primary purposes
  - **Delegation of administration:**
    - You can grant a user or group administrative authority over a specific OU without giving them rights over the entire domain
    - For example, a help desk team might be delegated the right to reset passwords only for users in the "Sales" OU
  - **Group Policy application:**
    - Group Policy Objects (GPOs) can be linked to OUs, allowing different policies to be applied to different departments, locations, or roles



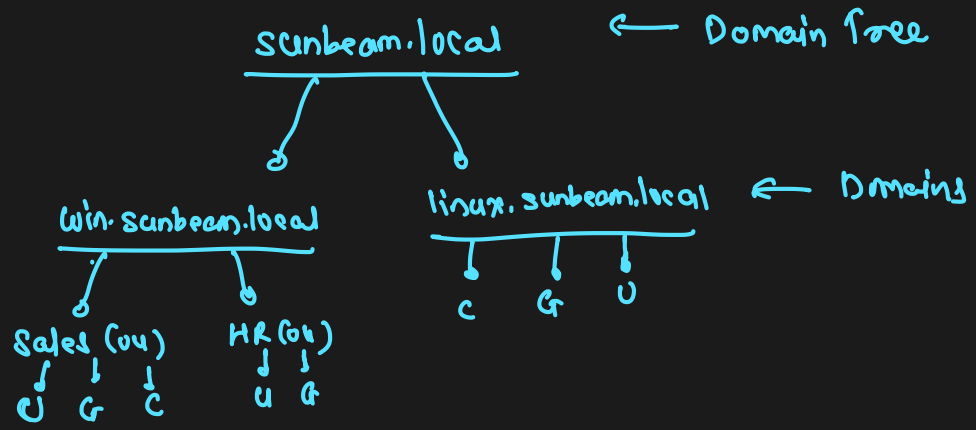
- OUs can be nested to create a hierarchy that mirrors the organization's structure
- Unlike domains, OUs are not security boundaries—they are administrative and policy boundaries
- OUs are not the same as **containers;** Containers (like the default "Users" or "Computers" container) are also objects that hold other objects, but you cannot link GPOs to containers, and you cannot delegate administration on them in the same granular way

Domain

- A **domain** is the core logical administrative unit in Active Directory
- It is a collection of objects (users, computers, groups) that share
  - **A common directory database:** Each domain has its own partition of the AD database
  - **A common security policy:** Password policies, account lockout policies, and Kerberos policies are defined at the domain level
  - **A common DNS namespace:** Domains are identified by DNS names (e.g., contoso.com, sales.contoso.com)
  - **Trust relationships:** Domains within a forest automatically trust each other through two-way transitive trusts
- Domains function as
  - **Replication boundaries:** Changes to objects in a domain are replicated to all domain controllers within that domain
  - **Administrative boundaries:** The Domain Admins group has full control over all objects in the domain
  - **Authentication boundaries:** Users authenticate to their own domain, and trust relationships allow access to resources in other domains

Domain Trees

- A **domain tree** is a hierarchical collection of domains that share a common root domain and a contiguous DNS namespace
- For example, contoso.com might have child domains sales.contoso.com, hr.contoso.com, and emea.contoso.com
- All domains in a tree share
  - A common schema
  - A common global catalog
  - Automatic two-way transitive Kerberos trusts
  - A common root domain



- Child domains can have their own child domains, creating a multi-level hierarchy
- The tree structure is primarily a naming and namespace convention; from an administrative perspective, all domains in a tree are peers (though the root domain holds the forest-level FSMO roles)

Forests

- A **forest** is the top-level logical container in Active Directory
- It is a collection of one or more domain trees that share
  - **A common schema:** All objects in the forest conform to the same definitions.
  - **A common global catalog:** A searchable, partial replica of all objects in the forest.
  - **A common configuration partition:** Replication topology and site link information.
  - **A common trust:** All domains in a forest trust each other automatically through two-way transitive trusts.
  - **A common root domain:** The first domain created in the forest (the forest root domain).
- The forest is the **primary security boundary** in Active Directory
- By default, no users from outside the forest can access resources inside the forest
- The forest is also the **replication boundary** for the schema and configuration partitions, and the boundary for the global catalog
- **FSMO roles at the forest level:** The Schema Master and Domain Naming Master roles exist only once per forest and are held by domain controllers in the forest root domain

The Global Catalog → directory used for searching across the domains

- The **global catalog** is a distributed data repository that contains a searchable, partial representation of every object in every domain in a multi-domain forest
- It is stored on **global catalog servers** (domain controllers configured to host the global catalog)
- The global catalog serves two critical functions:
  - **Forest-wide searches:** When a user searches for an object in a forest with multiple domains, the global catalog provides the fastest response because it contains information about all objects across all domains
  - **Logon and authentication:** During logon, a domain controller queries the global catalog to determine group membership for universal groups and to authenticate users from other domains
- The global catalog contains a **partial attribute set** for each object—only the attributes most commonly used for searches (e.g., first name, last name, email, department)
- The schema defines which attributes are replicated to the global catalog

Trust Relationships

- Trusts are logical relationships between domains or forests that allow users in one domain to be authenticated by a domain controller in another domain
- Active Directory supports several types of trusts
- **Parent-Child:** Automatic two-way transitive trust between a parent domain and its child domains
- **Tree-Root:** Automatic two-way transitive trust between the root domains of trees in the same forest
- **External:** Non-transitive trust between an AD domain and a Windows NT 4.0 domain or another AD forest
- **Forest:** Transitive trust between two AD forests
- **Realm:** Trust between an AD domain and a non-Windows Kerberos realm
- **Shortcut:** Explicit trust that shortens the trust path between domains in the same forest

Naming and identity

- **Distinguished Name (DN):** Full LDAP path of an object, read right-to-left
- **Relative DN (RDN):** The object's own name part of the DN
- **CN / OU / DC (in a DN):** Common Name / Organizational Unit / Domain Component
- **User Principal Name (UPN):** Email-style logon name
- **sAMAccountName:** Pre-Windows 2000 logon name (max 20 characters)
- **Security Identifier (SID):** Unique, never-reused ID of a security principal = Domain SID + RID
- **Relative Identifier (RID):** The last part of a SID, unique within the domain. Built-in Administrator is always RID 500
- **objectGUID:** 128-bit globally unique ID; never changes, even if the object is renamed or moved
- **SID history:** Old SIDs kept on an object after migration between domains
- **Security principal:** Anything that can be granted permissions: user, computer, security group, service account

Physical Structure

The Physical Structure

- The physical structure defines how Active Directory is actually implemented on servers, networks, and sites
- It is concerned with replication, authentication traffic, and network efficiency
- It contains
  - Domain Controllers (DCs)
  - The AD DS Data Store
  - Replication
  - Sites and Subnets
  - Read-Only Domain Controllers (RODCs)
  - Global Catalog Servers
  - Operations Masters (FSMO Roles)

Domain Controllers (DCs)

- A **domain controller** is a server that runs Active Directory Domain Services (AD DS)
- Every domain controller
  - Holds a writable copy of the AD DS database for its domain (except RODCs, which are read-only)
  - Authenticates users and computers
  - Participates in replication with other domain controllers
  - Hosts the SYSVOL folder, which contains Group Policy templates and logon scripts
  - Can host the global catalog (if configured)
  - Can hold one or more FSMO roles
- Domain controllers are the heart of Active Directory
- They are typically deployed in pairs or greater numbers for redundancy and load distribution
- If a domain controller fails, another DC in the same site can handle authentication and replication

The AD DS Data Store

- The AD DS database is stored in a file called **Ntds.dit**, which uses Microsoft Jet database technology
- By default, it is located in C:\Windows\NTDS\.
- The data store consists of:
  - **Ntds.dit:** The main database file
  - **Transaction log files:** Edb.log and additional log files (Edb00001.log, etc.) that record changes before they are committed to the main database
  - **Checkpoint file:** Edb.chk, used for recovery
  - **Reserved log files:** Edbres00001.jrs and Edbres00002.jrs, reserved for emergency recovery
- The database is designed to support very large numbers of objects (millions) and is optimized for read-heavy workloads

Replication

- **Replication** is the process of synchronizing the AD DS database across all domain controllers in a domain (and across domains for the schema and configuration partitions)
- Replication ensures that every domain controller has a consistent view of the directory
- Active Directory uses a **multi-master replication model:** every domain controller can accept changes and replicate them to other domain controllers. This differs from a single-master model, where only one server accepts writes
- Replication occurs
  - **Within a site:**
    - Automatically and frequently (every 15 seconds for urgent changes, every 5 minutes for normal changes)
    - Replication is uncompressed and fast because bandwidth is plentiful
  - **Between sites:**
    - According to a schedule defined by **site links**
    - Replication is compressed to save bandwidth and typically occurs every 180 minutes (3 hours) by default, though this can be customized

Replication

- **Knowledge Consistency Checker (KCC)**
  - The KCC is a process that runs on every domain controller and automatically generates the replication topology
  - It creates connections between domain controllers to ensure that changes propagate efficiently and reliably, with no more than three hops between any two DCs in a site
- **Urgent replication**
  - Certain changes—such as account lockouts, password changes, and LSA secrets—are replicated immediately (within seconds) to all domain controllers in the domain, regardless of site boundaries, to prevent security issues

Sites and Subnets

- A **site** is a physical or logical grouping of well-connected computers, typically corresponding to a local area network (LAN) or a high-speed WAN link
- **Sites are used to**
  - **Optimize replication:** Replication within a site is frequent and uncompressed; replication between sites is scheduled and compressed
  - **Optimize client authentication:** Clients attempt to authenticate with a domain controller in their own site first, reducing WAN traffic and improving logon performance
  - **Optimize resource location:** Clients use site information to find the nearest DFS servers, global catalog servers, and other services
- A **subnet** is a range of IP addresses (e.g., 192.168.1.0/24) that is associated with a site
- When a client logs on, it sends its IP address to a domain controller, which uses the subnet-to-site mapping to determine which site the client belongs to
- The client is then referred to a domain controller in that site (if one exists)
- **Site links:**
  - A site link is a logical connection between two or more sites that defines the cost and schedule for replication
  - Site links can be **bridged** to create transitive connections between sites, or **bridge all site links** can be enabled to automatically create transitive links

Read-Only Domain Controllers (RODCs)

- A **Read-Only Domain Controller (RODC)** is a special type of domain controller that hosts a read-only copy of the AD DS database
- RODCs are designed for branch offices and other locations where physical security cannot be guaranteed
- Key characteristics of RODCs
  - **Read-only database:** No changes can be made directly on an RODC; all changes must be made on a writable DC and replicated to the RODC
  - **Credential caching:**
    - By default, RODCs do not cache user credentials
    - You can explicitly allow specific users' credentials to be cached on the RODC, so they can log in even if the WAN link is down
  - **Filtered attribute set:** Certain sensitive attributes (e.g., passwords) are not replicated to RODCs
  - **Unidirectional replication:** RODCs receive replication from writable DCs but do not replicate changes back
  - **Reduced administrative risk:** If an RODC is compromised, the attacker cannot make changes to the directory, and only cached credentials are at risk

Global Catalog Servers

- A **global catalog server** is a domain controller that hosts the global catalog
- In a single-domain forest, all domain controllers are global catalog servers by default
- In a multi-domain forest, you can choose which domain controllers host the global catalog
- **Global catalog servers are essential for**
  - **User logon:**
    - When a user logs on, the authenticating DC contacts a global catalog server to determine the user's group membership (especially universal groups)
  - **Forest-wide searches:**
    - Applications and users can search the global catalog to find objects in any domain in the forest
  - **Exchange address book lookups:**
    - Microsoft Exchange uses the global catalog to resolve email addresses
- **Placement considerations**
  - In a multi-domain forest, at least one global catalog server should be placed in each site that has a domain controller, to avoid forcing clients to contact a global catalog server across a WAN link

Operations Masters (FSMO Roles)

- While Active Directory uses multi-master replication for most operations, certain operations are not well-suited to multi-master updates due to the potential for conflicts
- These operations are handled by **Operations Masters**, also known as **Flexible Single Master Operations (FSMO)** roles
- There are following roles
  - Schema Master: Controls all updates to the schema
  - Domain Naming Master: Controls the addition and removal of domains in the forest
  - RID Master: Allocates RID pools to domain controllers for creating unique SIDs
  - Infrastructure Master: Updates references from objects in its domain to objects in other domains
  - PDC Emulator: Handles password changes, account lockouts, time synchronization, and acts as the preferred DC for Group Policy and DFS
- FSMO roles are assigned to specific domain controllers
- If a domain controller holding an FSMO role fails, the role can be transferred or seized to another domain controller

Database and files

- **NTDS.dit:** The AD database file, default C:\Windows\NTDS\ntds.dit
- **ESE (Extensible Storage Engine):** The transactional database engine under NTDS.dit (8k pages, optional 32k in 2025)
- **edb.log / edbXXXXX.log:** Transaction logs; changes are written here first, then committed to NTDS.dit
- **edb.chk:** Checkpoint file marking which logs are committed
- **SYSVOL:** Shared folder (C:\Windows\SYSVOL) replicated to all DCs; holds Group Policy templates and logon scripts
- **NETLOGON share:** Share inside SYSVOL for logon scripts
- **DFSR (Distributed File System Replication):** Engine that replicates SYSVOL between DCs (replaced FRS, which is removed)

Tools

- **ADUC (dsa.msc):** Active Directory Users and Computers – classic GUI for objects
- **ADAC (dsac.exe):** Active Directory Administrative Center – modern GUI, Recycle Bin, PSOs, PowerShell history viewer
- **AD Sites and Services (dssite.msc):** Sites, subnets, site links, replication
- **AD Domains and Trusts (domain.msc):** Trusts, UPN suffixes, functional levels
- **GPMC (gpmc.msc):** Group Policy Management Console
- **ADSI Edit (adsiedit.msc):** Low-level editor for any attribute in any partition – use carefully
- **ActiveDirectory PowerShell module:** Get-ADUser, New-ADUser, Install-ADDSForest, etc.
- **repadmin, dcdiag, ntdsutil, nltest, netdom, dsquery, ldp:** Command-line diagnostics and maintenance
- **RSAT:** Remote Server Administration Tools – run all the above from a Windows 11 workstation

Authentication and Access Layer

Kerberos Authentication (Primary Protocol)

- Kerberos is the default authentication protocol in Active Directory environments
- It is an industry-standard protocol (version 5) defined by the IETF, and Windows Server implements it with extensions for public key authentication, authorization data transport, and delegation
- **The Three Parties Involved**
  - **The Client** — acting on behalf of a security principal (a user, computer, or service)
  - **The Service** — the resource the principal wants to access
  - **The Key Distribution Center (KDC)** — a trusted third party that runs on every domain controller and uses the Active Directory database as its account database
- **Kerberos Ticket Types**
  - **TGT (Ticket-Granting Ticket):** Proves the user's identity to the KDC; used to request service tickets
  - **Service Ticket (TGS):** Proves the user's identity to a specific service

Kerberos Authentication Flow

- **Step 1: Obtaining a Ticket-Granting Ticket (TGT)**
  - When a user signs in, the Kerberos client contacts the KDC to prove the user's identity
  - This is done through a process called **pre-authentication**, where the client encrypts a timestamp with a key derived from the user's password
  - The KDC decrypts this to verify the user knows the password.
    - f successful, the KDC issues a **Ticket-Granting Ticket (TGT)**
  - TGT is a credential that allows the client to request access to services without presenting the user's password again
  - The client stores the TGT in its ticket cache and reuses it until it expires
- **Step 2: Requesting a Service Ticket**
  - To connect to a service, the client presents its TGT to the KDC and requests a **service ticket** for that specific service
  - The KDC returns a service ticket, which the client presents directly to the target service
- **Step 3: Service Validates the Ticket**
  - Because the service ticket already contains the information the service needs to verify the client's identity, the service can authenticate the client **without contacting a domain controller** for every connection

Benefits of Kerberos

- **Single Sign-On (SSO)**
  - After the initial domain logon through Winlogon, Kerberos manages credentials across the forest whenever resource access is attempted
  - Users access permitted resources without repeated credential prompts
- **Delegated Authentication**
  - Kerberos supports a delegation mechanism that allows a service to act on behalf of its client when connecting to other services
  - This is essential for distributed applications where a front-end service must use the client's identity when connecting to back-end services on other computers
- **Interoperability**
  - Because Microsoft's implementation is based on IETF standards, Windows Kerberos interoperates with other systems that use Kerberos authentication

NTLM Authentication (Fallback Protocol)

- NTLM (NT LAN Manager) is a family of authentication protocols included in Windows, comprising LAN Manager versions 1 and 2, and NTLM versions 1 and 2
- NTLM authenticates users and computers based on a **challenge/response mechanism** that proves a user knows the password associated with an account
- **When NTLM is Used**
  - Kerberos version 5 is the preferred authentication method for Active Directory environments
  - However, NTLM is still supported and must be used for
    - **Workgroup environments:** Systems configured as members of a workgroup (not joined to a domain)
    - **Local logon authentication:** On non-domain controllers
    - **Legacy applications:** A Microsoft or non-Microsoft application may still use NTLM even in a Kerberos domain
    - **Remote access without domain trust:** When Kerberos cannot be used

NTLM Flow

- When NTLM is used, a resource server must either contact a domain authentication service on the domain controller (for domain accounts) or look up the account in the local account database (for local accounts) whenever a new access token is needed
- The challenge/response process works as follows
  - **Client sends authentication request** to the server
  - **Server sends an NTLM challenge** (a random number)
  - **Client encrypts the challenge** with a hash of the user's password and sends the response
  - **Server forwards the challenge and response** to the domain controller
  - **Domain controller verifies** the response against the account database and returns allow/deny

Hands-On

Prerequisites

- Before installing Active Directory, ensure the server meets the following conditions:
- **Static IP address** is configured, **computer name** has been set as desired (avoid changing the name after promotion)
- Sufficient disk space is available, and the server is updated with the latest patches
- Perform operations as a **local administrator**
- DNS should run locally on domain controllers, and in production environments at least two domain controllers should be deployed at each site

Network Settings

- Run this first to confirm the exact name of your network adapter
- It may be Ethernet, Ethernet0, or something else depending on the NIC

```
Get-NetAdapter
```

- **Clear any existing IP configuration and assign new IP address**

```
Remove-NetIPAddress -InterfaceAlias "Ethernet" -AddressFamily IPv4 -Confirm:$false
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.10.10 -PrefixLength 24 -DefaultGateway 192.168.10.1
```

- **Point DNS to Itself**

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 192.168.10.10
```

- **Set the Time Zone**

```
Set-TimeZone -Id "India Standard Time"
```

- **Rename the Computer and Restart**

```
Rename-Computer -NewName "DC01" -Restart
```

Install AD DS Role

- Installing the AD DS role does not require a restart; the -IncludeManagementTools parameter will also install management tools such as Active Directory Users and Computers and the ADDSDeployment PowerShell module

```
Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools
```

- After installation, you can verify the role status

```
Get-WindowsFeature AD-Domain-Services
```

Create a New Forest and Promote to Domain Controller

- This command creates the new forest and root domain for sunbeam.local
- The -InstallDns switch is critical because the Domain Controller must also provide DNS for the domain
- The -SafeModeAdministratorPassword is the Directory Services Restore Mode (DSRM) password, which you will need if you ever have to restore AD from a backup
- Use a strong password and store it securely

```
Install-ADDSForest `
  -DomainName "sunbeam.local" `
  -DomainNetbiosName "SUNBEAM" `
  -DomainMode "Win2025" `
  -ForestMode "Win2025" `
  -InstallDns:$true `
  -DatabasePath "C:\Windows\NTDS" `
  -LogPath "C:\Windows\NTDS" `
  -SysvolPath "C:\Windows\SYSVOL" `
  -CreateDnsDelegation:$false `
  -SafeModeAdministratorPassword (Read-Host -Prompt "DSRM Password:" -AsSecureString) `
  -Force:$true
```

Verify the Installation

- After the server automatically reboots, log in with a domain administrator account (local accounts will be disabled) and run the following command to verify

```
Get-ADForest
Get-ADDomain
```

- You can also check the domain controller status:

```
Get-ADDomainController -Filter * | Format-Table Name, OperatingSystem, IsGlobalCatalog
```

Create an Organizational Unit (OU)

- OUs help you organize users, groups, and computers
- This creates a dedicated OU for your lab users, which keeps them separate from the default Users container

```
New-ADOrganizationalUnit -Name "Sunbeam Users" -Path "DC=sunbeam,DC=local"
```

Create a test user

- To create a new Active Directory user, you can use the New-ADUser cmdlet in PowerShell

```
New-ADUser -Name "John Doe" `
           -GivenName "John" `
           -Surname "Doe" `
           -SamAccountName "jdoe" `
           -UserPrincipalName "jdoe@sunbeam.local" `
           -Path "OU=Users,DC=sunbeam,DC=local" `
           -AccountPassword (ConvertTo-SecureString "P@ssw0rd123" -AsPlainText -Force) `
           -Enabled $true `
           -ChangePasswordAtLogon $true
```

Joining Domain

Windows Machine

- **Prerequisites**
  - **Windows Edition:**
    - The machine must be running **Pro, Enterprise, or Education** edition
    - Home editions cannot join a domain
  - **Permissions:** You need an account with **local administrator rights** on the Windows machine to perform the join
  - **Network & DNS:**
    - The machine must be on the same network as the Domain Controller and, crucially, be configured to use the **Domain Controller's IP address as its DNS server**
    - This is required to locate the domain

Windows Machine

- **Using the Settings App (Windows 11)**
  - Open **Settings**
  - Go to **Accounts > Access work or school**
  - Click **Connect**
  - Select the option **"Join this device to a local Active Directory domain"**
  - Enter the domain name (e.g., yourdomain.com) and click **Next**
  - Enter the credentials of an account with permission to join computers to the domain
  - Restart the computer when prompted
- **Using PowerShell**
  - Open **PowerShell as Administrator**
  - **Execute following command**

```
Add-Computer -DomainName "sunbeam.local" -Credential (Get-Credential) -Restart
```

- **Verify the Join for sunbeam.local**

```
systeminfo | findstr /i "Domain"
```

Linux Machine

- **Install packages**

```
sudo dnf install -y realmd sssd oddjob oddjob-mkhomedir adcli samba-common-tools krb5-workstation
```

- **Discover the Domain**

```
sudo realm discover sunbeam.local
```

- **Join the Domain**

```
sudo realm join sunbeam.local -U Administrator
```

- **Set Crypto Policy Workaround**

```
sudo update-crypto-policies --set DEFAULT:AD-SUPPORT
```

- **Configure SSSD and Home Directories**

```
sudo authselect select sssd with-mkhomedir --force
sudo systemctl enable --now oddjobd
sudo systemctl restart sssd
```

Linux Machine

- **Verify Domain Membership**

```
realm list
```

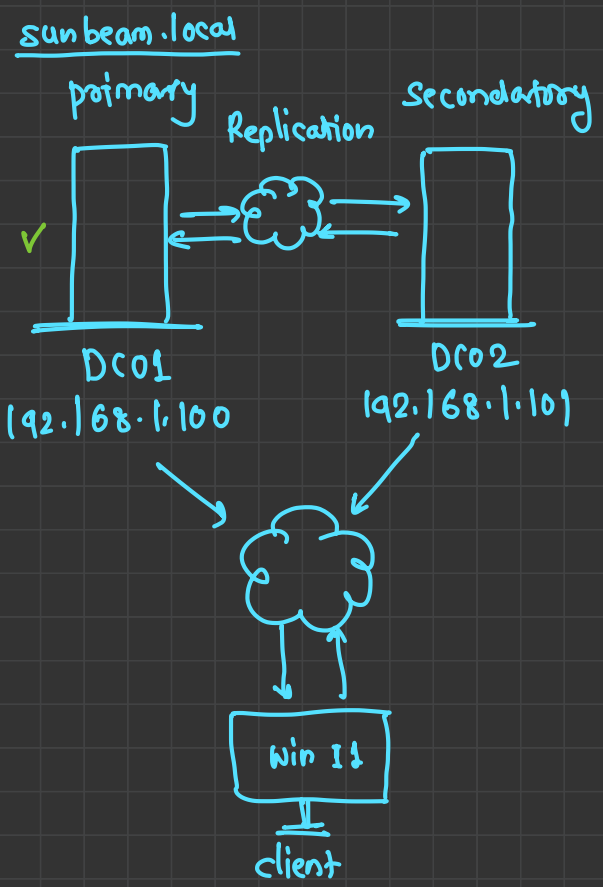
- **Test AD User Lookup**

```
sudo getent passwd your_ad_user@sunbeam.local
id your_ad_user@sunbeam.local
```

- **Test Kerberos Authentication**

```
kinit your_ad_user@SUNBEAM.LOCAL
```

Additional Domain Controller


sunbeam.local
primary
Replication
Secondaryory
DC01
192.168.1.100
DC02
192.168.1.101
Win 11
client

Introduction

- An **Additional Domain Controller (ADC)** is a domain controller added to an **existing Active Directory domain** to provide redundancy, load balancing, and fault tolerance alongside the first (original) domain controller
- An ADC is any writable or read-only domain controller installed **after** the first DC has already created the domain
- It replicates the existing AD database from a current DC
- The first DC creates the domain; the ADC joins that existing domain as a helper and backup server
- The term is commonly used in practice to distinguish a newly added DC from the original (first) DC in a domain
- Technically, after installation, all writable DCs in a domain are peers and participate in multi-master replication
- **Why Add an ADC?**
  - **Fault tolerance** — if one DC fails, the domain continues to function
  - **Load balancing** — distributes authentication and directory requests across multiple servers
  - **Improved performance** — clients connect to a nearby DC, reducing network traffic
  - **Site support** — provide local authentication for branch offices or remote sites

Key Considerations

- **FSMO Roles:** Can all be placed on one reliable DC; move them during maintenance if needed
- **Global Catalog:** In a single-domain forest, make all DCs GCs — no planning needed
- **Replication Monitoring:** Use **Repadmin** to verify replication health after adding an ADC
- **Backup:** Even with multiple DCs, regularly back up **System State** using Windows Server Backup
- **Physical Separation:** Place DCs in different physical locations or power circuits to avoid single points of failure

Types of Domain Controllers

- **Writable DC:** Full read/write copy of AD database; participates in replication → default
- **Read-Only Domain Controller (RODC):** Read-only copy; designed for branch offices
- **Global Catalog (GC) Server:** Holds partial replicas of all forest domains
- **FSMO Role Holder:** Holds one or more single-master operation roles

Requirements for Adding an ADC

- **Administrative Requirements**
  - **Domain Admin** or **Enterprise Admin** credentials
  - If adding a new domain to a forest: **Enterprise Admin**
  - Local **Administrator** rights on the target server
- **Hardware Requirements**
  - Processor: 1.4 GHz 64-bit (2+ GHz multi-core)
  - RAM: 512 MB - 2 GB+ (4–8 GB for large domains)
  - Disk Space: 32 GB - 100 GB+ (depends on AD size)
  - Network: 100 Mbps- Gigabit Ethernet
- **Software and Configuration Requirements**
  - **Windows Server** (~~2008, 2008 R2, 2012, 2012 R2, 2016,~~ 2019, 2022, 2025)
  - **NTFS-formatted** volumes (do **not** use ReFS for AD database, logs, or SYSVOL)
  - **Static IP address** (strongly recommended)
  - **DNS configuration** pointing to an existing DC/DNS server
  - **Network connectivity** to existing DCs
  - **Time synchronization** within 5 minutes of existing DCs
  - **Sufficient disk space** for ntds.dit, logs, and SYSVOL
  - adprep run if required (forest/domain prep for new OS versions)

Required Ports for Replication

- **TCP/UDP 53**: DNS
- **TCP/UDP 88**: Kerberos authentication
- **TCP/UDP 389**: LDAP
- **TCP 135**: RPC Endpoint Mapper
- **TCP/UDP 445**: SMB (for SYSVOL replication)
- **TCP 3268/3269**: Global Catalog
- **TCP/UDP 464**: Kerberos password change
- **Dynamic RPC ports**: 49152-65535 (for RPC/DCOM)

Features of an Additional Domain Controller

- **Authentication:** Verifies user/computer credentials via Kerberos or NTLM
- **Authorization:** Controls access to network resources
- **AD Database Storage:** Hosts ntds.dit and SYSVOL
- **Replication:** Multi-master replication with all other DCs
- **Group Policy:** Distributes and enforces GPOs
- **DNS Integration:** AD-integrated DNS zones
- **Global Catalog:** Optional; forest-wide search capability
- **FSMO Roles:** Can hold one or more FSMO roles
- **LDAP Support:** Directory access via LDAP
- **Kerberos KDC:** Acts as Key Distribution Center
- **Site Awareness:** Clients find nearest DC
- **Load Balancing:** Distributes authentication requests

Backup and Recovery of ADC

- **System State Backup**
  - Registry
  - COM+ Class Registration Database
  - Boot files
  - Active Directory (on DCs)
  - SYSVOL
  - Certificate Services (on CAs)
  - Cluster database (on cluster nodes)
  - IIS metabase (on IIS servers)
- **Recovery types**
  - Non-Authoritative: Restores objects; allows replication to overwrite
  - Authoritative: Restores objects; marks them not to be overwritten
  - Bare Metal Recovery: Full system restore
  - AD Recycle Bin: Restores deleted objects (Server 2008 R2+)

Best Practices

- **Minimum two DCs per domain:** Fault tolerance
- **Make all DCs GCs in single-domain forests:** Simplifies design
- **Place DCs in different physical locations:** True fault tolerance
- **Use static IP addresses:** Reliable DNS registration
- **Regular System State backups:** Protect against logical corruption
- **Monitor replication daily:** Early problem detection
- **Keep time sync accurate:** Kerberos requires <5 min difference
- **Use RODCs for branch offices:** Security in low-physical-security sites
- **Document all FSMO role holders:** Quick recovery
- **Test restores regularly:** Verify backup integrity
- **Implement tiered administration:** Least privilege
- **Clean up metadata after seizure:** Allow old DC to rejoin

Install Over the Network

- Install the **Active Directory Domain Services** role
- Run the **AD DS Configuration Wizard**
- Choose **Add a domain controller to an existing domain**
- Select options: **DNS Server**, **Global Catalog**, and/or **RODC**
- Select a source DC to replicate from (or allow the wizard to choose)
- Set paths for database, logs, and SYSVOL
- Complete prerequisites check and install

Benefits

- Redundancy / Fault Tolerance
  - If one DC fails, others continue authenticating users
- Load Balancing
  - Authentication requests distributed across multiple DCs
- Better Performance
  - Local DCs reduce WAN traffic and logon times
- Site Awareness
  - Clients use nearest DC for faster authentication
- Disaster Recovery
  - Surviving DC preserves the domain if one server is lost
- Reduced Downtime
  - Maintenance can be performed on one DC while others serve clients

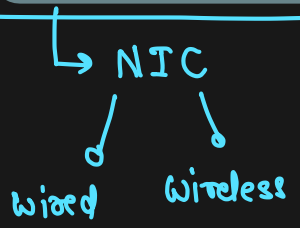
Disadvantages

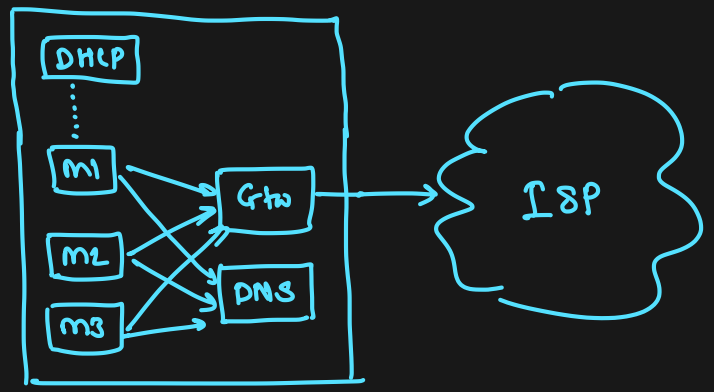
- Increased Cost
  - Requires additional server hardware, licensing, and power
- Replication Traffic
  - More DCs = more replication traffic on the network
- Management Complexity
  - More servers to patch, monitor, and secure
- Diminishing Returns
  - Each additional DC adds only slight availability gains but higher costs
- USN Rollback Risk
  - Improper VM snapshots or restores can corrupt AD (use supported backups only)

Network Management

Understanding IPv4 Networking

- In IPv4, each node needs its own IP address, written in dotted decimal notation
  - E.g. 192.168.4.22/24
- Each IP address must be indicated with the subnet mask behind it
- The default router or gateway specifies which server to forward packets to that have an external destination
- The DNS nameserver is the IP address of a server that helps resolving names to IP addresses
- IPv4 is still the most common IP version, but IPv6 addresses can be used as well
- IPv6 addresses are written in hexadecimal notation (fe80::143f:bf00:cd27:a844%en0)
- IPv4 and IPv6 can co-exist on the same network interface





Naming conventions

- Interface prefixes
  - en: ethernet
  - wl: wireless LAN (WLAN)
  - ww: Wireless Wide Area Network (WWAN)
  - ib: InfiniBand
- Interface naming conventions:
  - eno1: Onboard device index from firmware/BIOS
  - ens1: PCIe hotplug slot index
  - enp2s0: Physical location of hardware connector
  - enx525400d5e0fb: MAC address (optional)
  - eth0: Traditional kernel naming (fallback)

Core Management Tools

- nmtui (Text User Interface)
  - Menu-driven, curses-based interface
  - More intuitive for beginners
  - No need to remember command syntax

```
# Install if not present
sudo dnf install NetworkManager NetworkManager-tui

# Launch
nmtui
```

- nmcli (Command Line)
  - More powerful and scriptable
  - Faster for repeated tasks
  - Recommended for advanced users and automation

```
# Basic syntax
nmcli [OPTIONS] OBJECT {COMMAND}

# Common objects: connection, device, general
```

Basic Operations & Commands

Command [sub command] [arguments]

Viewing Configuration

```
# Show all connections
nmcli connection show

# Show active connections only
nmcli connection show --active

# Show detailed device info
nmcli device show enp0s3

# Show IP addresses
ip addr
```

Connection Management

```
# Bring connection up/down
nmcli connection up enp0s3
nmcli connection down enp0s3

# Down and up in sequence (minimal downtime)
nmcli con down enp0s3 && nmcli con up enp0s3

# Delete a connection
nmcli connection delete enp0s3

# Reload all connections
nmcli connection reload
```

Understanding the ip command

- The ip command can be used to manage all aspects of IP networking
- It replaces the legacy ifconfig command [so do not use ifconfig anymore]
- Use **ip addr** to manage address properties
- Use **ip link** to show the link properties
- Use **ip route** to manage the route properties

Setting up IP address

- Setting up IP address manually

```
# Step 1: Set method to manual (static)
nmcli connection modify "ens160" ipv4.method manual

# Step 2: Set IP address, gateway, and DNS
nmcli connection modify "ens160" \
  ipv4.addresses "192.168.50.1/24" \
  ipv4.gateway "192.168.50.254" \
  ipv4.dns "8.8.8.8 1.1.1.1"

# Step 3: Apply changes (MUST do this)
nmcli connection down "ens160" && nmcli connection up "ens160"
```

- Setting up IP address automatically

```
# To revert to DHCP (automatic IP)
nmcli connection modify "ens160" \
  ipv4.method auto \
  ipv4.addresses "" \
  ipv4.gateway "" \
  ipv4.dns ""

# Apply
nmcli connection down "ens160" && nmcli connection up "ens160"
```

Managing Networks

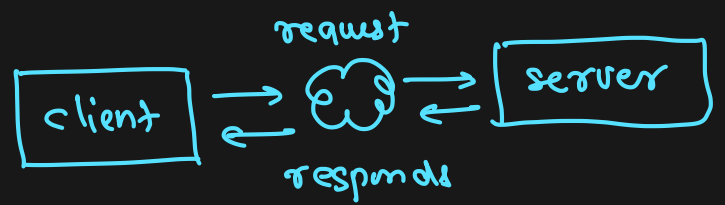
- **ip link show**: shows the network interfaces
- **ip –s link show**: shows the network interfaces along with statistics
- **ip route**: shows routing table
- **ping <ip>**: used to check the connectivity between hosts
- **traceroute <domain>**: used to see the route taken to reach domain
- **tracepath <domain>**: used to see the path taken to reach domain
- **ss –ta**: used to investigate socket issues
- **nmcli**: used to modify interface settings and save to /etc/sysconfig/network-scripts/
- **dig <domain>**: use to show the dns record of the domain

DHCP Server

DHCP

- A network management protocol used to automatically assign IP addresses and other network configuration parameters to devices on a TCP/IP network
- Eliminates the need for manual (static) IP configuration
- Operates at the Application Layer of the OSI model
- Uses UDP as the transport protocol
- Ports
  - DHCP Server: UDP port 67
  - DHCP Client: UDP port 68
- Why DHCP?
  - Automation: Reduces administrative overhead
  - Avoids Conflicts: Prevents two devices from getting the same IP address
  - Efficient IP Use: Dynamically allocates IPs from a pool, releasing unused ones
  - Mobility: Devices get valid IPs automatically when moving between networks
  - Centralized Management: Configuration (DNS, gateway) is managed from a DHCP server

Key Components



- **DHCP Server** → DHCP service is running
  - A device (router or server) that holds IP addresses and configuration.
- **DHCP Client**
  - Any device requesting an IP (laptop, phone, printer).
- **Scope** → pool of IP addresses
  - A range of IP addresses available for lease to clients.
- **Lease**
  - The time duration for which an IP is assigned to a client.
- **Reservation**
  - A permanent lease (static mapping) of an IP to a specific MAC address.
- **Options**
  - Additional parameters (e.g., default gateway, DNS server, domain name).

DHCP Messages

- DHCPDISCOVER
  - Client to Server (broadcast)
  - Client searches for a DHCP server
- DHCPOFFER
  - Server to Client (unicast/broadcast)
  - Server offers an available IP address
- DHCPREQUEST
  - Client to Server (broadcast)
  - Client accepts the offer
- DHCPACK
  - Server to Client (unicast)
  - Server confirms assignment and provides lease + options
- DHCPNAK
  - Server to Client
  - Denies the request (e.g., IP no longer available or invalid)
- DHCPDECLINE
  - Client to Server
  - Client detects IP is already in use (e.g., via ARP)
- DHCPRELEASE
  - Client to Server
  - Client voluntarily gives up its IP address
- DHCPINFORM
  - Client to Server
  - Client already has an IP (manually) but needs other options (DNS, gateway)

DHCP Lease Lifecycle

- Initialization (Booting)
  - Client sends DISCOVER: obtains IP (DORA)
- Normal Operation
  - Client uses the IP
- Renewal (T1 - 50% of lease time)
  - Client sends unicast REQUEST to original server to renew lease
- Rebinding (T2 - 87.5% of lease time)
  - If renewal fails, client broadcasts REQUEST (any server can respond)
- Expiration
  - If no response, client stops using IP and starts DORA over

Best Practices

- Only one DHCP server per network - Disable DHCP on your router to avoid conflicts
- Server needs static IP - DHCP servers should not obtain their own IP via DHCP
- Plan your IP range - Ensure your DHCP pool doesn't overlap with static IPs used by servers, printers, or network equipment
- Backup configuration - Always backup /etc/dhcp/dhcpd.conf before making changes
- Monitor lease exhaustion - Use tools like dhcpd-pools to monitor IP usage

dhcp server

- Configure server IP address
  - sudo nmcli connection modify "ens160" ipv4.address <ip>
  - sudo nmcli connection modify "ens160" ipv4.method manual
  - sudo nmcli connection down "ens160"
  - sudo nmcli connection up "ens160"
  - ip address show
- Install dhcpd server
  - sudo dnf update -y
  - sudo yum install -y dhcp-server
- Configure server (/etc/dhcp/dhcpd.conf)

```
default-lease-time 600;
max-lease-time 7200;
authoritative;
subnet 192.168.50.0 netmask 255.255.255.0 {
  range 192.168.50.100 192.168.50.200;
  option routers 192.168.50.1;
  option subnet-mask 255.255.255.0;
  option domain-name-servers 8.8.8.8;
}
```

- Validate configuration
  - sudo dhcpd -t
- Bind dhcp with interface
  - sudo vim /etc/sysconfig/dhcpd
  - DHCPDARGS=ens160
- Start server
  - sudo systemctl enable --now dhcpd
- Firewall settings
  - sudo firewall-cmd --add-service dhcp --permanent
  - sudo firewall-cmd --reload

Kea

Introduction

- It is an open-source DHCP server developed by the Internet Systems Consortium (ISC)
- It is designed to provide scalable, high-performance, and secure DHCP services for IPv4 and IPv6 networks. Kea is a modern DHCP server that replaces the ISC DHCP server (isc-dhcp-server) and is intended to be more efficient, flexible, and secure
- It was first released in 2015 as a replacement for the ISC DHCP server
- It was designed to provide a more modern and scalable DHCP server for IPv4 and IPv6 networks
- Kea has undergone several major releases since its initial release, with the latest version being 2.1.6. Kea has gained popularity in the open-source community due to its flexibility, scalability, and security features

Features

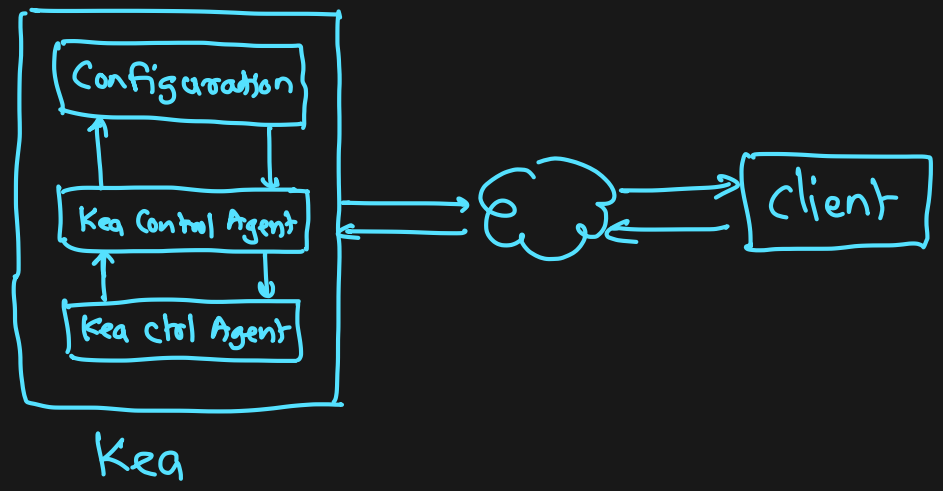
- IPv4 and IPv6 Support: Kea supports both IPv4 and IPv6 address allocation
- Scalability: Kea is designed to handle large and complex networks with a high number of clients
- Flexibility: Kea provides a flexible configuration system that allows administrators to define custom policies and behavior
- Security: Kea includes various security features such as authentication, authorization, and encryption to protect against unauthorized access and malicious activities
- High Performance: Kea is optimized for high-performance DHCP operations, ensuring fast and efficient address allocation
- API and Hooks: Kea provides a robust API and hook system, allowing developers to create custom scripts and plugins to extend its functionality
- Logging and Monitoring: Kea includes a built-in logging and monitoring system, enabling administrators to track and analyze DHCP activity

Pros and Cons

- Pros of Kea
  - Scalability: Kea is designed to handle large and complex networks with a high number of clients
  - Flexibility: Kea provides a flexible configuration system that allows administrators to customize the DHCP server to meet their specific needs
  - Security: Kea includes several security features, including support for DHCPv6-Shield and DHCPv4-leasequery
  - Ease of Use: Kea is relatively easy to configure and manage, especially when compared to other DHCP servers
  - Open-Source: Kea is open-source, which means that it is free to use and customize
- Cons of Kea
  - Complexity: Kea can be complex to configure and manage, especially for large and complex networks
  - Resource-Intensive: Kea can be resource-intensive, which means that it may require significant amounts of CPU and memory
  - Limited Support: Kea has limited support for certain features, such as subnets and pools
  - Steep Learning Curve: Kea has a steep learning curve, which can make it difficult for administrators to learn and use

Kea Components

- kea-dhcp4: This is the IPv4 DHCP server component, responsible for allocating IPv4 addresses to clients
- kea-dhcp6: This is the IPv6 DHCP server component, responsible for allocating IPv6 addresses to clients
- kea-ctrl-agent: This component manages the communication between the Kea DHCP server and other components, such as the Kea Control Agent
- kea-control-agent: This component provides a centralized management interface for Kea, enabling administrators to configure and monitor the DHCP server



Common use cases

- DHCP Server for IPv4 and IPv6 Networks: Kea can be used as a DHCP server for both IPv4 and IPv6 networks
- Scalable DHCP Server: Kea can be used as a scalable DHCP server for large and complex networks
- Secure DHCP Server: Kea can be used as a secure DHCP server with features such as DHCPv6-Shield and DHCPv4-leasequery
- Flexible DHCP Server: Kea can be used as a flexible DHCP server with a customizable configuration system

Alternatives to Kea

- **ISC DHCP Server:** The ISC DHCP server is a popular open-source DHCP server that is widely used
- **Dnsmasq:** Dnsmasq is a lightweight DHCP server that is designed for small to medium-sized networks
- **Pump:** Pump is a lightweight DHCP server that is designed for small to medium-sized networks
- **Barracuda:** Barracuda is a commercial DHCP server that is designed for large and complex networks

Kea Configuration

- kea.conf
  - This is the main configuration file for Kea, where administrators define the DHCP server's behavior and policies
- dhcp4.conf
  - This file contains IPv4-specific configuration settings, such as subnet and pool definitions
- dhcp6.conf
  - This file contains IPv6-specific configuration settings, such as subnet and pool definitions

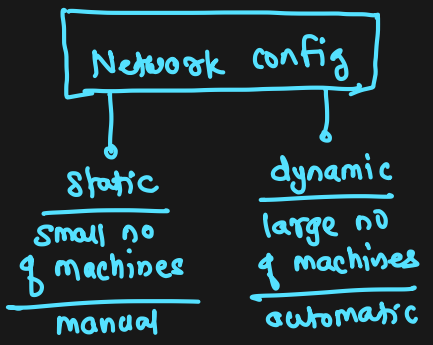
DHCP Server

dynamically assigns IP address

DCHP Server

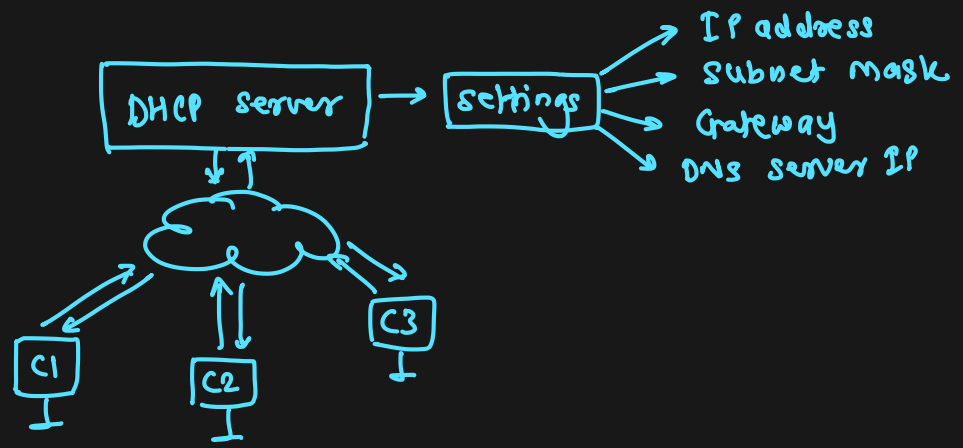
- Subnet mask
- Gateway
- DNS server

- DHCP is a network management protocol used on IP networks
- Its primary function is to **automatically assign IP addresses** and other **network configuration parameters** to devices (hosts) so they can communicate on the network
- Without DHCP, network administrators would have to manually configure IP addresses, subnet masks, gateways, and DNS servers for every device, which is impractical for large networks (like Wi-Fi hotspots or corporate networks)
- **Key Characteristics**
  - **Protocol Type:** Client-Server model
  - **Transport Layer:** Uses **UDP** (User Datagram Protocol)
  - **Ports:** Server listens on **UDP Port 67**; Client listens on **UDP Port 68**
  - **Broadcast Domain:** Initial communication uses broadcasts because the client doesn't have an IP address yet
  - **Standards:** Defined in RFC 2131 (IPv4) and RFC 3315 (DHCPv6)
- **Ports**
  - DHCP Server: UDP port 67
  - DHCP Client: UDP port 68



Why DHCP ?

- **Automation:** Reduces administrative overhead
- **Avoids Conflicts:** Prevents two devices from getting the same IP address
- **Mobility:** Devices get valid IPs automatically when moving between networks
- **Scalability:** Essential for large networks where devices join and leave frequently (BYOD, IoT)
- **Centralized Management:** Administrators can change network settings (like DNS servers) in one place, and all devices get the update
- **IP Address Reuse:** When a device leaves the network, its IP address is returned to the pool for reuse, preventing exhaustion of addresses



Key Components

- **DHCP Server** : program (service | daemon)
  - A device (router or server) that holds IP addresses and configuration.
- **DHCP Client** : program (ipconfig)
  - Any device requesting an IP (laptop, phone, printer).
- **Scope** 192.168.1.150 to 192.168.1.160 → address pool (collection of addresses)
  - A range of IP addresses available for lease to clients.
- **Lease** : when lease time is over, the client renews settings
  - The time duration for which an IP is assigned to a client.
- **Reservation** : useful for servers
  - A permanent lease (static mapping) of an IP to a specific MAC address.
- **Options**
  - Additional parameters (e.g., default gateway, DNS server, domain name).

DHCP Process

- **Step 1: DHCP Discover (Client -> Server)**
  - The client boots up without an IP
  - It broadcasts a DHCPDISCOVER message to the entire network (Destination: 255.255.255.255)
  - Source IP: 0.0.0.0, Destination Port: 67
  - Goal: "Is there a DHCP server out there?"
- **Step 2: DHCP Offer (Server -> Client)**
  - All DHCP servers that receive the Discover message respond with a DHCPOFFER
  - The offer contains a proposed IP address, subnet mask, lease duration, and the server's IP
  - If multiple servers respond, the client picks one.
  - Goal: "I have an IP for you."
- **Step 3: DHCP Request (Client -> Server)**
  - The client broadcasts a DHCPREQUEST message
  - It requests the specific IP offered by the chosen server
  - This broadcast also informs other DHCP servers that their offers were declined
  - Goal: "I accept your offer."
- **Step 4: DHCP Acknowledgment (Server -> Client)**
  - The chosen server sends a DHCPACK (Acknowledgment)
  - This contains the final configuration (IP, DNS, Gateway) and starts the lease timer
  - If the IP is no longer available (e.g., taken by someone else in the meantime), the server sends a DHCPNAK (Negative Acknowledgment), and the client must restart the process
  - Goal: "Here is the IP and settings; you are good to go."

DHCP Lease and Renewal Process

- An IP address is not given forever; it is "leased."
- **Lease Time:** The duration the IP is valid (e.g., 24 hours, 7 days)
- **T1 Timer (50% of Lease):** When 50% of the lease time has passed, the client sends a **unicast DHCPREQUEST** to the server that gave it the IP to renew the lease
- **T2 Timer (87.5% of Lease):** If the server doesn't respond to the T1 renewal, the client waits until 87.5% of the lease has passed. It then **broadcasts** a DHCPREQUEST to any available DHCP server to renew or get a new IP
- **Expiry (100%):** If no response is received by the end of the lease, the client must stop using the IP and restart the DORA process

DHCP Messages

- DHCPDISCOVER
  - Client to Server (broadcast)
  - Client searches for a DHCP server
- DHCPOFFER
  - Server to Client (unicast/broadcast)
  - Server offers an available IP address
- DHCPREQUEST
  - Client to Server (broadcast)
  - Client accepts the offer
- DHCPACK
  - Server to Client (unicast)
  - Server confirms assignment and provides lease + options
- DHCPNAK
  - Server to Client
  - Denies the request (e.g., IP no longer available or invalid)
- DHCPDECLINE
  - Client to Server
  - Client detects IP is already in use (e.g., via ARP)
- DHCPRELEASE
  - Client to Server
  - Client voluntarily gives up its IP address
- DHCPINFORM
  - Client to Server
  - Client already has an IP (manually) but needs other options (DNS, gateway)

DHCP Allocation Methods

- **Dynamic Allocation**
  - The server assigns an IP from a pool for a limited time (lease)
  - Most common method
  - Used for general clients (laptops, phones)
- **Automatic Allocation** → deprecated
  - The server assigns a permanent IP to a client from the pool
  - Once assigned, the IP is tied to that client's MAC address forever
  - Rarely used today
- **Manual Allocation (Static DHCP / Reservations)**
  - The administrator assigns a specific IP to a specific MAC address
  - The client always gets the same IP
  - Used for servers, printers, and routers

Best Practices

- **Only one DHCP server per network** - Disable DHCP on your router to avoid conflicts
- **Server needs static IP** - DHCP servers should not obtain their own IP via DHCP
- **Plan your IP range** - Ensure your DHCP pool doesn't overlap with static IPs used by servers, printers, or network equipment
- **Backup configuration** - Always backup /etc/dhcp/dhcpd.conf before making changes
- **Monitor lease exhaustion** - Use tools like dhcpd-pools to monitor IP usage

DNS

DNS

- The Domain Name System (DNS) is a hierarchical and distributed naming system for computers, services, and other resources in the Internet or other Internet Protocol (IP) networks
- It associates various information with domain names assigned to each of the associated entities
- Most prominently, it translates readily memorized domain names to the numerical IP addresses needed for locating and identifying computer services and devices with the underlying network protocols
- The Domain Name System has been an essential component of the functionality of the Internet since 1985
- Uses port no 53

Hierarchy of Domain Names

students.sunbeamapps.org.

- Root Domain:
  - The dot (.) at the end (often invisible). Example: example.com.
- Top-Level Domain (TLD):
  - .com, .org, .net, .uk, .edu
- Second-Level Domain (SLD)
  - The registrable part. Example: google in google.com
- Subdomain
  - Optional prefix. Example: mail.google.com (mail is subdomain)

Terminologies

- **DNS Root Servers**
  - Name server for root zone of DNS of the internet
  - It directly answers requests for records in the root zone and answers other requests by returning a list of the authoritative name servers for the appropriate Top Level Domain (TLD)
  - TLDs are controlled by the root zone

- **Top Level Domain (TLD)**
  - A top-level domain (TLD) is one of the domains at the highest level in the hierarchical Domain Name System of the Internet
  - IANA currently distinguishes the following groups of top-level domains:
    - country-code top-level domains (ccTLD)
    - generic top-level domains (gTLD)
      - sponsored top-level domains (sTLD)
      - unsponsored top-level domains
    - infrastructure top-level domain (.arpa)

Terminologies

- **Subdomain**
  - Anything between the host and TLS is known as subdomain
  - E.g. sunbeaminfo.com is subdomain of .com TLD

- **Zone and zone files**
  - A zone of zone file is a mapping of IP address and the hosts for a given subdomain
  - The zone file for sunbeaminfo.com would contain a record for www

- **Name Server**
  - A server that runs the DNS service and can either store or cache the information for the DNS platform
  - Whether it stores or caches depends on whether it is referred from a higher level

- **Authoritative**
  - The organizations run name servers that become authoritative
  - They can answer the queries with authority

Terminologies

- **Resource Record (RR)**
  - This is information of element for network resources
  - The types of information elements are categorized and organized with a list of DNS record types
  - Resource records of the same type are described as a *resource record set* (RRset)
  - DNS resolvers return the entire set upon query, but servers may implement round-robin ordering to achieve load balancing
  - Each record has a
    - NAME: Name of the node to which this record pertains
    - TYPE: Type of RR in numeric form (e.g., 15 for MX RRs)
    - CLASS: Class code
    - TTL: Count of seconds that the RR stays valid (The maximum is $2^{31}-1$, which is about 68 years)
    - RDATA: Additional RR-specific data
    - RDLENGTH: Length of RDATA field (specified in octets)

Terminologies

- **Hosts**
  - A record in the zone file: www
- **FQDN**
  - Fully Qualified Domain Name
  - The host and the domain name
- **Registrar**
  - A domain name registrar is a company that manages the reservation of Internet domain names
  - A domain name registrar must be accredited by a generic top-level domain (gTLD) registry or a country code top-level domain (ccTLD) registry
  - A registrar operates in accordance with the guidelines of the designated domain name registries
- **Resolver**
  - The client side of the DNS is called a DNS resolver
  - A resolver is responsible for initiating and sequencing the queries that ultimately lead to a full resolution (translation) of the resource sought, e.g., translation of a domain name into an IP address

Types of DNS Records

- **A (Address):** Maps domain to IPv4 address
- **AAAA:** Maps domain to IPv6 address
- **CNAME (Canonical Name):** Aliases one domain to another (e.g., www to @)
- **MX (Mail Exchange):** Directs email to mail servers (with priority values)
- **TXT (Text):** Holds human/machine-readable text (used for verification)
- **NS (Name Server):** Specifies the authoritative DNS servers for the domain
- **PTR (Pointer):** Maps IP to domain (reverse DNS lookup)
- **SOA (Start of Authority):** Contains administrative info about the zone

Types of DNS Servers

- **Recursive Resolver:** Does all the legwork (steps 3-6) for the client
- **Root Name Server:** Top of hierarchy (13 logical sets worldwide). Knows TLD servers
- **TLD Name Server:** Handles specific extension (.com, .org, etc.). Knows Authoritative servers
- **Authoritative Name Server:** Final source of truth for a domain's records

How DNS Resolution Works (8 Steps)

- **User types** example.com into browser
- **Check cache:** Browser to OS to Router to ISP Resolver (if found, skip to end)
- **Query Root Server:** Resolver asks root "Where is .com?" Root replies with TLD server address
- **Query TLD Server:** Resolver asks .com server "Where is example.com?" TLD replies with Authoritative server
- **Query Authoritative Server:** Resolver asks the domain's own DNS server "What is the IP of example.com?"
- **Response:** Authoritative server returns A record (IP address)
- **Caching:** Resolver caches IP for future queries (based on TTL)
- **Deliver:** Resolver sends IP to browser, browser loads website

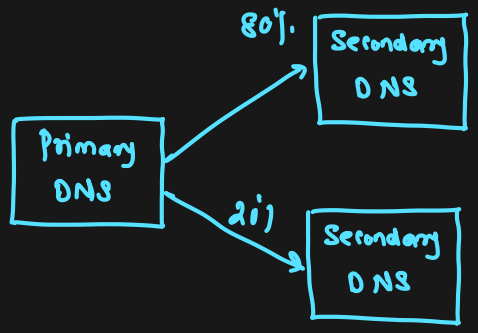
Bind

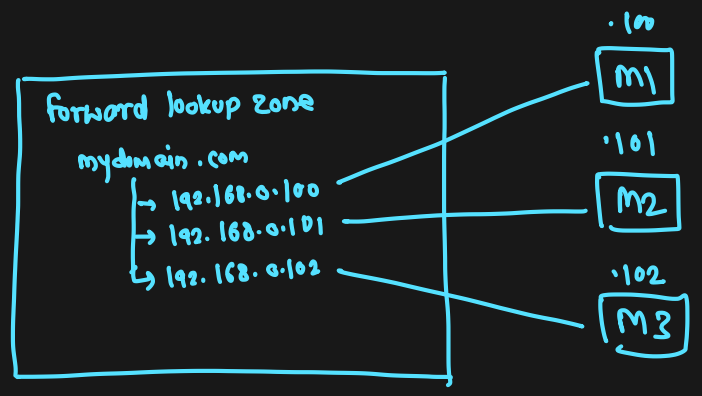
What is Bind ?

- Bind (Berkeley Internet Name Domain) is a popular open-source DNS server software for Linux and Unix-like systems
- It is widely used to provide DNS services to local networks and the internet
- Bind is a freely available, open-source DNS server software that was first released in 1989 by the University of California, Berkeley. The initial version was written by Paul Vixie, a renowned DNS expert
- The first version of Bind was called Berkeley Internet Name Domain (BIND) and was designed to provide a DNS server that was more reliable and efficient than the existing DNS server software available at that time
- Over the years, Bind has undergone significant changes and improvements, with major releases happening in 1994, 1997, 2001, 2004, 2006, 2008, 2011, and 2016
- Each major release has introduced new features, improved performance, and enhanced security

Features

- **Support for DNSSEC:** Bind supports DNSSEC (Domain Name System Security Extensions), which provides a secure way to authenticate DNS responses and prevent DNS spoofing attacks
- **Dynamic Updates:** Bind supports dynamic updates, which allow DNS records to be updated in real-time without requiring manual intervention
- **Load Balancing:** Bind supports load balancing, which enables multiple DNS servers to work together to distribute the workload and improve performance
- **Failover:** Bind supports failover, which allows DNS servers to automatically switch to a backup server in case the primary server becomes unavailable
- **BIND9 Features:** Bind9, the latest version of Bind, introduced several new features, including support for IPv6, improved performance, and enhanced security





Pros

- **Open-Source:** Bind is an open-source software, which means it's free to use and distribute
- **Highly Scalable:** Bind is designed to handle large volumes of DNS traffic and can scale to meet the needs of even the largest networks
- **Highly Configurable:** Bind offers a wide range of configuration options, which allows administrators to customize the behavior of the DNS server to meet their specific needs
- **Highly Secure:** Bind supports DNSSEC and other security features to prevent DNS spoofing attacks and other types of DNS-related threats
- **Wide Community Support:** Bind has a large and active community of developers and users, which provides extensive support and resources

Cons

- **Steep Learning Curve:** Bind has a complex configuration file format and a steep learning curve, which can make it difficult for new administrators to get started
- **Resource-Intensive:** Bind requires significant system resources, particularly memory and CPU, to handle large volumes of DNS traffic
- **Complex Maintenance:** Bind requires regular maintenance, including updates and upgrades, to ensure optimal performance and security
- **Potential for Configuration Errors:** Bind's complex configuration file format makes it easy to introduce configuration errors, which can cause problems with DNS resolution

Usage of Bind

- Enterprise Networks: Bind is commonly used in enterprise networks to provide DNS services to employees and other stakeholders
- Content Delivery Networks (CDNs): Bind is used in CDNs to provide DNS services to end-users and improve the performance of online content
- Cloud Providers: Bind is used in cloud providers to provide DNS services to customers and improve the performance of cloud-based applications
- ISP Networks: Bind is used in ISP networks to provide DNS services to customers and improve the performance of online applications

Best Practices

- **Regularly Update and Upgrade Bind:** Regularly update and upgrade Bind to ensure optimal performance and security
- **Use DNSSEC:** Use DNSSEC to secure DNS responses and prevent DNS spoofing attacks
- **Use Dynamic Updates:** Use dynamic updates to update DNS records in real-time without requiring manual intervention
- **Use Load Balancing and Failover:** Use load balancing and failover to improve performance and ensure high availability
- **Monitor System Resources:** Monitor system resources to ensure that Bind is properly configured and optimized for performance

Configuration

- Installation
  - sudo dnf install bind -y
- Configuration
  - sudo nano /etc/named.conf
  - Basic configuration

```
include "/etc/named.rfc1912.zones";
zone "example.com" IN {
    type master;
    file "/var/named/example.com.zone";
    };
zone "localhost" IN {
    type master;
    file "/var/named/localhost.zone";
    };
```

  - Defined two zones: example.com and localhost
  - The type master keyword indicates that this DNS server is authoritative for these zones

- Forward and Reverse Lookup

```
zone "example.com" IN {
    type master;
    file "/var/named/example.com.zone";
    };

zone "2.96.172.in-addr.arpa" IN {
    type master;
    file "/var/named/reverse.zone";
    };
```

  - A reverse lookup zone for the IP address 2.96.172.in-addr.arpa
  - This allows clients to perform reverse DNS lookups

Configuration

- Cache Manager

```
options {
        directory "/var/named";
        recursion no;
        pid-file "/var/run/named/named.pid";
        };

controls {
        inet 127.0.0.1 port 953
                allow {127.0.0.1;} keys {rndc-key;};
        };
```

  - We have enabled the cache manager with the recursion no keyword

- Start the service
  - sudo systemctl start bind

DNS

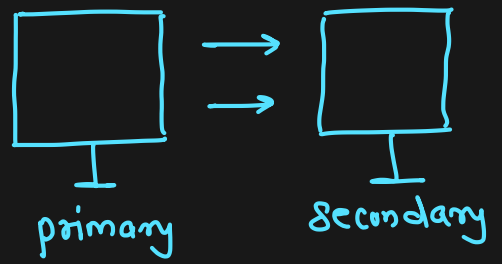
DNS

→ resolver
→ root DNS

→ high availability → replication

- The Domain Name System (DNS) is a hierarchical and distributed naming system for computers, services, and other resources in the Internet or other Internet Protocol (IP) networks
- It associates various information with domain names assigned to each of the associated entities
- Most prominently, it translates readily memorized domain names to the numerical IP addresses needed for locating and identifying computer services and devices with the underlying network protocols
- The Domain Name System has been an essential component of the functionality of the Internet since 1985 →
- Uses port no 53
- **Why DNS is Needed**
  - Humans remember names, not IP addresses
  - IP addresses can change; names remain stable
  - Enables scalability, decentralization, and load distribution

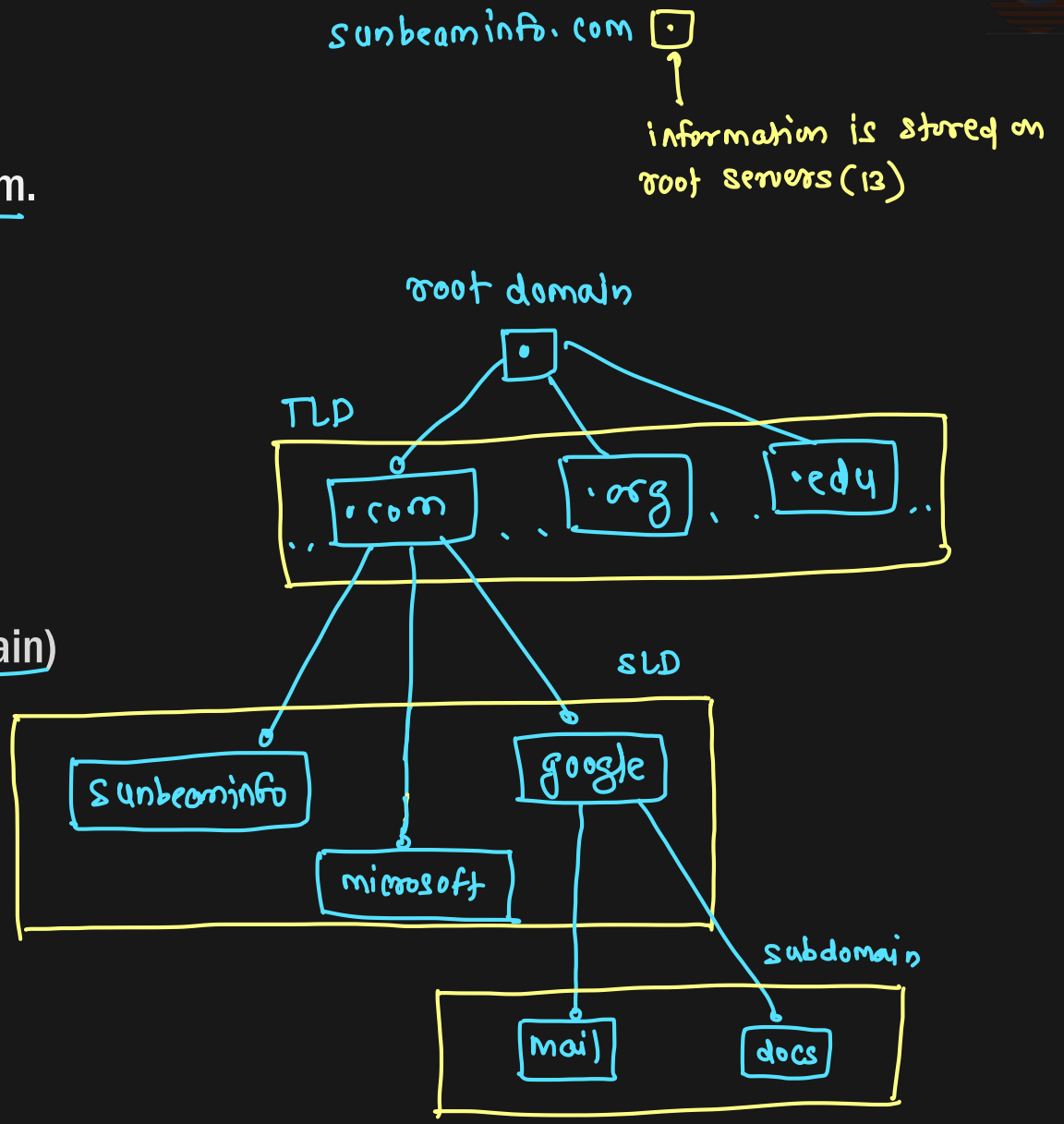
translate → Domain name → IP address (public network)
translate → Hostname → IP address (private network)



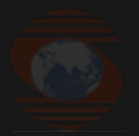
linux/macos : /etc/hosts
windows : c:\windows\system32\drivers\etc\hosts

Hierarchy of Domain Names

- **Root Domain:**
  - The dot (.) at the end (often invisible). Example: example.com.
- **Top-Level Domain (TLD):**
  - .com, .org, .net, .uk, .edu
- **Second-Level Domain (SLD)**
  - The registrable part. Example: google in google.com
- **Subdomain**
  - Optional prefix. Example: mail.google.com (mail is subdomain)



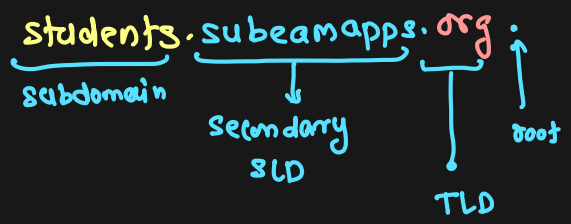
Terminologies



- **DNS Root Servers** → stores information about TLD servers
  - Name server for root zone of DNS of the internet
  - It directly answers requests for records in the root zone and answers other requests by returning a list of the authoritative name servers for the appropriate Top Level Domain (TLD)
  - TLDs are controlled by the root zone

- **Top Level Domain (TLD)**
  - A top-level domain (TLD) is one of the domains at the highest level in the hierarchical Domain Name System of the Internet
  - IANA currently distinguishes the following groups of top-level domains:
    - country-code top-level domains (ccTLD) → .co.in, .in, .uk, .us
    - generic top-level domains (gTLD)
      - sponsored top-level domains (sTLD) → .com
      - unsponsored top-level domains → .edu
    - infrastructure top-level domain (.arpa)

Terminologies



- **Subdomain**
  - Anything between the host and TLS is known as subdomain
  - E.g. sunbeaminfo.com is subdomain of .com TLD

- **Zone and zone files** → database of domain name to IP address mapping
  - A zone of zone file is a mapping of IP address and the hosts for a given subdomain
  - The zone file for sunbeaminfo.com would contain a record for www

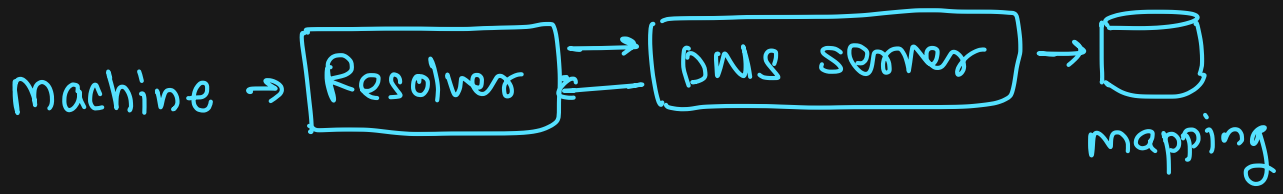
- **Name Server** → machine with DNS server configured
  - A server that runs the DNS service and can either store or cache the information for the DNS platform
  - Whether it stores or caches depends on whether it is referred from a higher level

- **Authoritative**
  - The organizations run name servers that become authoritative
  - They can answer the queries with authority
    - eg. GoDaddy.com

Terminologies

- **Resource Record (RR)** → stored within the zones
  - This is information of element for network resources
  - The types of information elements are categorized and organized with a list of DNS record types
  - Resource records of the same type are described as a *resource record set* (RRset)
  - DNS resolvers return the entire set upon query, but servers may implement round-robin ordering to achieve load balancing
  - Each record has a
    - NAME: Name of the node to which this record pertains
    - TYPE: Type of RR in numeric form (e.g., 15 for MX RRs)
    - CLASS: Class code
    - TTL: Count of seconds that the RR stays valid (The maximum is $2^{31}-1$, which is about 68 years)
    - RDATA: Additional RR-specific data
    - RDLENGTH: Length of RDATA field (specified in octets)

Terminologies



- **Hosts**
  - A record in the zone file: www
- **FQDN**
  - Fully Qualified Domain Name
  - The host and the domain name
- **Registrar** : godaddy | AWS | GCP
  - A domain name registrar is a company that manages the reservation of Internet domain names
  - A domain name registrar must be accredited by a generic top-level domain (gTLD) registry or a country code top-level domain (ccTLD) registry
  - A registrar operates in accordance with the guidelines of the designated domain name registries
- **Resolver**
  - The client side of the DNS is called a DNS resolver
  - A resolver is responsible for initiating and sequencing the queries that ultimately lead to a full resolution (translation) of the resource sought, e.g., translation of a domain name into an IP address

Types of DNS Records → resource records

redirection

domain1.com → domain2.com

- **A (Address):** Maps domain to IPv4 address
- **AAAA:** Maps domain to IPv6 address
- **CNAME (Canonical Name):** Aliases one domain to another (e.g., www to @)
- **MX (Mail Exchange):** Directs email to mail servers (with priority values)
- **TXT (Text):** Holds human/machine-readable text (used for verification)
- **NS (Name Server):** Specifies the authoritative DNS servers for the domain
- **PTR (Pointer):** Maps IP to domain (reverse DNS lookup)
- **SOA (Start of Authority):** Contains administrative info about the zone

DNS Zone : database of RRs

- It is a **portion of the DNS namespace** that is managed, administered, and stored by a single organization or administrator
- It is the **administrative boundary** for DNS data
- A zone is NOT the same as a domain
  - A domain is a node in the namespace; a zone is the part of the namespace that a particular server is authoritative for
- A zone contains **resource records** stored in a **zone file** on an authoritative name server
- **Why Zones Exist**
  - DNS is distributed — no single server holds all records
  - Zones **delegate authority**: an organization controls its own zone without asking the root or TLD servers every time
  - Zones enable **local management**, **redundancy**, and **scalability**
- **Zone vs Domain**

A zone can store multiple domain's information

Domain	Zone
A node in the DNS namespace	A portion of namespace administered together
Logical concept	Administrative concept
example.com is a domain	example.com may be one or more zones
Can contain many subdomains	Contains records for names within its boundary

Types of DNS Zones

- **Forward Zone (Forward Lookup Zone)**
  - Maps **domain names → IP addresses**
  - Most common type
  - Contains A, AAAA, CNAME, MX, TXT, etc
  - E.g. www.example.com → 93.184.216.34
- **Reverse Zone (Reverse Lookup Zone)**
  - Maps **IP addresses → domain names**
  - Uses **PTR records**
  - Uses special domain in-addr.arpa (IPv4) and ip6.arpa (IPv6)
  - E.g. 34.216.184.93.in-addr.arpa → www.example.com
- **Primary Zone (Master Zone)**
  - The **writable, authoritative copy** of a zone
  - Edits are made here first
  - Hosted on the **primary name server**
- **Secondary Zone (Slave Zone)**
  - **Read-only copy** of the primary zone
  - Gets updates via **zone transfer** (AXFR/IXFR)
  - Provides **redundancy** and **load balancing**
  - Cannot be edited directly

Zone File Components

- Every zone file must have
  - SOA (Start of Authority) — one per zone; defines zone parameters
  - NS records — authoritative servers for the zone
  - A/AAAA records — host addresses
  - Other records — MX, CNAME, TXT, etc
- SOA Record Fields
  - Primary NS: Master server for the zone
  - Responsible person: Admin email (with . instead of @)
  - Serial: Version number; incremented on changes
  - Refresh: How often secondary checks for updates
  - Retry: Wait time before retrying failed refresh
  - Expire: When secondary stops answering if no refresh
  - Minimum TTL: Default TTL for records / negative caching

Types of DNS Servers

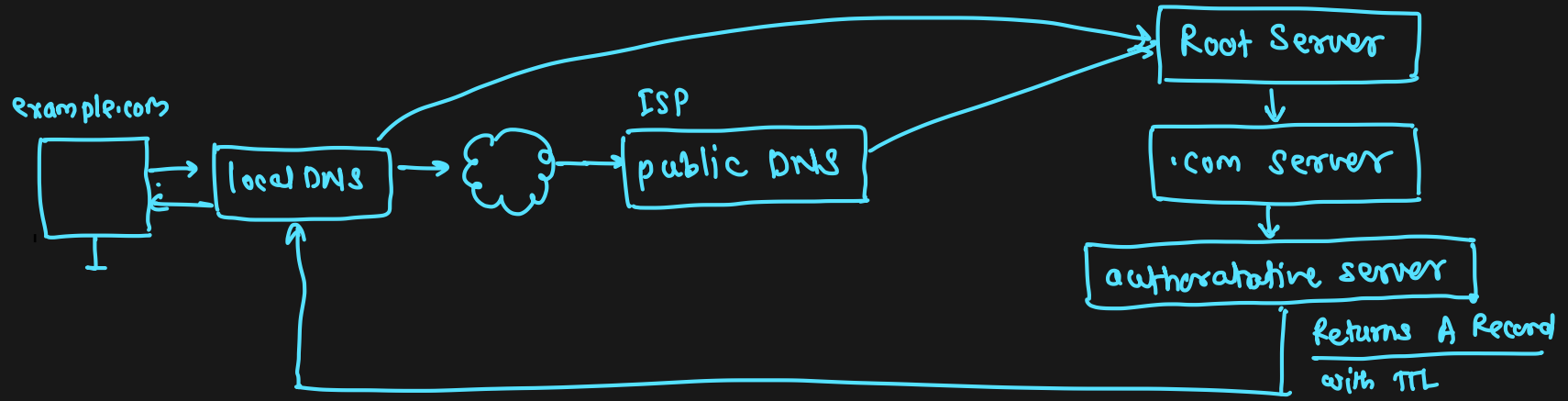
- **Authoritative DNS Servers**
  - In **authoritative server** is the **final source of truth** for a specific zone
  - It holds the actual DNS records (A, AAAA, MX, etc.) and gives definitive answers
  - **Characteristics**
    - Returns answers with the **AA (Authoritative Answer)** flag set
    - Does **not** query other servers on behalf of clients
    - Only answers for zones it is configured to manage
    - Can be queried directly by resolvers
- **Recursive DNS Servers (Resolvers)**
  - A **recursive resolver** does the "legwork" of finding answers for clients
  - It queries multiple servers (root → TLD → authoritative) and returns the final answer
  - **Characteristics**
    - Accepts queries from clients (stub resolvers)
    - **Recursively** queries other servers to find the answer
    - **Caches** results for future queries (based on TTL)
    - Does **not** hold authoritative data (unless combined with authoritative role)
    - Returns answers with the **RA (Recursion Available)** flag

Type of DNS Server

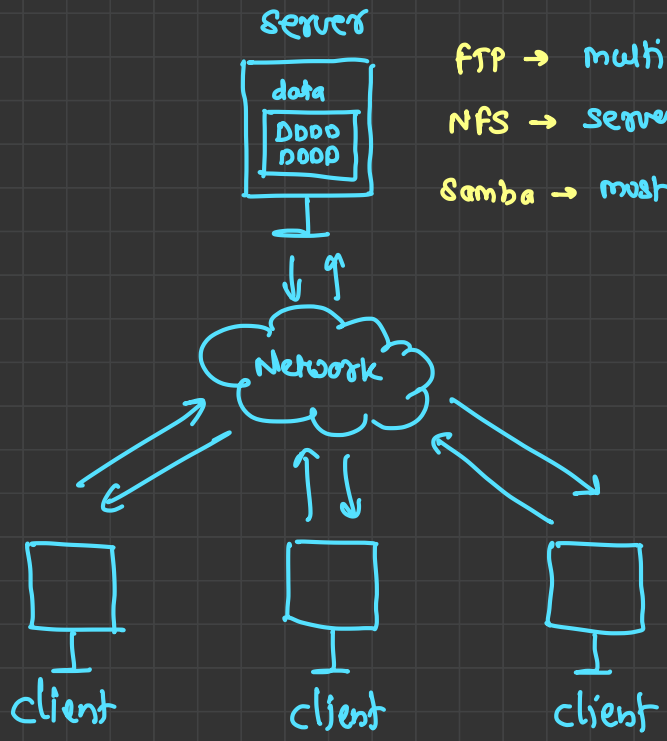
- **Root Name Servers**
  - The **root servers** sit at the very top of the DNS hierarchy
  - **Characteristics**
    - **13 logical root server clusters** (named A through M)
    - Hundreds of physical instances worldwide via **anycast**
    - Managed by ICANN and various organizations (Verisign, NASA, etc.)
    - Know the addresses of all **TLD name servers**
    - Do **not** know individual domain IPs — only where to find TLD servers
  - **Role:** When a resolver has no cache, it asks a root server: "Where is .com?" — the root replies with the .com TLD server addresses
- **TLD (Top-Level Domain) Name Servers**
  - These servers handle **top-level domains** like .com, .org, .net, .in, .uk.
  - **Characteristics**
    - Maintained by **TLD registries** (e.g., Verisign for .com)
    - Know the **authoritative name servers** for every domain under their TLD
    - Return **referrals** to authoritative servers
  - **Types of TLDs**
    - **gTLD** — Generic (.com, .org, .net, .info)
    - **ccTLD** — Country code (.in, .uk, .jp)
    - **Sponsored TLD** — .edu, .gov, .mil
    - **New gTLD** — .app, .dev, .xyz

How DNS Resolution Works (8 Steps)

- **User types** example.com into browser
- **Check cache:** Browser to OS to Router to ISP Resolver (if found, skip to end)
- **Query Root Server:** Resolver asks root "Where is .com?" Root replies with TLD server address
- **Query TLD Server:** Resolver asks .com server "Where is example.com?" TLD replies with Authoritative server
- **Query Authoritative Server:** Resolver asks the domain's own DNS server "What is the IP of example.com?"
- **Response:** Authoritative server returns A record (IP address)
- **Caching:** Resolver caches IP for future queries (based on TTL)
- **Deliver:** Resolver sends IP to browser, browser loads website



File server → Server used to shares files with clients



FTP → multi-os environment

NFS → server & client are linux

Samba → most of clients are windows

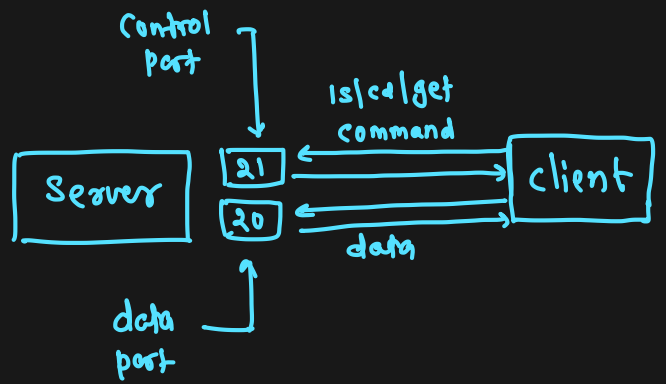
FTP

FTP

- FTP stands for **File Transfer Protocol**
- It is a standard network protocol used for the transfer of computer files from a server to a client on a computer network
- It was designed to allow users to upload, download, rename, move, and delete files on a remote server
- It uses a **Client-Server** model
  - **Client:** The computer requesting the file (e.g., your PC) → client → ftp | filezilla
  - **Server:** The computer hosting the file → server → vsftpd → 20|21
- Developed in the 1970s, it is one of the oldest protocols still in use today

Mechanics of FTP

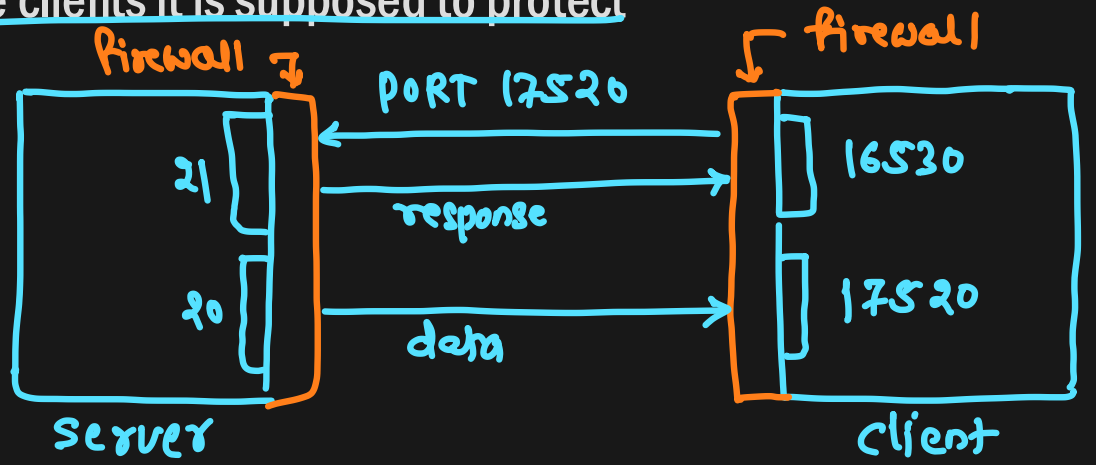
- Although this information isn't crucial to your being able to get an FTP server up and running, it is important when you need to troubleshoot FTP issues—especially issues that don't clearly manifest themselves as being related to FTP
- These details might help you pinpoint issues such as whether the problem is with the network, FTP server, or FTP client
- Client/Server Interactions
  - FTP facilitates the exchange of files between an FTP client and an FTP server, its design has some built-in nuances
  - One of its nuances stems from the fact that it uses two ports: a control port (port 21) and a data port (port 20)
  - The control port serves as a communication channel between the client and the server for the exchange of commands and replies
  - The data port is used purely for the exchange of data, which can be a file, part of a file, or a directory listing
  - FTP can operate in two modes: active FTP mode and passive FTP mode



Active FTP : Server is actively sending data to the client

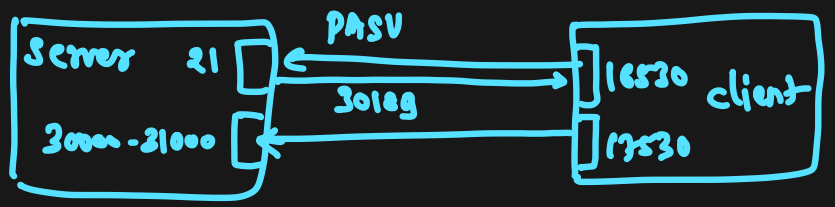
- Active FTP was traditionally used in the original FTP specifications
- In this mode, the client connects from an ephemeral port (a port number greater than 1024) to the FTP server's command port (port 21)
- When the client is ready to transfer data, the server opens a connection from its data port (port 20) to the IP address and ephemeral port combination provided by the client
- The key here is that the client does not make the actual data connection to the server but instead informs the server of its own port by issuing the PORT command; the server then connects back to the specified port
- The server can be regarded as the active party (or the agitator) in this FTP mode
- From the perspective of an FTP client that is behind a firewall, the active FTP mode poses a slight problem: the firewall on the client side might frown upon (or disallow) connections originating or initiated from the Internet from a privileged service port (such as data port 20) to non-privileged service ports on the clients it is supposed to protect

- port → unsigned integer used communicate with the process
  - → assigned to socket for listening incoming request
  - → standard → well known ports
    - → reserved ports → 1 to 1024
    - → non-reserved ports → Mysql (3306), mongo (27017)
  - → Ephemeral ports → non-standard ports
    - → 1024 > ports < 65535



Passive FTP

- The FTP client issues the PASV command to indicate that it wants to access data in passive mode, and the server then responds with an IP address and an ephemeral port number on itself to which the client can connect to transfer the data
- The PASV command issued by the client tells the server to “listen” on a data port that is not its normal data port (that is, port 20) and to wait for a connection rather than initiate one
- The key difference in comparison with active FTP mode is that in passive FTP mode, it is the client that initiates the connection to the port and IP address provided by the server
- In this regard, the server can be considered the passive party in the data communication
- From the perspective of an FTP server that is behind a firewall, passive FTP mode is a little problematic because a firewall’s natural instinct would be to disallow connections that originate from the Internet that are destined for ephemeral ports of the systems that it is supposed to protect
- A typical symptom of this behavior occurs when a client appears to be able to connect to the server without a problem, but the connection seems to hang whenever an attempt to transfer data occurs
- To address some of the issues pertaining to FTP and firewalls, many firewalls implement application-level proxies for FTP that keep track of FTP requests and open up those high ports when needed to receive data from a remote site



Common FTP commands

- **USER / PASS:** Login credentials
- **LS / DIR:** List files in the current directory
- **CD:** Change directory
- **GET:** Download a file from the server
- **PUT:** Upload a file to the server
- **MKDIR:** Create a new directory
- **QUIT:** End the session

Pros and Cons of FTP

- **Advantages**
  - **Efficiency:** Very fast for transferring large batches of files
  - **Simplicity:** Easy to set up and widely supported
  - **Resume:** Supports resuming interrupted transfers
- **Disadvantages**
  - **Security:** Vulnerable to eavesdropping (packet sniffing) in its standard form
  - **Complexity:** Managing multiple ports (Active/Passive) can be difficult with firewalls
  - **Encryption:** Lacks built-in encryption

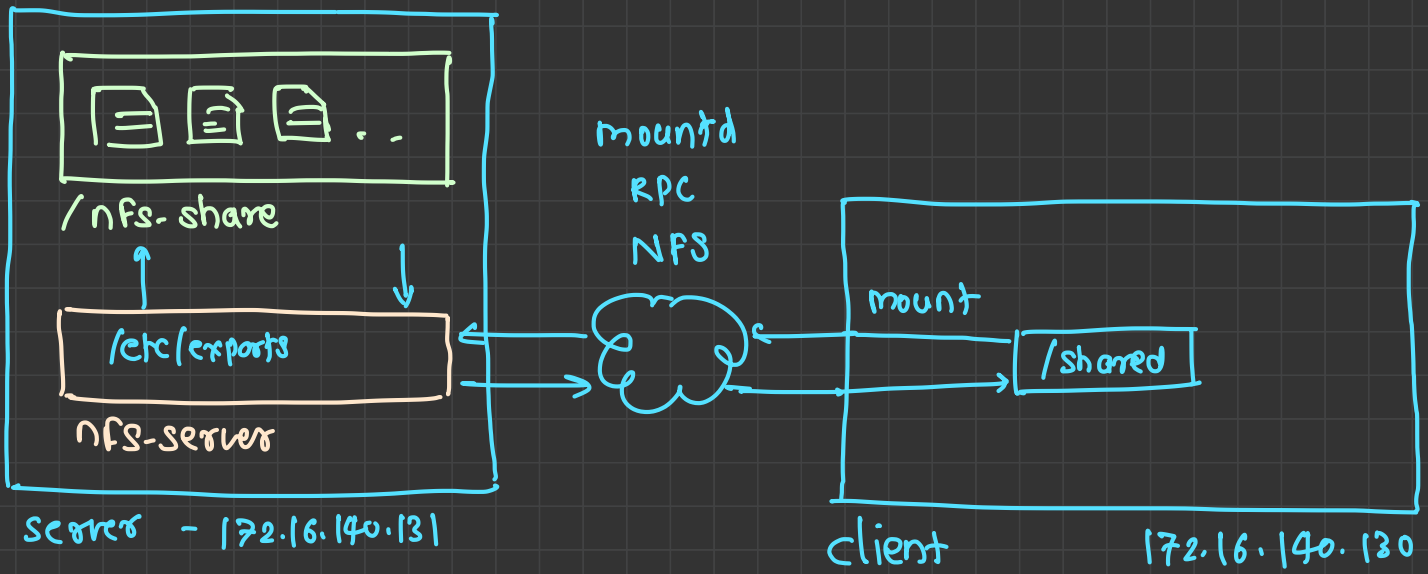
Configuration

- Installation
  - sudo yum install vsftpd ftp
- User creation
  - sudo useradd user1
  - sudo passwd user1
  - sudo vim /etc/vsftpd/chroot_list
- Configuration
  - sudo vim /etc/vsftpd/vsftpd.conf
  - pam_service_name=vsftpd
  - userlist_enable=YES
  - use_localtime=YES
  - chroot_local_user=YES
  - chroot_list_enable=YES
- Start Service
  - sudo systemctl enable vsftpd
  - sudo systemctl start vsftpd
- Add Firewall Rule
  - sudo firewall-cmd --add-service=ftp --permanent
  - sudo firewall-cmd --reload

NFS

NFS → protocol
NFS-utils → package (server + client)
NFS-server → nfs server program

NFS → Network File System
RPC → Remote procedure call
Mountd → mount daemon



NFS

- NFS (Network File System) is a distributed file system protocol that allows a client system to access files over a network as though the files were on local storage
- An NFS server exports directory hierarchies from its local file systems → /etc/exports
- Remote clients with appropriate permissions can mount these directories, and the exported directory appears to the client as if it were a local directory
- Built on a Client-Server model using Remote Procedure Calls (RPC) for communication between the client and server
- Originally developed by Sun Microsystems in the 1980s, NFS has become a standard for file sharing in Unix/Linux environments and is also supported on Windows Server
- Centralized storage provisioning, improved data consistency, and reliability across heterogeneous systems

How NFS Works

FS commands → NFS → RPC → server
ls/mkdir/cd
virtual Node

- **Vnode Interface** : uses RPC to execute the commands on Remote server
  - To implement NFS as an extension of the UNIX file system while minimizing changes required to the local file system, the NFS designers split the file system program by introducing an interface that provides **vnodes** (virtual nodes)
  - This abstraction allows NFS to intercept file system calls and route them over the network transparently.
- **RPC-Based Communication**
  - NFS operations are implemented as Remote Procedure Calls
  - Client makes a file system call (such as read()), and the vnode layer translates it into an RPC to the server
- **File Handles**
  - Each file on the server is identified by **file handle:** an opaque identifier that the server uses to locate the file
  - Clients use file handles to reference files in subsequent operations
- **Stateless Design (V2/V3):**
  - Early NFS versions were designed to be stateless on the server side
  - The server does not maintain information about open files or clients between requests.
  - This design simplifies crash recovery—if the server reboots, clients simply retry their requests

Mechanics of NFS

sudo mount -t <fs type> <device> <mount point>

sudo mount -t nfs 172.16.140.131:/nfs-share /shared

- As with most network-based services, NFS follows the usual client and server paradigms—that is, it has its client-side components and its server-side components
- For clientA to access the /home share being exported by serverA, clientA needs to make an NFS mount request for /home so that it can mount it locally, such that the remote share appears locally as the /home directory
- Here is a simple command to trigger this
  - [master@clientA ~]$ sudo mount serverA:/home /home -t nfs
- After executing the previous command on clientA, all the users on clientA would be able to view the contents of /home as if it were just another directory or local file system
- Linux would take care of making all of the network requests to the server
- Remote procedure calls (RPCs) are responsible for handling the requests between the client and the server
- RPC technology provides a standard mechanism for any RPC client to contact the server and find out to which service the calls should be directed
- Thus, whenever a service wants to make itself available on a server, it needs to register itself with the RPC service manager, portmap, which tells the client where the actual service is located on the server

NFS Versions

- The protocol behind NFS has evolved and changed a lot over the years
- NFSv2
  - Mount requests are granted on a per-host basis and not on a per-user basis
  - This version uses Transmission Control Protocol (TCP) or User Datagram Protocol (UDP) as its transport protocol
  - Version 2 clients have a file size limitation of less than 2GB that they can access
- NFSv3
  - This version includes a lot of fixes for the bugs in NFSv2
  - It has more features than version 2, has performance gains over version 2, and can use either TCP or UDP as its transport protocol
  - Depending on the local file system limits of the NFS server itself, clients can access files larger than 2GB in size
  - Mount requests are also granted on a per-host basis and not on a per-user basis
- NFSv4
  - This version of the protocol uses a stateful protocol such as TCP or Stream Control Transmission Protocol (SCTP) as its transport
  - It has improved security features thanks to its support for Kerberos;
  - It was designed with the Internet in mind. This version of the protocol is firewall-friendly and listens on the well-known port 2049
  - The services of the RPC binding protocols (such as rpc.mountd, rpc.lockd, and rpc.statd) are no longer required in this version of NFS because their functionality has been built into the server; in other words, NFSv4 combines these previously disparate NFS protocols into a single protocol specification. (The portmap service is no longer necessary.)
  - It includes support for file access control list (ACL) attributes and can support both version 2 and version 3 clients

Advntages

- **Transparency:** Remote files appear as local files. Users and applications interact with them using standard file operations (open, read, write, close) without needing to know the files are on a remote server.
- **Centralized Storage Management:** Data is stored and managed in one location. Backups, updates, and security policies are applied centrally, reducing administrative overhead across many client machines.
- **Heterogeneous Platform Support:** NFS works across different operating systems and hardware architectures. A Linux client can access files on a Solaris server; Windows Server can serve NFS shares to Unix clients. This makes it ideal for mixed environments.
- **Cost-Effective:** NFS is built into virtually all Unix/Linux operating systems at no additional cost. No proprietary licensing fees are required, unlike some commercial storage solutions.
- **Scalability (with pNFS):** NFSv4.1's parallel NFS extension separates metadata from data operations, allowing the system to scale horizontally by adding storage devices rather than overloading a single server.
- **Strong Security (NFSv4):** Kerberos integration provides authentication, integrity checking, and encryption. NFSv4 ACLs interoperate with both POSIX and Windows permissions.
- **Mature and Well-Understood:** Decades of deployment mean extensive documentation, community knowledge, and proven reliability in production environments.
- **Diskless Client Support:** NFS enables diskless workstations to boot and operate by mounting their root file system from a remote server, reducing hardware costs and simplifying management.

Disadvantages

- **Weak Consistency Model:** NFS provides **close-to-open consistency** only. If two clients access the same file simultaneously, one may see stale data. This makes NFS **unsuitable for multiclient database applications** that require strict coherence.
- **Single Point of Failure:** In traditional NFS (without pNFS or clustering), the NFS server is a single point of failure. If it goes down, all clients lose access to their mounted file systems.
- **Performance Bottlenecks:** All data flows through the NFS server. Under heavy load, the server's CPU, memory, and network bandwidth become bottlenecks. This limits scalability compared to distributed or parallel file systems.
- **Network Dependency:** NFS requires a reliable, low-latency network. Network congestion, packet loss, or high latency directly impact file access performance. A network outage means complete loss of access to data.
- **Security Complexity:** Traditional AUTH_SYS authentication is insecure—UIDs and GIDs are sent in plain text and can be spoofed. Properly configuring Kerberos requires significant expertise and infrastructure (KDC, keytabs, etc.).

Configuration on Server

- Install nfs-server
  - sudo yum -y install nfs-utils
- Update configuration
  - sudo vim /etc/idmapd.conf
- Add share directories
  - sudo vim /etc/exports
  - sudo exportfs -a
- Start the servers
  - sudo systemctl start nfs-server
  - sudo systemctl start rpcbind
- Add firewall service
  - sudo firewall-cmd --add-service=nfs3 --permanent
  - sudo firewall-cmd --add-service=mountd --permanent
  - Sudo firewall-cmd --add-service=rpc-bind --permanent

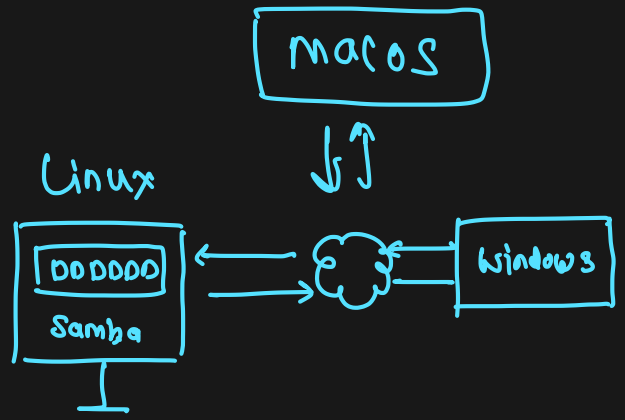
Configuration on Client

- Install nfs-server
  - sudo yum -y install nfs-utils
- Verify the NFS share
  - sudo showmount –e <server ip>
- Mount the nfs volume
  - sudo mount -t nfs <server ip>:<nfsshare> <mount point>

Automount NFS drive

- Create a file at
  - /etc/systemd/system/mnt-nfs-agent.mount
- Add following configuration

```
[Unit]
Description=Mount NFS share for Agentic AI Application
After=network-online.target
Wants=network-online.target

[Mount]
What=192.168.1.100:/export/data
Where=/mnt/nfs-agent
Type=nfs
Options=vers=4.2,rw,hard,timeo=600,nosuid,nodev,_netdev

[Install]
WantedBy=multi-user.target
```



Samba

SMB

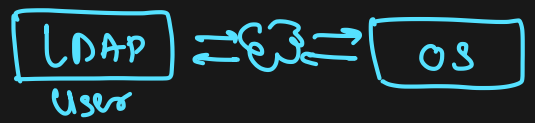
Samba : implementation of SMB protocol

- Samba is a powerful suite of applications that provides an open source implementation of the Server Message Block/ Common Internet File System (SMB/CIFS) protocol
- Among other things, Samba helps Linux-based systems interoperate with Windows-based operating systems
- Samba transparently provides file and print sharing services to Windows clients as well as other networked clients running other operating systems
- It does this through the use of the native Microsoft networking protocols SMB/CIFS
- From a system administrator's point of view, this means you can deploy a Linux-based server and use it to provide file-sharing, authentication, print, and other services to other non-native Linux clients such as Microsoft Windows systems
- Using Samba means that Windows systems can use their native tongue to talk to the Linux server—which means fewer hassles for you and seamless integration for your users

Windows natively supports SMB | CIFS

Linux natively supports NFS

Mechanics of Samba



- Relative to Samba, several options are available for handling username and password issues in heterogeneous environments, including the following:
  - Linux pluggable authentication modules (PAMs)
    - Allow you to authenticate users against a DC
    - This means you still have two user lists—one local and one on the DC—but your users need to keep track of their passwords only on the Windows system
  - Samba as a DC
    - Allows you to keep all your logins and passwords on the Linux system, while all your Windows boxes authenticate with Samba
    - When Samba is used with a Lightweight Directory Access Protocol (LDAP) back-end for this, you will have a scalable and extensible solution
  - Custom script
    - Allows you to use your own custom script
    - For sites with a well-established system for maintaining logins and passwords, it isn't unreasonable to come up with a custom script. This can be done using a scripting language with good cross-platform support.
    - Such scripts can be coaxed to allow changes to the Security Access Manager (SAM) to update the DC's password list

Advantages

- **Cross-Platform Compatibility:** Samba's core value is enabling Linux/Unix servers to seamlessly integrate into Windows-dominated networks. Windows clients require **no additional software**—they access Samba shares directly through Windows Explorer .
- **Zero Licensing Cost:** As open-source software under GPL, Samba eliminates the licensing fees associated with Windows Server and commercial file-sharing solutions. Deployment is as simple as apt install samba or yum install samba .
- **Active Directory Integration:** Samba 4 can function as a full AD Domain Controller, providing centralized authentication, Kerberos, LDAP, and DNS. This allows organizations to replace Windows Server DCs with Linux-based infrastructure .
- **Flexible Permission Control:** Samba supports both POSIX permissions and Windows ACLs, with configurable mapping and inheritance. Administrators can define fine-grained access rules per share, user, or group .
- **Printer Sharing:** Samba integrates with CUPS to provide network printer sharing to Windows clients, making it a complete file-and-print server solution .
- **Rich Protocol Support:** Samba 4.x supports SMB3 with encryption, multichannel, directory leasing, and SMB Direct (RDMA) for high-performance scenarios

Disadvantages

- **Performance Bottlenecks:** In high-concurrency scenarios (200+ simultaneous connections), Samba's latency can be approximately **30% higher than NFSv4.2**. Small-file operations are particularly affected. Testing shows Samba's IOPS is about 30% lower than NFS in 100-user concurrent scenarios
- **Security Configuration Complexity:** While Samba supports SMB3 encryption, default configurations may be insecure. Legacy versions use weak encryption (DES). Misconfigured anonymous shares (map to guest = Bad User) can lead to data leaks if not combined with IP restrictions
- **Historical Vulnerabilities:** Samba has had high-risk vulnerabilities such as **CVE-2017-7494 ("EternalRed")** . Regular updates to the latest stable version are essential for security
- **Limited Advanced Features:**
  - Compared to Microsoft's implementation, Samba lacks:
    - Full distributed file system (DFS) implementation
    - Policy-based file screening (Windows FSRM equivalent)
    - Native Windows domain controller role in older versions

Configuration

- Install server
  - sudo yum install samba
- Configure share
  - sudo vim /etc/samba/smb.conf

```
[share]
   comment = My Share
   path = /var/smb/share
   writeable = yes
   public = yes
   browseable = yes
   create mask = 0644
   directory mask = 0755
   write list = user
```

- Add samba user
  - sudo smbpasswd -a <username>
- Start server
  - sudo systemctl enable smb
  - sudo systemctl start smb
- Add firewall rules
  - sudo firewall-cmd --add-service=samba --permanent
  - sudo firewall-cmd --reload

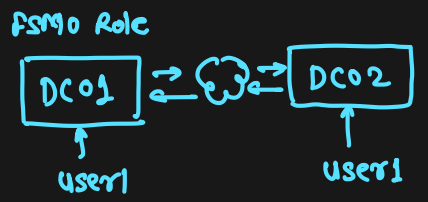
Comparison

Feature	FTP	NFS	Samba (SMB/CIFS)
Storage Model	**Application/File**	**File-level** (Network File System)	**File-level** (Server Message Block)
Primary Use Case	Explicit file uploads/downloads; website maintenance; bulk data transfer	Transparent file sharing between Unix/Linux systems; HPC clusters	Seamless file and printer sharing in Windows/mixed-OS environments
Access Type	Discrete transfers (PUT/GET); files are edited locally then re-uploaded	Transparent mount; remote directories appear as local folders	Transparent mount; remote shares appear as network drives/folders
Protocol Level	Application Layer (TCP ports 20/21)	Kernel-level (RPC over TCP/UDP)	Application/Session Layer (TCP 445/139)
Cross-Platform	Excellent (Universal)	Primarily Linux/Unix (requires tools for Windows)	Excellent (Windows-native, strong macOS/Linux support)
Multi-User Concurrent Access	Yes (discrete transfers)	**Yes** (Designed for shared file access)	**Yes** (Designed for shared file access)
Security	**Plaintext by default** (FTPS/SFTP for encryption)	Kerberos (v4); AUTH_SYS (v3)	**Kerberos, SMB Encryption, Signing**
Performance Profile	Good for large files; overhead from explicit client interaction	Efficient for small files on Linux; low overhead	Good for mixed OS; tuned for large sequential I/O; higher CPU load

FSMO Roles

Introduction

- FSMO (Flexible Single Master Operations) roles are specialized tasks assigned to specific Domain Controllers (DCs) in Active Directory
- While AD uses a **multi-master replication model** by default, certain operations cannot tolerate conflicts from simultaneous updates across multiple DCs
- FSMO roles solve this by **designating a single DC as the authority** for these specific operations
- **FSMO roles are the exceptions to multi-master replication—operations that must be handled by one DC at a time**
- **Why FSMO roles matter?**
  - **Conflict Prevention**
    - Operations like schema modifications and RID allocation, if performed simultaneously on multiple DCs, would create inconsistencies
    - FSMO enforces single-point authority
  - **Critical Functionality**
    - The PDC Emulator directly affects password authentication and time sync
    - The RID Master affects the ability to create new objects
  - **Security Value**
    - CISA recommends **distributing FSMO roles** across different DCs to increase the difficulty for an attacker to fully compromise the domain—they would need to compromise multiple DCs to seize all roles, giving defenders more detection opportunities



FMSO Roles

- **Forest-Level Roles (One per Forest)**
  - Schema Master
    - Controls all updates and modifications to the Active Directory schema
    - Required when extending the schema (e.g., installing Exchange, running adprep /forestprep)
  - Domain Naming Master
    - Controls the addition and removal of domains and application partitions in the forest
    - Ensures domain name uniqueness
- **Domain-Level Roles (One per Domain)**
  - PDC Emulator
    - Receives priority replication of password changes; processes account lockouts; preferred DC for Group Policy and DFS management; authoritative time source for the domain
  - RID Master
    - Allocates RID (Relative ID) pools to DCs, ensuring new security principals (users, groups, computers) receive unique SIDs
  - Infrastructure Master
    - Maintains cross-domain object references (e.g., when a user from one domain is a member of a group in another domain)
    - Note
      - In environments where all DCs are Global Catalogs, or AD Recycle Bin is enabled, this role has no functional work
      - It should still be assigned to a valid DC to avoid monitoring errors

Pros and Cons

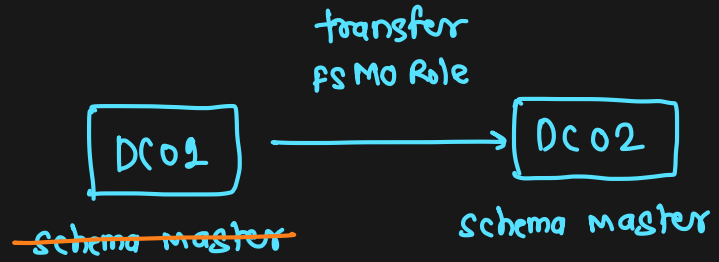
- **Pros**
  - **Conflict Avoidance:** Single-master model ensures uniqueness for critical operations
  - **Clear Accountability:** Administrators know which DC to contact for role-specific operations
  - **Scalability Support:** Enables large distributed environments by isolating operations that don't suit multi-master replication
- **Cons / Risks**
  - **Single Point of Failure:** If a role holder goes offline, role-dependent operations fail
  - **PDC Emulator Load Concentration:** Password changes, time sync, and GPO operations converge on one DC, requiring high-performance hardware
  - **Placement Complexity:** Improper role placement in multi-domain/multi-site environments causes performance issues

Best Practices

- **Role Placement Guidelines**
  - **Configure a standby operations master** for forest-level roles
  - **Configure a standby operations master** for domain-level roles
  - **Place domain-level roles on a high-performance DC**
  - **Leave forest-level roles on a DC in the forest root domain**
  - **In the forest root domain, transfer the three domain-level roles to another high-performance DC**
  - **In all other domains, leave the domain-level roles on the first DC**
- **PDC Emulator Specifics**
  - **Highest workload** of all FSMO roles; requires the most reliable DC
  - The forest root domain's PDC Emulator should synchronize with an **external time source**
  - Ensure 24/7 availability
- **Infrastructure Master Specifics**
  - **Do not** place IM on a Global Catalog server (unless all DCs are GCs)
  - **Exception**: If all DCs are GCs, or AD Recycle Bin is enabled, IM has no functional work, but should still be assigned to a valid DC
- **Security Considerations**
  - **Distribute FSMO roles** across different DCs to increase attacker difficulty
  - Monitor for netdom query fsmo usage, which may indicate attackers enumerating role holders

Transferring FSMO Roles

- Transferring a FSMO role means **moving the responsibility of a specific single-master operation from one Domain Controller (DC) to another DC**
- Example
  - Your company has one **authorized signatory** who can approve payments (this is the FSMO role holder)
  - When that person goes on leave or leaves the company, you **formally hand over** that signing authority to another person
  - After the handover, only the new person can approve payments—the old person no longer has that authority
- In Active Directory terms
  - **One DC** currently holds the role (e.g., PDC Emulator)
  - You **move that role** to **another DC**
  - After the transfer, the **new DC** handles those responsibilities
  - The **old DC** stops performing that role



Transfer process

- **Step 1: Pre-checks**
  - The new role holder (DC2) contacts the current holder (DC1) to request the role
  - The current holder verifies the request is legitimate
- **Step 2: Role Handover**
  - DC1 **releases** the role and records that it no longer holds it
  - DC2 **accepts** the role and records itself as the new holder
  - This change is **written to Active Directory** (specifically to the FSMO-Role-Ownership attribute on the DC objects)
- **Step 3: Replication**
  - The updated role ownership information **replicates** to all other DCs in the domain/forest
  - Until replication completes, some DCs may still think DC1 holds the role (temporary inconsistency)
- **Step 4: DC2 Begins Work**
  - **Important**: DC2 does **not** immediately start performing the role
  - It **waits for inbound replication** of the role's naming context to complete
  - This ensures DC2 has the most up-to-date data before taking over
  - Only after replication completes does DC2 actively perform the role's duties

What Changes After Transfer

Aspect	Before Transfer	After Transfer
Password changes	DC1 (old PDC) receives priority	DC2 (new PDC) receives priority
Time sync source	DC1 serves as authoritative time	DC2 serves as authoritative time
GPO management	DC1 is preferred DC	DC2 is preferred DC
RID pool allocation	DC1 distributes RID pools	DC1 still does this (if RID Master wasn't transferred)
Role ownership in AD	DC1 object holds the role	DC2 object holds the role

Why To Transfer?

- **DC Retirement**
  - You're decommissioning an old DC (e.g., Windows Server 2012 R2)
  - Before shutting it down, you must transfer its FSMO roles to another DC—otherwise, those operations will fail
- **Hardware Upgrade**
  - You're replacing an old physical server with a new, more powerful one
  - Transfer the PDC Emulator (the heaviest role) to the new hardware for better performance
- **Planned Maintenance**
  - You need to take a DC offline for repairs or updates
  - Transfer its roles to another DC first to avoid service disruption
- **Performance Optimization**
  - A DC holding the PDC Emulator is overloaded (slow logons, password sync delays)
  - Transfer the role to a less busy DC.
- **Site/Location Changes**
  - You're restructuring your network
  - Move FSMO roles to DCs in the appropriate sites to reduce replication latency.
- **Security Hardening**
  - Per CISA recommendations, distribute FSMO roles across multiple DCs so an attacker can't compromise the entire domain by taking down one DC

Hands-on

- Check the current FSMO Roles

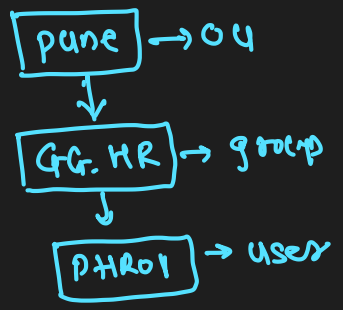
```
netdom query fsmo
```

- PowerShell Method

```
# Forest-level roles (Schema Master, Domain Naming Master)
Get-ADForest | Select-Object SchemaMaster, DomainNamingMaster

# Domain-level roles (PDC Emulator, RID Master, Infrastructure Master)
Get-ADDomain | Select-Object PDCEmulator, RIDMaster, InfrastructureMaster
```

- Transfer FSMO Roles

```
# Transfer Schema Master
Move-ADDirectoryServerOperationMasterRole -Identity "<TargetDC>" -OperationMasterRole SchemaMaster

# Transfer PDC Emulator
Move-ADDirectoryServerOperationMasterRole -Identity "<TargetDC>" -OperationMasterRole PDCEmulator
```



Group Policies

Used to add restrictions on objects (users | groups | computers)

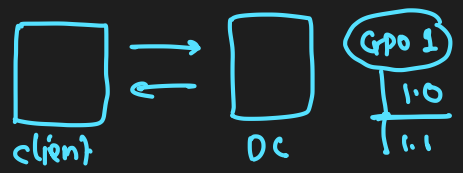
Hardening operating system

- CPO
- Security
- installation / application

Introduction

- Group Policy is the built-in mechanism in Active Directory Domain Services (AD DS) for centrally configuring and enforcing settings on domain-joined Windows computers and the users who log on to them
  - → restrictions
- One administrator can change a setting once and have it applied to thousands of machines automatically
- **Group Policy controls**
  - **Security:** password rules, account lockout, user rights, audit, firewall, BitLocker, Credential Guard, LAPS
  - **Desktop and user experience:** Start menu, Control Panel access, wallpaper, drive maps, printers, shortcuts
  - **Software:** application install (MSI), AppLocker / App Control for Business (WDAC), Windows Update for Business settings → WSUS server
  - **Scripts:** startup/shutdown (computer) and logon/logoff (user) scripts
  - **Data:** folder redirection, offline files, roaming profile behaviour
- **Importance of Group Policies**
  - **Consistency:** every machine in an OU gets the same baseline, which removes configuration drift
  - **Security hardening:** CIS / Microsoft Security Baselines are delivered as GPOs
  - **Delegation:** OU-level GPOs let junior admins manage their own scope without Domain Admin rights
  - **Auditability:** GPMC reports show exactly which settings apply and wheres

Core concepts



- A Group Policy Object (GPO) is a named container of settings
- It does nothing until it is **linked** to a site, domain or OU; one GPO can be linked to many places, and one container can have many GPOs linked
- **The two halves of every GPO**

  CN = Policies, CN=System, DC=sunbeam, DC = local

  - Group Policy Container (GPC)
    - Stored in: AD database: CN={GUID},CN=Policies,CN=System,DC=domain,DC=com
    - Contains: Version number, status, WMI filter link, CSE GUIDs, permissions
  - Group Policy Template (GPT)
    - Stored in: SYSVOL \\domain\SYSVOL\domain\Policies\{GUID}
    - Contains: GPT.INI, Machine\ and User\ folders, Registry.pol, scripts, security template GptTmpl.inf, Preferences XML
- The GUID links the two halves
- If the GPC and GPT version numbers differ on a DC (replication lag), clients may skip or partially apply the GPO

Types of GPOs

- **Local GPOs** → client | computer
  - Local GPOs apply only to the specific computer on which they reside and the users who log on to that computer
  - Every Windows computer has one built-in local GPO, which is managed locally using the Local Group Policy Editor
  - In an Active Directory environment, local GPOs are the least influential and have the lowest precedence in processing order (Local > Site > Domain > OU)
- **Non-local (Domain-based) GPOs**
  - Non-local GPOs are the primary mechanism for Group Policy in a domain environment
  - They are stored in Active Directory and replicate to all domain controllers
  - These GPOs do not take effect until they are **linked** to an Active Directory container, such as a site, domain, or organizational unit (OU)
  - You can link a single GPO to multiple containers, and a single container can have multiple GPOs linked to it
- **Starter GPOs**
  - Starter GPOs are templates that contain a pre-configured set of policy settings, acting as a baseline for creating new policies
  - They are designed to save administrators time by providing a standardized starting point for common configurations

Default GPOs : created by active directory

- **Default Domain Policy**
  - Linked to The domain itself
  - Applies to **all users and computers** in the domain.
  - It is pre-configured with critical **Password Policy, Account Lockout Policy, and Kerberos Policy** settings
  - Microsoft strongly recommends not adding unrelated settings here
- **Default Domain Controllers Policy**
  - The **Domain Controllers** OU
  - Applies **only to domain controllers** (or other objects placed in that OU)
  - It is used to manage **Audit Policy and User Rights Assignment** specifically for your DCs
- **Key Points**
  - Precedence: The Default Domain Policy has the highest precedence among all GPOs linked to the domain
  - Best Practice: Keep the Default Domain Policy reserved for its core account policies. Create separate, additional GPOs for any other settings you need to apply broadly across the domain
  - Don't Move DCs: Moving a domain controller out of the Domain Controllers OU will stop the Default Domain Controllers Policy from applying, which can cause inconsistent behavior

Processing order (LSDOU)

- It determines which settings win when multiple GPOs configure the same setting
- Settings are processed in this order, from lowest to highest precedence
- Local
  - Scope: The computer itself
  - The built-in local GPO
  - Lowest precedence; easily overridden
- Site
  - Scope: Active Directory site
  - GPOs linked to the site object. Rarely used, but useful for site-specific settings (e.g., regional printers)
- Domain
  - Scope: The entire domain
  - GPOs linked to the domain root, including the Default Domain Policy
- OU
  - Scope: Organizational Units
  - GPOs linked to OUs, processed from the parent OU down to the child OU. The deepest, most specific OU has the highest precedence

Configuration – Computer Configuration

- **Computer Configuration** contains settings that apply to the **computer itself**, regardless of who logs on
- These settings are processed when the operating system starts, during background refresh, and roughly every 90–120 minutes thereafter
- **What lives here**
  - **Software Settings:** Software installation packages assigned to the computer (installs at startup)
  - **Windows Settings:** Security settings (password policy, audit policy, user rights), scripts (startup/shutdown), and network configuration
  - **Administrative Templates:** Registry-based policies that govern system behavior, such as disabling USB storage or restricting Control Panel access
- **Practical example:** If you enable "Deny access to Control Panel" under Computer Configuration, no user who logs on to that machine can access it, even administrators

Configuration - User Configuration

- **User Configuration** contains settings that apply to the **user account**, regardless of which computer they log on to
- These settings are processed at user logon and during background refresh.
- **What lives here**
  - **Software Settings:** Software assigned or published to users
  - **Windows Settings:** Folder Redirection (Desktop, Documents to network shares), logon/logoff scripts, and Internet Explorer maintenance
  - **Administrative Templates:** Policies that shape the user's environment, such as Start menu layout, desktop restrictions, and mapped drives
- **Practical example:** If you enable "Hide Internet Explorer icon on desktop" under User Configuration, that user will not see the icon on any computer they log on to

Processing behaviour and refresh

- Computer policy applies at startup and user policy at logon (foreground); both then refresh in the background every 90 minutes with a random offset of 0–30 minutes
- Domain controllers refresh every 5 minutes
- **Foreground**
  - Computer startup, user logon
  - Software Installation, Folder Redirection, some Drive Maps
  - Asynchronous (Fast Logon Optimization)
- **Background**
  - Periodic refresh, gpupdate
  - Asynchronous
- **Forcing an update** → must be executed on the machine on which the policy need to be applied
  - gpupdate: Applies changed policy (computer + user)
  - gpupdate /force: Re-applies all settings, changed or not
  - gpupdate /target: computer Computer settings only
  - gpupdate /force /boot or /logoff: Reboots/logs off if a foreground-only setting needs it

↳ logoff and login

Common Tasks

Password Policy

- It defines password complexity, lifetime, and history requirements for all user accounts in the domain
- It is a foundational component of your Active Directory security baseline
- Password Policy is found under
  - Computer Configuration > Policies > Windows Settings > Security Settings > Account Policies > Password Policy
- **Critical Scope Rule**
  - Only a GPO linked at the **domain root level** (typically the **Default Domain Policy**) can enforce password settings on **domain user accounts**
  - If you configure Password Policy in an OU-linked GPO, it will only affect the **local accounts** on computers within that OU, not domain users

Password Policy - Core Settings

Setting	Purpose	Recommended
Enforce password history	Prevents users from reusing recent passwords. Stores a specified number of previous passwords.	24 (maximum)
Maximum password age	Number of days before a password expires and must be changed.	60–90 days
Minimum password age	Number of days a user must wait before changing a password again. Prevents bypassing history by rapidly cycling passwords.	1 day
Minimum password length	Minimum number of characters required.	8–14 characters
Password must meet complexity requirements	Requires at least three of four character categories: uppercase, lowercase, digits, special characters.	Enabled
Store passwords using reversible encryption	Stores passwords in a reversible format. Only needed for specific legacy authentication scenarios.	Disabled
Minimum password length audit	(Windows Server 2016+) Logs events when passwords shorter than a target length are set, allowing you to measure impact before enforcing a stricter length.	Use as needed

Fine-Grained Password Policies (FGPP)

- Fine-Grained Password Policies (FGPP) allow you to apply stricter password requirements to specific users or groups, such as Domain Admins, without affecting standard user accounts
- **Step 1: Open ADAC and Navigate to the Container**
  - Open **Active Directory Administrative Center**
  - In the left navigation pane, expand your domain, open the **System** container, and select **Password Settings Container**
- **Step 2: Create the Password Settings Object (PSO)**
  - In the **Tasks** pane, click **New** and select **Password Settings**
  - **Name**: Enter a descriptive name (e.g., Domain Admin Password Policy)
  - **Precedence**: Enter a number (e.g., 1). Lower numbers mean higher priority
  - **Settings**: Configure the desired password length, history, and complexity. For privileged accounts, a minimum length of **16** or **25** characters is recommended
- **Step 3: Apply the Policy to a Group**
  - Under **Directly Applies To**, click **Add**
  - Type the name of the **Global Security Group** (e.g., Domain Admins), click **Check Names** to validate, and click **OK** . Click **OK** again to commit the creation

Account Lockout Policy

- **Account Lockout Policy** mitigates brute-force password-guessing attacks by disabling a user account after a defined number of failed sign-in attempts
- **Account Lockout Policy is found under**
  - Computer Configuration > Policies > Windows Settings > Security Settings > Account Policies > Account Lockout Policy
- **Critical Scope Rule**
  - For domain accounts, this policy must be configured in a GPO linked at the **domain root level** (typically the **Default Domain Policy**)
  - If configured in an OU-linked GPO, it applies only to the **local SAM accounts** of computers in that OU, not domain users

Setting	Purpose	Recommended
Account lockout threshold	Number of failed sign-in attempts that triggers a lockout. Setting to 0 disables lockout entirely	5–10 attempts
Account lockout duration	Minutes the account remains locked. Setting to 0 requires an administrator to manually unlock	15–30 minutes
Reset account lockout counter after	Minutes of no failed attempts before the failure count resets to zero	15–30 minutes
Allow Administrator account lockout	Extends lockout to the built-in Administrator account for network sign-ins. Enabled by default on current installations	Enabled

Kerberos Policy

- **Kerberos Policy** controls ticket lifetimes and clock synchronization tolerance for the Kerberos V5 authentication protocol in a domain environment
- It is part of the Active Directory default security baseline and typically **does not need to be modified**, as the default values balance security and usability
- **Kerberos Policy is located at**
  - Computer Configuration > Policies > Windows Settings > Security Settings > Account Policies > Kerberos Policy
- **Critical Scope Rule**
  - Like Password Policy and Account Lockout Policy, Kerberos Policy **must be linked at the domain root level** (typically the **Default Domain Policy**) to affect domain user accounts.
  - It can only be set on **domain controllers**

Audit Policy

- **Audit Policy** turns security policy into visibility
- Without it, you cannot tell who changed a privileged group, who accessed a sensitive share, or which account is being brute-forced
- Audit Policy is found under
  - Computer Configuration > Policies > Windows Settings > Security Settings > Advanced Audit Policy Configuration
- **Critical Scope Rule**
  - Use **Advanced Audit Policy Configuration** (61 subcategories) instead of the legacy **Basic Audit Policy** (9 broad categories)
  - Enable the security option **"Audit: Force audit policy subcategory settings (Windows Vista or later)"** to make advanced settings authoritative. Without this option, results are unpredictable

Administrator Account

- Disabling or renaming the built-in Administrator account is a common hardening step
- For the local built-in Administrator on domain-joined workstations and member servers, the recommendation is different: **disable and rename it**
- Disable
  - Computer Configuration > Policies > Windows Settings > Security Settings > Local Policies > Security Options > Accounts: Administrator account status ⟶ Set to Disabled.
- Rename
  - Computer Configuration > Policies > Windows Settings > Security Settings > Local Policies > Security Options > Accounts: Rename administrator account ⟶ Enter a name that does not suggest administrative access
- **Create and edit a new GPO**
  - Computer Configuration > Policies > Windows Settings > Security Settings > Local Policies > User Rights Assignment
  - **Deny access to this computer from the network**
  - **Deny log on as a batch job**
  - **Deny log on as a service**
  - **Deny log on through Remote Desktop Services**

Removable Storage Restrictions

- Removable Storage / USB Restrictions in Group Policy allow you to block unauthorized USB drives to prevent data exfiltration and malware introduction
- Create and edit a new GPO
  - Computer Configuration > Policies > Administrative Templates > System > Removable Storage Access
- Enable the following policies
  - All Removable Storage classes: Deny all access
  - Removable Disks: Deny read access
  - Removable Disks: Deny write access
  - Removable Disks: Deny execute access

Control Panel Restrictions

- **Control Panel Restrictions** in Group Policy allow you to block access to the Control Panel and Windows Settings entirely, or hide only specific items you do not want users to access
- Because these are **User Configuration** policies, they apply to the user account regardless of which computer they sign in to
- Create and edit a GPO
  - User Configuration > Policies > Administrative Templates > Control Panel
- **Option 1: Block Everything**
  - It prevents users from launching the Control Panel and the modern Settings app
  - **Policy to Configure: "Prohibit access to Control Panel and PC settings"**
  - Prevents Control.exe and SystemSettings.exe from starting .
  - Removes the Control Panel from the Start screen and File Explorer .
  - If a user tries to open a Control Panel item from a context menu, an error message appears explaining that a setting prevents the action .
  - An error appears when clicking the Settings icon in the Start menu
  - **Limitation:** Users can still access configuration tools using Run, CMD, or Control.exe directly. To fully enforce this restriction, you need to combine it with **AppLocker** or **Software Restriction Policies**

Control Panel Restrictions

- **Option 2: Hide Specific Control Panel Items**
  - **Policy to Configure: "Hide specified Control Panel items"**
  - Network and Sharing Center: Microsoft.NetworkAndSharingCenter
  - Windows Update: Microsoft.WindowsUpdate
  - User Accounts: Microsoft.UserAccounts
  - Programs and Features: Microsoft.ProgramsAndFeatures
  - Windows Defender Firewall: Microsoft.WindowsDefender
  - Device Manager: Microsoft.DeviceManager
  - System: Microsoft.System
  - Administrative Tools: Microsoft.AdministrativeTools

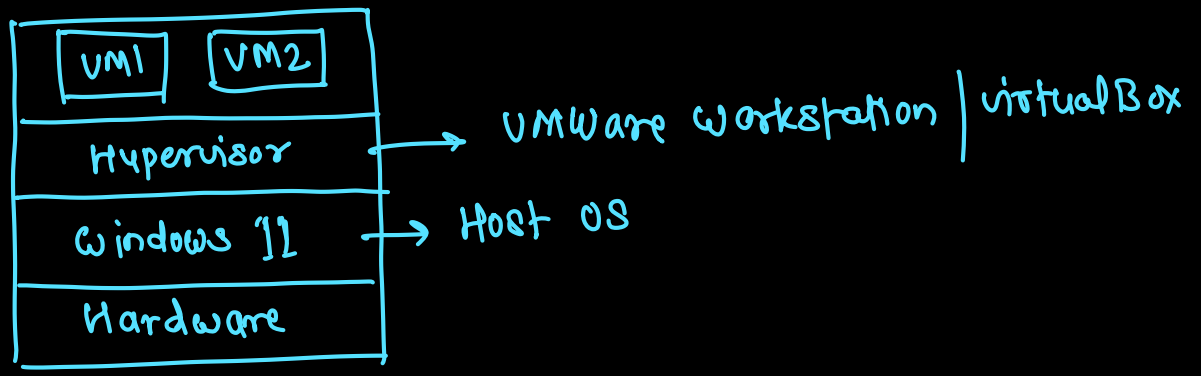
Desktop Restrictions

- Desktop Restrictions in Group Policy allow you to control what users see on their desktop and prevent them from making unwanted changes
- These are **User Configuration** policies, so they follow the user account regardless of which computer they sign in to
- Create and edit a GPO
  - User Configuration > Policies > Administrative Templates > Desktop
- Policy to configure
  - Hide and disable all items on the desktop
  - Remove Recycle Bin icon from desktop
  - Remove Computer icon on the desktop
  - Hide Internet Explorer icon on desktop
  - Remove Properties from the Recycle Bin context menu
  - Remove Properties from the Computer icon context menu
  - Remove Desktop Cleanup Wizard

Hyper-V

What is virtualization

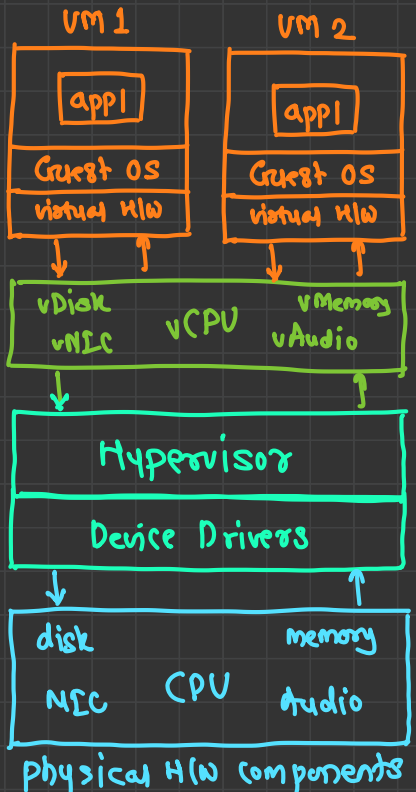
- Virtualization is the creation of a virtual -- rather than actual -- version of something, such as an operating system (OS), a server, a storage device or network resources
- Virtualization uses software that simulates hardware functionality in order to create a virtual system → Hypervisor
- This practice allows IT organizations to operate multiple operating systems, more than one virtual system and various applications on a single server
- Types
  - Network virtualization
  - Storage virtualization → LVM | iSCSI
  - Data virtualization
  - Desktop virtualization
  - Application virtualization
  - Hardware virtualization
  - Operating System virtualization
    - → containerization



Hardware Virtualization

- Hardware virtualization is the process of creating a **virtual version of a physical computing environment** using a layer of software called a **hypervisor**
- Instead of running directly on physical hardware, multiple **virtual machines (VMs)** can share the same physical resources (CPU, memory, storage, network). Each VM behaves like an independent computer with its own operating system and applications
- **Hypervisor (Virtual Machine Monitor – VMM)**
  - The software layer that enables virtualization.
  - Types:
    - Type 1 (Bare-metal): Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V, Xen)
    - Type 2 (Hosted): Runs on top of a host OS (e.g., VirtualBox, VMware Workstation)
- **Virtual Machine (VM)**
  - A software-based emulation of a physical computer
  - Each VM has its own virtual CPU, memory, disk, and network interface
- **Hardware Support**
  - Modern CPUs (Intel VT-x, AMD-V) and hardware (IOMMU, SR-IOV for networking) include extensions to make virtualization efficient and secure

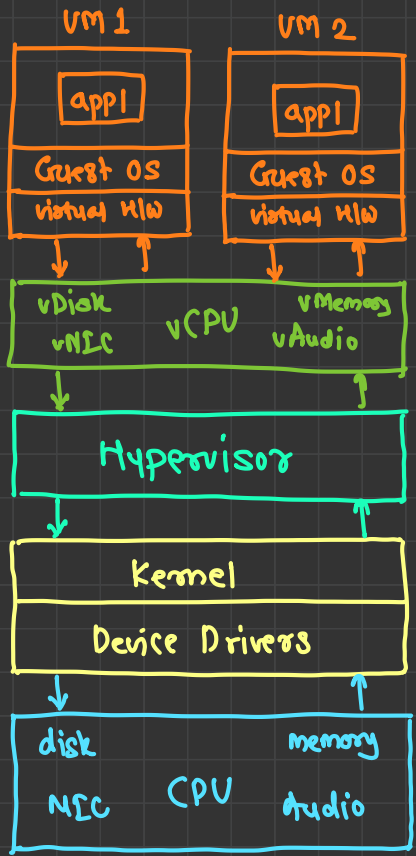
Type I → Bare Metal



Virtual HW components
virtual appliances

Hyper-V, KVM, Xen.
VMWare ESXi

Type II - Hosted



Virtual HW components
virtual appliances

VMWare Workstation,
Virtual box, Parallels, UTM

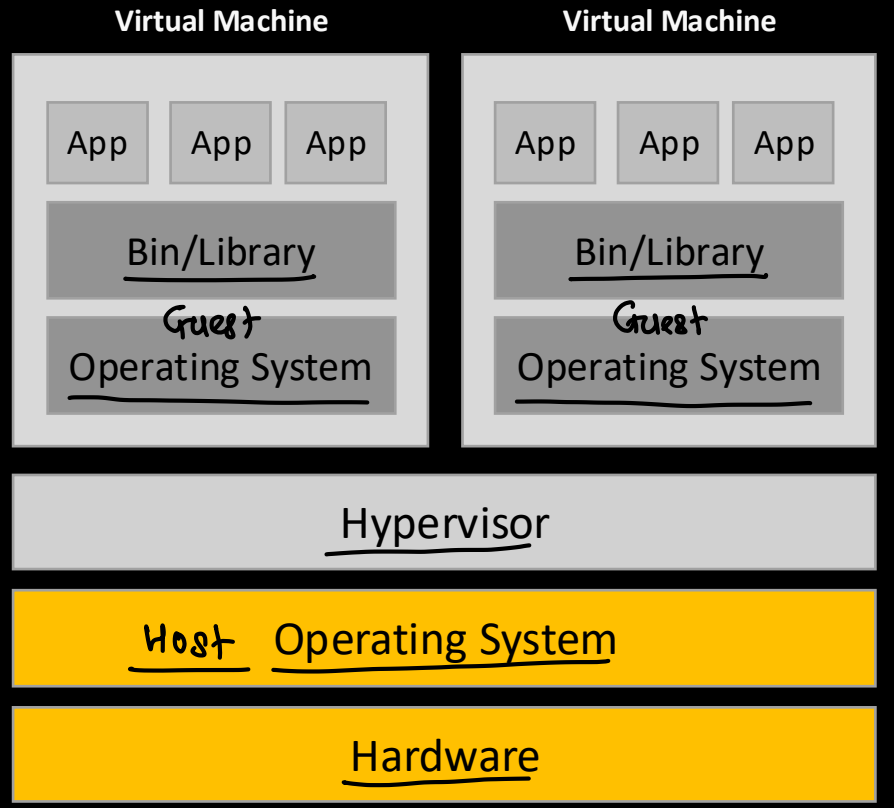
Host Operating System

Types of hardware virtualization

- Type I → used by cloud providers
  - A Type 1 hypervisor runs directly on the host machine's physical hardware, and it's referred to as a bare-metal hypervisor
  - It doesn't have to load an underlying OS first
  - With direct access to the underlying hardware and no other software, it is more efficient and provides better performance
  - It is best suited for enterprise computing or data centers
  - E.g. VMware ESXi, Microsoft Hyper-V server and open source KVM
- Type II → used by developers | testers | End users
  - A Type 2 hypervisor is typically installed on top of an existing OS, and it's called a hosted hypervisor → Host OS
  - It relies on the host machine's pre-existing OS to manage calls to CPU, memory, storage and network resources
  - E.g. VMware Fusion, Oracle VM VirtualBox, Oracle VM Server for x86, Oracle Solaris Zones, Parallels and VMware Workstation

Virtualized Deployment

- It allows you to run multiple Virtual Machines (VMs) on a single physical server's CPU
- Virtualization allows applications to be isolated between VMs and provides a level of security as the information of one application cannot be freely accessed by another application
- Virtualization allows better utilization of resources in a physical server and allows better scalability because
  - an application can be added or updated easily
  - reduces hardware costs
- With virtualization you can present a set of physical resources as a cluster of disposable virtual machines
- Each VM is a full machine running all the components, including its own operating system, on top of the virtualized hardware



Advantages of virtualization

- **Lower costs**
  - Virtualization reduces the amount of hardware servers necessary within a company and data center
  - This lowers the overall cost of buying and maintaining large amounts of hardware
- **Easier disaster recovery**
  - Disaster recovery is very simple in a virtualized environment
  - Regular snapshots provide up-to-date data, allowing virtual machines to be feasibly backed up and recovered
  - Even in an emergency, a virtual machine can be migrated to a new location within minutes
- **Easier testing**
  - Testing is less complicated in a virtual environment
  - Even if a large mistake is made, the test does not need to stop and go back to the beginning
  - It can simply return to the previous snapshot and proceed with the test
- **Quicker backups**
  - Backups can be taken of both the virtual server and the virtual machine
  - Automatic snapshots are taken throughout the day to guarantee that all data is up-to-date
  - Furthermore, the virtual machines can be easily migrated between each other and efficiently redeployed
- **Improved productivity**
  - Fewer physical resources results in less time spent managing and maintaining the servers
  - Tasks that can take days or weeks in a physical environment can be done in minutes
  - This allows staff members to spend majority of their time on more productive tasks, like raising revenue and fostering business initiatives

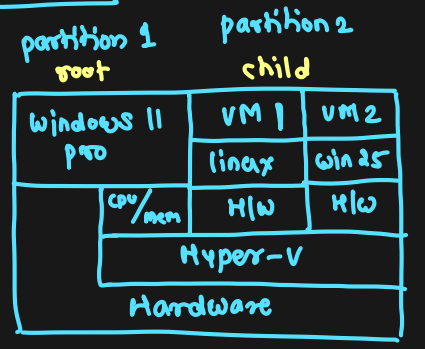
Hyper-V

Introduction

- Hyper-V is Microsoft's hardware virtualization technology for x64 versions of Windows
- It allows multiple isolated operating systems to run simultaneously on a single physical hardware platform
- Hyper-V is a **Type 1 (bare-metal) hypervisor**, meaning it runs directly on the hardware rather than on top of a host operating system
- **System Requirements**
  - Before deploying Hyper-V, verify the following hardware requirements:
  - 64-bit processor with **SLAT (Second Level Address Translation)**
  - Processor supporting hardware-assisted virtualization (**Intel VT** or **AMD-V**)
  - Processor supporting **VM Monitor Mode extensions**
  - Enough memory for both the host and guest VMs
  - Minimum 4 GB RAM (8 GB or more recommended)

Core Architecture

- Hyper-V architecture is built around the concept of **partitions**
- A partition is a **logical unit of isolation**, supported by the hypervisor, in which operating systems execute
- **Root (Parent) Partition**
  - The root partition is the first partition created when the hypervisor starts
  - It runs the Windows operating system
  - It is responsible for:
    - Running the virtualization management stack and having **direct access to hardware devices**
    - Creating child partitions via the **Hypercall API**
    - Managing hardware devices (except **processor scheduling** and **physical memory allocation**, which are handled by the hypervisor)
    - Handling power management, Plug and Play operations, and recording hardware failure events
- **Child Partition**
  - Child partitions ~~host~~ guest operating systems
  - A child partition does **not have direct access to physical hardware resources**; instead, it receives a virtual view of the resources
  - Requests for virtual devices are redirected to devices in the root partition for processing via the **VMBus** or the hypervisor



Technical Features

- **Live Migration:** Allows a running VM to be moved from one physical host to another with **no downtime or disruption of connectivity**. Supports simultaneous migration, compression, and SMB transport options
- **Hyper-V Replica:** Asynchronously replicates VMs from a primary site to a replica site over a standard IP network, with configurable replication frequencies (e.g., 5, 10, 15 minutes). In a disaster, it enables planned or unplanned failover
- **Checkpoints:** Captures the **state, data, and hardware configuration** of a running VM at a specific point in time, allowing the VM to be easily reverted to a previous state. Supports standard checkpoints and production checkpoints (which use VSS to create data-consistent backups)
- **Failover Clustering:** Combined with Windows Server Failover Clustering to provide **high availability**. If a physical host fails, the VMs running on it automatically restart on another healthy host in the cluster
- **Virtual Storage:** Uses the **VHDX** virtual hard disk format, supporting disks up to **64 TB** with resilience to data corruption. Also supports pass-through disks and Storage Spaces Direct integration
- **Security Features**
  - **Secure Boot:** Verifies digital signatures of boot files to prevent malware
  - **Shielded VMs:** Encrypted virtual disks that can only run on specific guarded hosts
  - Guests cannot communicate with each other (except through traditional networks) and cannot perform DMA attacks

Supported Guest Operating Systems

- Hyper-V supports the following guest operating systems:
- All supported Windows versions
- Linux distributions: Red Hat Enterprise Linux, Debian, Oracle Linux, SUSE, Ubuntu
- FreeBSD
- Microsoft only provides support for issues found in Microsoft operating systems and Integration Services

Typical Use Cases

- **Server Consolidation:** Consolidate multiple physical servers onto fewer, more powerful hosts, saving space and energy
- **Development and Test Environments:** Quickly provision and restore virtualized test environments without affecting production systems
- **VDI (Virtual Desktop Infrastructure):** Combine with Remote Desktop Virtualization for centralized desktop management
- **Private Cloud Infrastructure:** Integrate with Storage Spaces Direct and SDN to run on-premises virtualized workloads
- **Disaster Recovery and Business Continuity:** Use live migration and replica features to minimize planned and unplanned downtime

Hyper-V vs VMWare

hypervisor for windows (pro onwards)

customized Linux distro

Aspect	Hyper-V	VMware ESXi
Hypervisor Design	Microkernelized: Device drivers run in the root/parent partition, not in the hypervisor itself	Monolithic: Device drivers are an integral part of the hypervisor layer
Driver Location	Root partition (Windows OS)	Hypervisor layer
Initialization	Faster, as drivers load in the OS layer	Slower, as drivers initialize in the hypervisor layer
Hardware Compatibility	Broad, since drivers are standard Windows drivers → shared with root partition	Limited to VMware's Hardware Compatibility List (HCL)
Security Patching	Root partition requires OS patches	Hypervisor layer is minimal and requires fewer patches

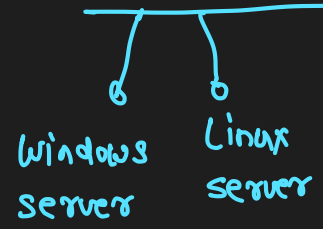
When to choose

- **Choose Hyper-V If**
  - Your organization is **Microsoft-centric** (Windows Server, Active Directory, System Center, Azure)
  - You want **lower licensing costs** and a predictable CAPEX model
  - You need **tight Azure hybrid cloud integration**
  - Your team is already skilled in **PowerShell and Windows administration**
  - You primarily run **Windows workloads**
- **Choose VMware If**
  - You need **maximum scalability** (thousands of VMs, very high-density clusters)
  - You run **heterogeneous workloads** (mixed Linux/Windows, legacy OS support, macOS in some cases)
  - You require **advanced networking** (NSX micro-segmentation)
  - You need **true Fault Tolerance** for zero-downtime critical applications
  - You have the **budget** to absorb **Broadcom**'s subscription costs
  - You want the **most mature enterprise ecosystem** and third-party integrations

Create VM

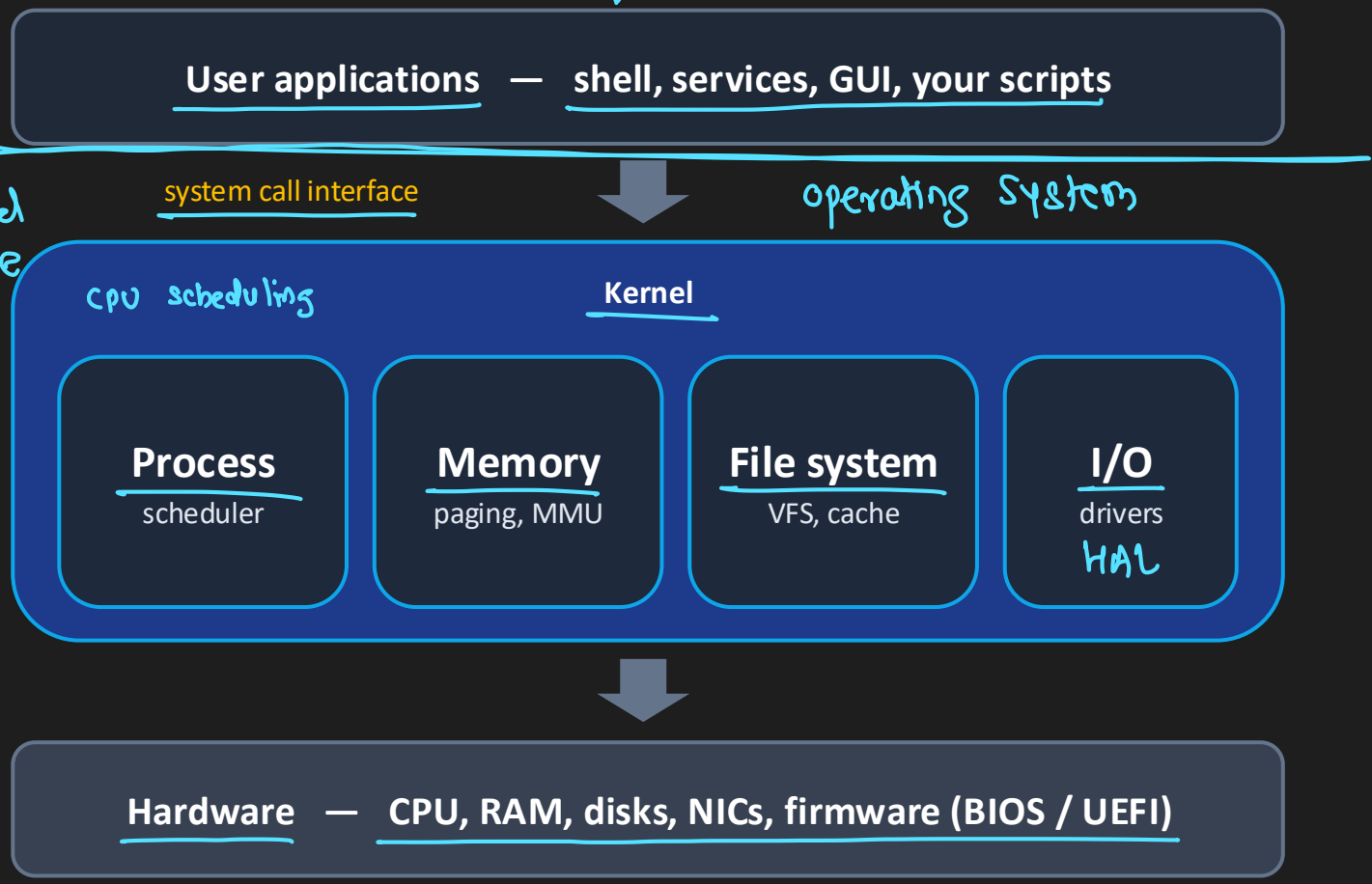
- **Open Hyper-V Manager:** Search for "Hyper-V Manager" in your Start menu and open it
- **Start the Wizard:** In the right-hand "Actions" pane, click **New > Virtual Machine**
- **Specify Name and Location:** Give your VM a descriptive name. You can optionally choose a custom folder to store its configuration files
- **Specify Generation:** Choose **Generation 2** unless you need to run a legacy OS that requires BIOS. Generation 2 offers UEFI and Secure Boot.
- **Assign Memory:** Allocate the startup RAM. For Windows 11, 4GB (4096 MB) is a good minimum. You can also enable **Dynamic Memory** to let Hyper-V adjust RAM usage automatically
- **Configure Networking:** Select a virtual switch to connect the VM to your network. If you haven't created one, you can skip this and configure it later
- **Connect Virtual Hard Disk:** This is the key storage configuration step
  - **Create a virtual hard disk:** This is the default. You can name the disk, choose its location, and set its size
  - **Use an existing virtual hard disk:** Select this if you already have a .vhd or .vhdx file
  - **Attach a virtual hard disk later:** You can add a disk after the VM is created
- **Installation Options:** Select **Install an operating system from a bootable image file** and browse to your OS .iso file.
- **Finish:** Review the summary and click **Finish**

OS Administration



What an Operating System Actually Does

- An OS is a **resource manager** plus an **abstraction layer**
  - Applications never touch hardware; they ask the kernel through **system calls**
  - The kernel arbitrates CPU, memory, storage and devices between processes
- The four classic subsystems
  - **Process management** — creation, scheduling, signals, termination
  - **Memory management** — virtual address spaces, paging, protection
  - **File system** — naming, permissions, buffering, journaling
  - **I/O and device management** — drivers, interrupts, DMA, buffering
- Why the user/kernel split matters to an administrator
  - A crash in user space kills one process; a fault in kernel space kills the machine (Linux kernel panic, Windows bug check / BSOD)
  - Drivers run in kernel space — an unsigned driver is a full system compromise
  - Every privilege model you will configure this term is enforced at this boundary

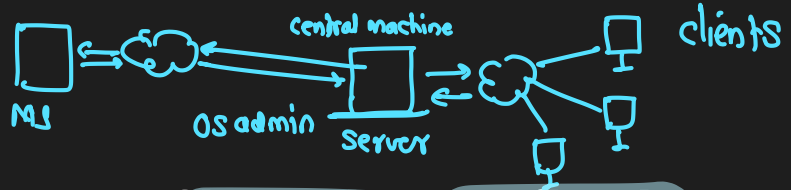


Windows calls the top half Win32/NT subsystems; Linux calls it glibc + POSIX. The shape is identical.

Windows and Linux: The Same Jobs, Different Words

Administrative job	Windows Server 2025	Linux Server
Install unattended	Answer file (autounattend.xml) + WDS	Kickstart file (ks.cfg) + PXE
Boot loader	Windows Boot Manager (bcdedit)	GRUB2 (grub2-mkconfig)
Start-up state model	Services + Service Control Manager	systemd units and targets
Software installation	MSI / Roles & Features / winget	RPM + DNF / DEB + APT
Central identity	Active Directory Domain Services	OpenLDAP / NIS / SSSD + Kerberos
Name resolution	Microsoft DNS (AD-integrated)	BIND (named)
Address assignment	Windows DHCP Server	ISC dhcpd / kea
File sharing	SMB shares + DFS	NFS exports + Samba
Web service	IIS	Apache httpd
Policy enforcement	Group Policy Objects	Config files + SELinux + Ansible
Logs	Event Viewer (evtx)	journald + rsyslog (/var/log)
Automation language	PowerShell	Bash

OS Administration



- **OS administration** is the practice of **installing, configuring, managing, maintaining, securing, and troubleshooting an operating system** so that it runs reliably, efficiently, and securely to support the applications and services that depend on it
- **Core Responsibilities of OS Administrator**

Installation & Deployment	Installing the OS, applying configurations, automating deployments (PXE, WDS, MDT, unattend files)
Configuration Management	Setting up roles, features, services, registry, and system settings
User & Access Management	Creating users/groups, assigning permissions, managing authentication (AD, local accounts)
Patch & Update Management	Applying security updates, cumulative updates, firmware patches (WSUS, Azure Update Manager)
Performance Monitoring	Tracking CPU, memory, disk, network; identifying bottlenecks; tuning
Storage Management	Managing disks, volumes, file systems, quotas, backups
Network Configuration	IP addressing, DNS, DHCP, firewall rules, NIC teaming
Security Hardening	Applying baselines, disabling unused services, enforcing policies, auditing
Backup & Recovery	Creating backups, testing restores, disaster recovery planning
Troubleshooting	Diagnosing and resolving OS-level issues (boot failures, service crashes, performance problems)
Automation	Using PowerShell, DSC, Ansible, or scripts to reduce manual work
Documentation	Recording configurations, procedures, and changes

OS Administration Roles

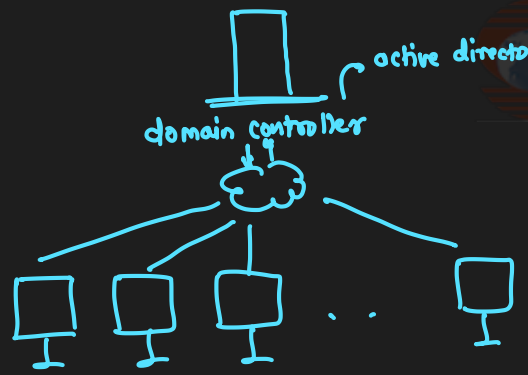
Role	Focus	Example Tasks
OS Administrator	The operating system itself	Patching, user accounts, services, performance tuning
Network Administrator	Network infrastructure	Routers, switches, firewalls, VLANs, VPNs
Database Administrator (DBA)	Database systems	SQL Server, backups, query optimization, indexing
Application Administrator	Specific applications	Exchange, SharePoint, IIS app pools
Cloud Administrator	Cloud resources	Azure VMs, storage accounts, networking, identity
Security Administrator	Security controls	Firewalls, SIEM, vulnerability management, compliance
DevOps Engineer	Automation and CI/CD	Pipelines, containers, infrastructure as code

Key Skills for OS Administrators

- **OS internals:** Processes, services, registry, file systems, memory management
- **Active Directory:** Users, groups, OUs, GPOs, replication, FSMO roles
- **PowerShell scripting:** Automation, reporting, bulk operations → Bash scripting
- **Virtualization:** Hyper-V, VMware, VM lifecycle management →
- **Networking basics:** TCP/IP, DNS, DHCP, subnetting, firewalls
- **Storage:** RAID, SAN/NAS, volumes, file systems, quotas
- **Security:** Hardening, patching, auditing, access control, encryption
- **Backup/Recovery:** VSS, Windows Server Backup, Azure Backup
- **Monitoring tools:** Performance Monitor, Event Viewer, SCOM, Azure Monitor : observability
- **Cloud platforms:** Azure, AWS, hybrid management (Azure Arc)

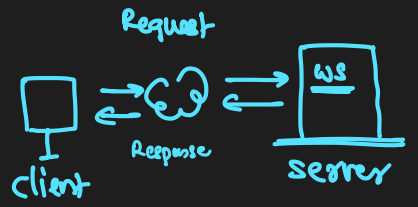
Common Services

LDAP / Active Directory



- **Service Name:** protocol ↳ MS implementation of LDAP
  - LDAP (Lightweight Directory Access Protocol) / Active Directory
- **What It Does:**
  - LDAP is a protocol used to access and maintain distributed directory information services over a network
  - Active Directory is Microsoft's directory service that implements LDAP (along with Kerberos) to provide centralized authentication, authorization, and directory management for users, computers, groups, and other objects
- **Common Use Case:**
  - Centralized identity management and single sign-on (SSO) across an enterprise
  - Enables users to log in once and access multiple resources, enforces security policies via Group Policy, and provides a central repository for user and computer accounts
- **Windows Implementation:** Active Directory Domain Services
- **Linux Implementation:** OpenLDAP, 389 Directory Server, FreeIPA

Web Server : host websites → http | https (80, 443)



- **Service Name:** Web Server
- **What It Does:**
  - Serves web content (HTML pages, images, CSS, JavaScript, APIs) to clients over HTTP or HTTPS
  - Processes incoming requests and returns the appropriate response, whether static files or dynamically generated content
- **Common Use Case:**
  - Hosting websites, web applications, intranets, and REST APIs
  - Used for everything from public-facing e-commerce sites to internal enterprise applications and cloud services
- **Windows Implementation:** Internet Information Services (IIS) : natively support .net
- **Linux Implementation:** Apache HTTP Server, Nginx, Lighttpd

REST → REpresentational state Transfer → design pattern

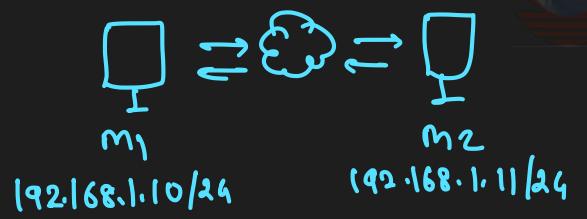
Apache HTTP Server → httpd

DNS Server

use case 1 : public network → translates domain name to IP address
use case 2 : local network → translates hostname to IP address

- **Service Name:** DNS (Domain Name System) Server
- **What It Does:**
  - Resolves human-readable domain names into IP addresses that computers use to communicate
  - It acts as the "phonebook" of the internet and local networks, translating names to addresses and vice versa
- **Common Use Case:**
  - Essential for nearly all network communication
  - Used for resolving website names, locating domain controllers, finding mail servers (MX records), and enabling service discovery (SRV records)
- **Windows Implementation:** DNS Server Role (integrated with Active Directory)
- **Linux Implementation:** BIND (Berkeley Internet Name Domain), dnsmasq, PowerDNS, Knot DNS

DHCP Server



- **Service Name:** DHCP (Dynamic Host Configuration Protocol) Server
- **What It Does:**
  - Automatically assigns IP addresses, subnet masks, default gateways, DNS servers, and other network configuration parameters to devices when they join the network
  - Manages IP address leases and prevents conflicts
- **Common Use Case:**
  - Simplifying network configuration for workstations, laptops, phones, printers, and IoT devices
  - Eliminates manual IP configuration and enables devices to move between networks seamlessly
- **Windows Implementation:** DHCP Server Role (supports failover and load balancing)
- **Linux Implementation:** ISC DHCP Server, dnsmasq, Kea DHCP , dhcpd

Mail Server (SMTP, IMAP, POP3)

- **Service Name:** Mail Server (SMTP, IMAP, POP3)
- **What It Does:**
  - Handles the sending, receiving, storage, and retrieval of email
  - **SMTP** sends and relays outgoing mail between servers
  - **IMAP** allows clients to access and manage messages stored on the server, keeping them synchronized across multiple devices
  - **POP3** downloads emails to a single client device and typically deletes them from the server
- **Common Use Case:**
  - Enterprise email communication, automated notifications, and mailing lists. SMTP handles outbound delivery, IMAP enables multi-device access with server-side storage, and POP3 supports simple offline email access from one device
- **Windows Implementation:** Microsoft Exchange Server, hMailServer
- **Linux Implementation:** Postfix (SMTP), Dovecot (IMAP/POP3), Sendmail, Exim

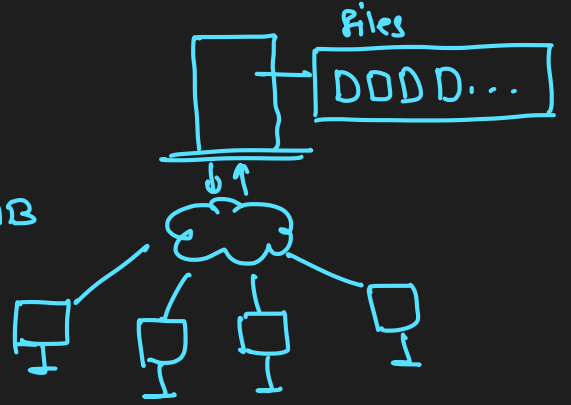
Virtualization Server : VM management | VMM - virtual machine manager | hypervisor

- **Service Name:** Virtualization Server (hardware virtualization)
- **What It Does:**
  - Hosts and manages virtual machines (VMs) by abstracting physical hardware resources (CPU, memory, storage, network) and allocating them to multiple isolated virtual environments
  - Enables consolidation, isolation, and rapid provisioning of workloads
- **Common Use Case:**
  - Server consolidation to reduce hardware costs, running multiple operating systems on a single physical host, creating development and test environments, and enabling live migration and high availability for critical workloads
    - ↳ horizontally scaled
- **Windows Implementation:** Microsoft Hyper-V
- **Linux Implementation:** KVM (Kernel-based Virtual Machine), Xen, Proxmox VE, VMware ESXi (bare-metal hypervisor)

Virtualization server
- ↳ bare metal : xen | vmware ESXi
- ↳ hosted : hyper-v | vmware workstation | virtual Box

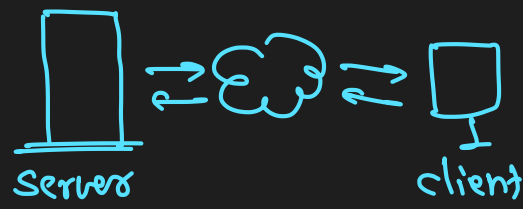
File Servers (FTP, NFS, SMB)

Server OS	client OS		Servers
Linux	windows	→	FTP
Linux	Linux	→	NFS
windows	win | linux	→	FTP | SMB
windows	windows	→	SMB



- **Service Name:** File Server (FTP, NFS, SMB)
- **What It Does:**
  - Provides centralized storage and sharing of files over a network
  - **SMB/CIFS** is the standard for Windows file sharing
  - **NFS** is the standard for Linux/Unix file sharing
  - **FTP** is a legacy protocol for transferring files between systems, often used for bulk uploads and downloads
- **Common Use Case:**
  - Shared document storage for teams, central data repositories for applications, home directories for users, backup targets, and cross-platform file exchange between Windows and Linux systems
- **Windows Implementation:** File and Storage Services (SMB/CIFS shares), FTP Server role in IIS
- **Linux Implementation:** Samba (SMB/CIFS), NFS (Network File System), vsftpd or ProFTPD (FTP)

Remote Desktop Server



- **Service Name:** Remote Desktop Server
- **What It Does:**
  - Provides remote access to a full desktop environment or individual applications hosted on a server
  - Users connect over the network and interact with the remote session as if they were sitting in front of the machine
- **Common Use Case:**
  - Enabling remote work, providing contractors and branch offices with secure access to corporate applications, application virtualization (RemoteApp), and virtual desktop infrastructure (VDI) deployments
- **Windows Implementation:** Remote Desktop Services (RDS) — includes Session Host, Connection Broker, Gateway, and Web Access roles
- **Linux Implementation:** XRDP, VNC (TigerVNC, TightVNC), NoMachine, Apache Guacamole

Windows Server

Windows : client Editions → consumer (End user's) desktop

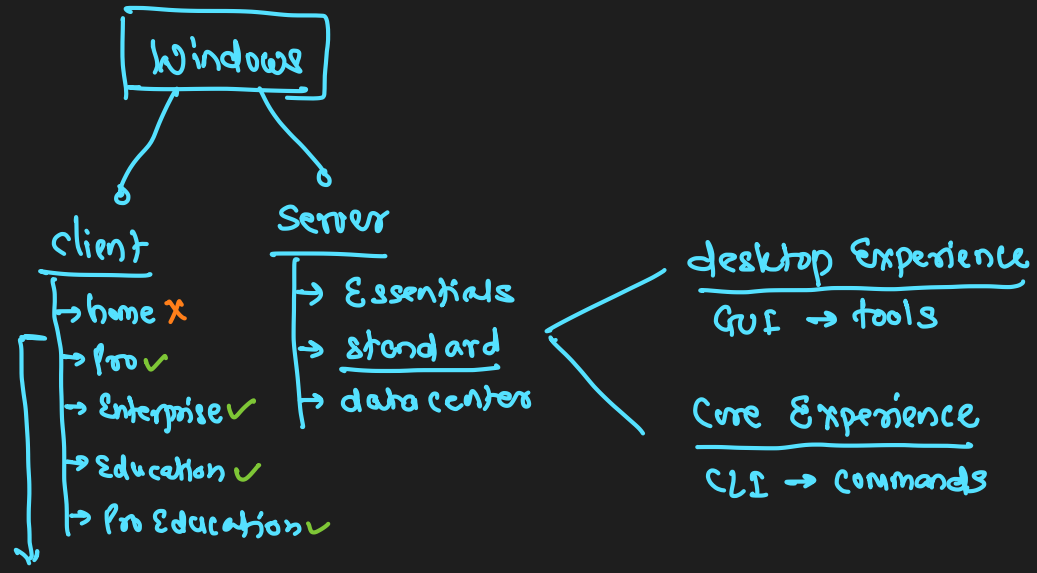
- Windows is a family of graphical operating systems developed by Microsoft, first launched in 1985 as a graphical shell running on MS-DOS
- **Windows Client Versions**
  - **Windows 1.0 through 3.1 (1985-1992):** MS-DOS-based graphical shells; required DOS to be running
  - **Windows 95/98/ME (1995-2000):** Consumer line; Windows 95 was first to boot directly into Windows and introduced the Start menu and Taskbar
  - **Windows NT 3.1 through 4.0 (1993-1996):** First true 32-bit OS with separate client and server versions; NT 4.0 adopted the Windows 95 interface
  - **Windows 2000 (2000):** Updated NT 4 with Plug and Play and Active Directory support
  - **Windows XP (2001):** United consumer and professional lines; widely used and long-supported
  - **Windows Vista (2006):** Introduced Aero UI and UAC; widely criticized for bugs
  - **Windows 7 (2009):** Greatly improved stability over Vista; very popular
  - **Windows 8/8.1 (2012):** Touch-first interface; controversial Start screen removal
  - **Windows 10 (2015):** 64-bit only; supports 32-bit applications but not 16-bit DOS programs
  - **Windows 11 (2021):** Introduced "Windows as a Service"; final version to support 32-bit architecture

Client Editions (Windows 10/11)

- **Home** → can Not join the domain
  - Consumer-focused with Microsoft Edge, Windows Hello, virtual desktops, device encryption, and automatic updates
- **Pro**
  - Small/medium businesses, advanced users
  - Adds BitLocker, Hyper-V, domain join, Group Policy management, Remote Desktop, Windows Autopilot, and MDM support
- **Enterprise**
  - Large enterprise organizations
  - Adds BranchCache, Credential Guard, AppLocker, Application Guard, App-V, UE-V, and advanced deployment options
  - Available for Volume licensing customers only
- **Pro Education**
  - School staff, administrators, teachers, students
  - Available for Academic volume licensing
- **Education**
  - School staff, administrators, teachers, students
  - Available for Academic volume licensing

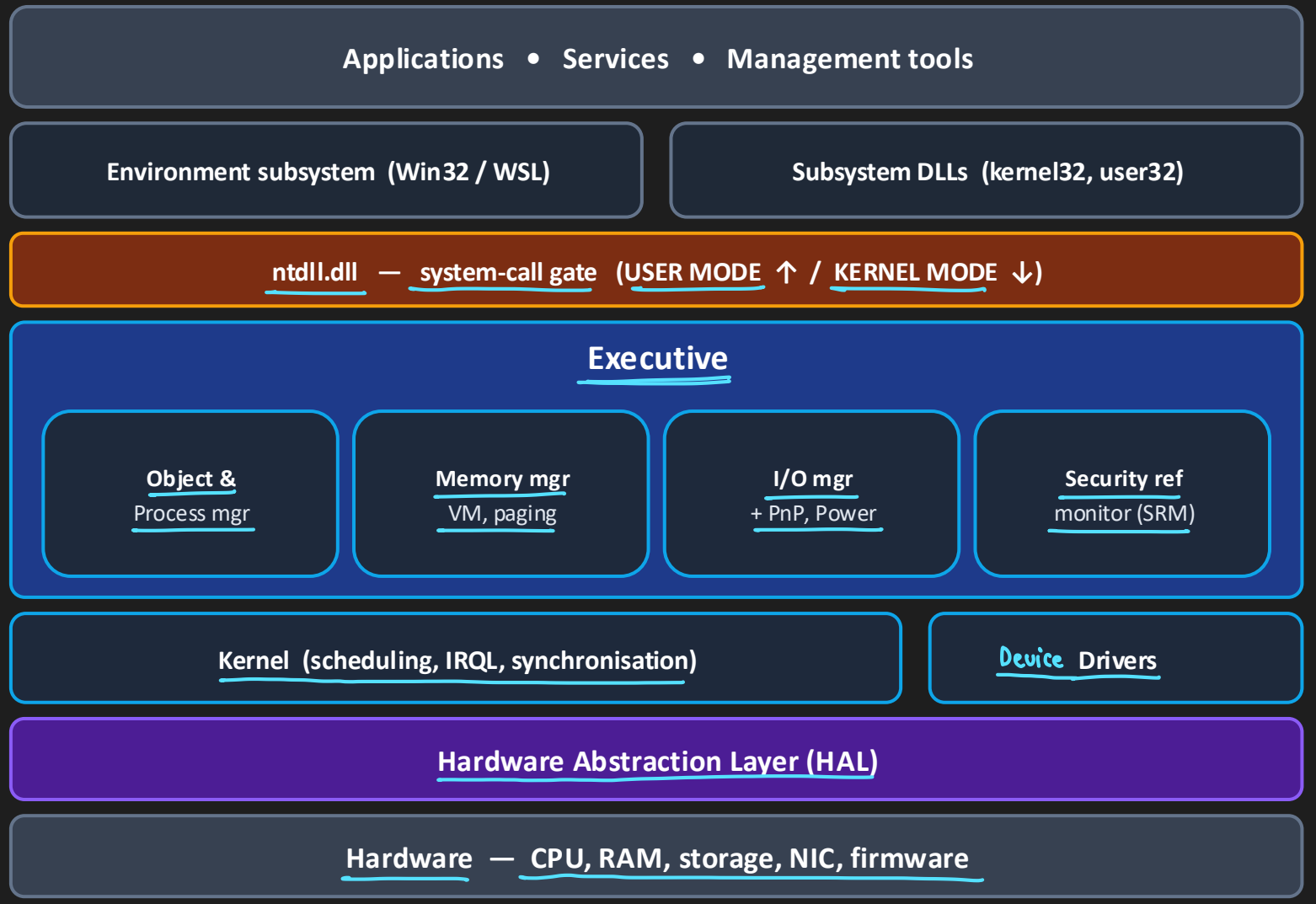
Windows Server

- Windows Server is Microsoft's enterprise-grade server platform for running and protecting applications, services, and workloads across on-premises, hybrid, and cloud environments
- It is the infrastructure backbone for millions of organizations worldwide, powering everything from file servers and web applications to complex enterprise workloads and AI-driven solutions
- A secure, scalable, high-performance compute foundation that evolves as organizations grow
- Server Editions
  - Essentials
    - Small organizations or first server deployments
    - Limited availability
  - Standard
    - Physical or lightly virtualized environments
    - General purpose
  - Datacenter
    - Highly virtualized datacenters and hybrid cloud
    - Large-scale scenarios



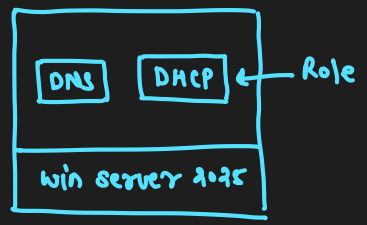
Windows Server Architecture

- **Two privilege rings**
  - **User mode** — applications and environment subsystems; each process gets its own private virtual address space
  - **Kernel mode** — one shared address space; a fault here stops the machine
- **Layers**
  - **HAL** hides hardware details, it is a collection of device drivers
  - **Microkernel** does thread scheduling, interrupt dispatch, synchronisation
  - **Executive** holds the managers: Object, Process, Memory, I/O, Security Reference Monitor, Plug-and-Play, Power, Configuration (registry)
  - **Drivers** are loadable kernel modules
- **Where your administration lands**
  - Services (`services.msc`) are user-mode processes started by the SCM
  - Security tokens issued at logon are checked by the SRM on every object access → Kerberos
  - `ntoskrnl.exe` + `hal.dll` are the two files this architecture is made of



System Components

- **OS Kernel** — Manages hardware resources, memory, processes, and security; ensures stability and performance across workloads (ntoskrnl.exe)
- **Modular Server Roles** — Active Directory, DNS, DHCP, IIS, etc., can be installed independently on demand, reducing system overhead and potential attack surface → provides program and required configuration
- **Hyper-V Virtualization** — Runs multiple VMs on a single physical server; supports nested virtualization and live migration. → Hosted H/W virtualization + container service
- **Storage and Networking** — Storage Spaces Direct provides fault-tolerant storage; Software-Defined Networking (SDN) enables dynamic configuration
- **Security Infrastructure** — Credential Guard, Secured-Core Server, BitLocker, combined with Group Policy and AD for access control
- **Management and Automation** — Windows Admin Center provides centralized GUI management; PowerShell and DSC enable automation



Server Roles

- **Active Directory Domain Services:** Identity and access management, authentication, Group Policy
- **DNS Server:** Name resolution, integrated with AD for dynamic updates
- **DHCP Server:** Automatic IP address assignment, scope management, failover
- **File and Storage Services:** SMB shares, DFS, File Server Resource Manager (FSRM), deduplication
- **Web Server (IIS):** Hosting ASP.NET, PHP, static sites; supports HTTP/2, TLS 1.3
- **Hyper-V:** Virtualization platform for VMs, containers, and virtual switches
- **Remote Desktop Services (RDS):** Virtual desktop infrastructure (VDI), session-based desktops
- **Windows Server Update Services (WSUS):** Centralized patch management for Windows systems
- **Print and Document Services:** Centralized print management, driver deployment
- **Certificate Authority:** Public Key Infrastructure (PKI), certificate issuance and management

Management Tools

- **Server Manager:** Central dashboard for roles, features, and server groups
- **Windows Admin Center:** Modern browser-based management for servers, clusters, and Azure services
- **Active Directory Users and Computers (ADUC):** Manage users, groups, computers, OUs
- **Group Policy Management Console (GPMC):** Create, edit, and link Group Policy Objects (GPOs)
- **DNS Manager:** Manage DNS zones, records, and forwarders
- **DHCP Manager:** Manage scopes, reservations, and options
- **Hyper-V Manager:** Create and manage VMs, virtual switches, checkpoints
- **Failover Cluster Manager:** Configure and monitor cluster roles and resources
- **Event Viewer:** View and filter system, application, and security logs
- **Performance Monitor:** Track system performance counters in real time

Storage Technologies

- **NTFS:** Standard file system; supports quotas, EFS, compression, shadow copies
- **ReFS:** Resilient File System; integrity streams, auto-repair, supports large volumes (up to 35 PB)
- **Storage Spaces:** Pool physical disks; create virtual disks with redundancy (mirror, parity)
- **Storage Spaces Direct (S2D):** Uses local disks across cluster nodes for software-defined storage
- **DFS (Distributed File System):** Namespace (DFS-N) and replication (DFS-R) for file shares
- **Data Deduplication:** Reduces storage footprint by eliminating duplicate data blocks
- **File Server Resource Manager (FSRM):** Quotas, file screening, storage reports
- **SMB Shares:** Standard file sharing protocol; SMB 3.x adds encryption and RDMA
- **iSCSI Target:** Serve block storage to initiators
- **Storage Replica:** Synchronous/asynchronous replication for disaster recovery

Monitoring Tools

- **Performance Monitor:** Real-time and logged counter analysis
- **Resource Monitor:** Detailed CPU, disk, network, and memory usage
- **Task Manager:** Quick overview of processes and resource usage
- **Event Viewer:** System, application, and security logs
- **Windows Admin Center:** Performance dashboards and alerts
- **Azure Monitor:** Cloud-based metrics, logs, and alerts
- **SCOM (System Center Operations Manager):** Enterprise monitoring and alerting

Key Differences

10 years support → 5 years active support + 5 years update/fix support → compatability

2000 2003 2008 2011 2016 2019 2022 2025 || distro dependent

Aspect	Windows Server	Windows Client
Purpose	Host server roles and enterprise workloads	End-user productivity
Management	Server Manager, WAC, PowerShell, Group Policy	Settings, Intune, Group Policy
Roles	AD, DNS, DHCP, IIS, Hyper-V, etc.	Limited (Hyper-V, IIS optional)
Licensing	Per-core, CALs required for users/devices	Per-device or per-user
Updates	Long-term servicing, cumulative updates	Monthly cumulative updates
Security	Secured-Core, Credential Guard, LSASS protection	Defender, BitLocker, Hello
Performance	Optimized for background services and throughput	Optimized for foreground user experience
GUI	Optional (Server Core, Desktop Experience) ✓	Full GUI
Cost	Higher; includes enterprise features → 90 day trials	Lower; consumer-focused

iSCSI

Introduction

- iSCSI (Internet Small Computer System Interface) is a TCP/IP-based storage networking standard for connecting data storage services
- It provides an inexpensive and easy-to-implement block-level access to remote storage by transporting SCSI commands over a standard Ethernet network
- Effectively, iSCSI emulates the high-performance local storage bus subsystem and creates a logical Storage Area Network (SAN)
- **Key Characteristics**
  - iSCSI requires no specialized cabling or connectivity devices
  - It runs over existing switching and IP infrastructure
  - **Default TCP port**: iSCSI uses **TCP port 3260** by default
  - iSCSI provides **block-level access**, not file-level access
  - Windows Server natively includes both the iSCSI Target Server role and the iSCSI Initiator, with no additional licensing cost

iSCSI Architecture and Components

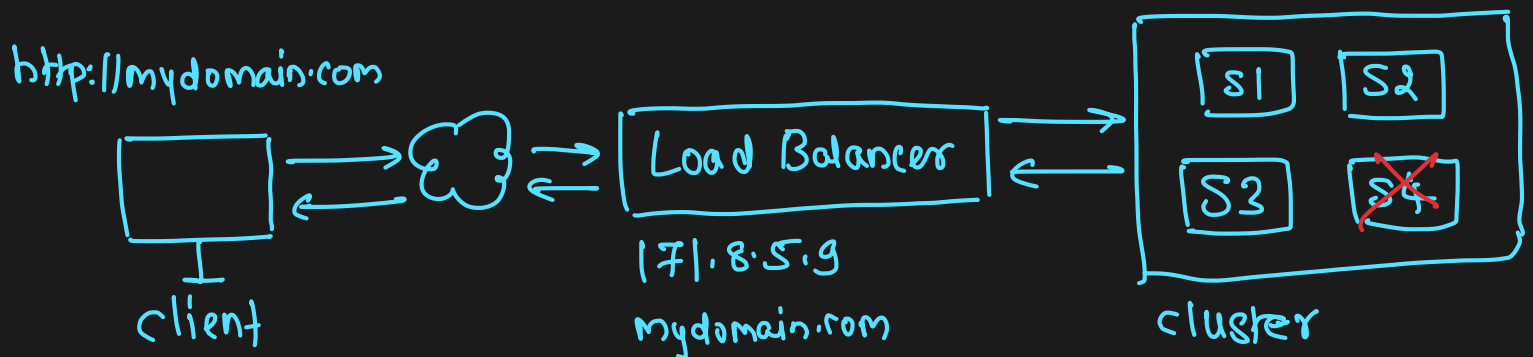
- IP Network
  - Standard Ethernet adapters and switches connect to iSCSI storage devices.
  - The network should provide at least 1 Gbps throughput and include multiple physical paths to the iSCSI target
- iSCSI Target → Server
  - Allows access to its storage over an IP network
  - Conceptually equivalent to a SCSI controller providing access to local storage.
  - Windows Server provides the iSCSI Target Server as a role service of the File and Storage Services role
- iSCSI Initiator → client
  - Client-side component that provides connectivity to a remote iSCSI target
  - The Windows operating system includes the iSCSI initiator software by default
- IQN (iSCSI Qualified Name)
  - Unique identifiers that iSCSI uses to designate and locate initiators and targets on an iSCSI network
  - When configuring an iSCSI target, you must configure the IQN for the iSCSI initiators that will connect
- iSNS (Internet Storage Name Service)
  - Facilitates discovery, management, and configuration of iSCSI targets

iSCSI Functionality

- **iSCSI Target Server Role Service**
  - **Authentication:** Implements CHAP (Challenge Handshake Authentication Protocol) to authenticate initiator connections, and reverse CHAP to enable the initiator to authenticate the iSCSI target
  - **Virtual Disks:** Implements iSCSI virtual disks as **VHD or VHDX files**, with VHDX supporting up to **64 TB** capacity
  - **Management:** Supports management via Server Manager, Windows PowerShell, and System Center Virtual Machine Manager
  - **Scale Limits:** Scales to **256 iSCSI targets** and **512 virtual disks** per server
- **iSNS Role Service**
  - Hosts a database of active iSCSI nodes (initiators, targets, and management systems)
  - Supports automatic registration of initiators and targets, allowing initiators to query the iSNS server for available targets
  - Implements a State Change Notification Service that triggers notifications to registered clients when database changes occur
  - Offers a Discovery Domain Service that restricts connectivity between designated initiators and targets by grouping them into discovery domains

Deployment Considerations

- **Network Design**
  - When implementing iSCSI networking, you should use a **separate, dedicated network** or configure **Quality of Service (QoS)** on your existing network
  - To ensure sufficient performance, the network should provide at least 1 Gbps throughput and include multiple physical paths to the iSCSI target
- **High Availability**
  - For highly available storage, you can choose between Fibre Channel storage, a SCSI storage array, or a **Storage Spaces Direct cluster**, with the iSCSI Target Server configured as a **cluster role**
- **Network Adapter Considerations**
  - Network adapters with hardware offload capabilities such as **large send/TCP segmentation offload** and **checksum offload** are recommended
  - These features move processing from the CPU to the adapter, freeing the processor for other tasks
  - A Gigabit or 10 GB network provides low latency while maximizing throughput



Load Balancer

HAProxy

Nginx

What is load balancing ?

- A **load balancer** is a system that sits in front of a group of servers and distributes incoming network traffic across them
- Instead of all requests hitting one server, the load balancer decides which server should handle each request → algorithms
- Modern applications must process millions of users simultaneously and return the correct text, videos, images, and other data to each user in a fast and reliable manner
- To handle such high volumes of traffic, most applications have many resource servers with duplicate data between them
- A load balancer is a device that sits between the user and the server group (cluster) and acts as an invisible facilitator, ensuring that all resource servers are used equally

Problems LB solves

- Overload – One server gets crushed under traffic
  - Spreads requests across many servers
- Downtime – If the server crashes, the app goes offline
  - Detects failures and reroutes to healthy servers
- Slow performance – Users wait longer
  - Distributes work so no single server is a bottleneck
- Scaling limits – Can't grow beyond one machine
  - Lets you add more servers easily (horizontal scaling)
- Maintenance – Can't update a server without downtime
  - Drains traffic from one server while you patch it

Load Balancing algorithms – Static Load Balancing

- Static load balancing algorithms follow fixed rules and are independent of the current server state
- The following are examples of static load balancing
- Round-robin method
  - Servers have IP addresses that tell the client where to send requests
  - The IP address is a long number that is difficult to remember
  - To make it easy, a Domain Name System maps website names to servers
  - In the round-robin method, an authoritative name server does the load balancing instead of specialized hardware or software. The name server returns the IP addresses of different servers in the server farm turn by turn or in a round-robin fashion
- Weighted round-robin method
  - In weighted round-robin load balancing, you can assign different weights to each server based on their priority or capacity
  - Servers with higher weights will receive more incoming application traffic from the name server
- IP hash method
  - The load balancer performs a mathematical computation, called hashing, on the client IP address
  - It converts the client IP address to a number, which is then mapped to individual servers

Load Balancing algorithms – Dynamic Load Balancing

- Dynamic load balancing algorithms examine the current state of the servers before distributing traffic
- Least connection method
  - A connection is an open communication channel between a client and a server
  - When the client sends the first request to the server, they authenticate and establish an active connection between each other
  - In the least connection method, the load balancer checks which servers have the fewest active connections and sends traffic to those servers
  - This method assumes that all connections require equal processing power for all servers
- Weighted least connection method
  - Weighted least connection algorithms assume that some servers can handle more active connections than others
  - Therefore, you can assign different weights or capacities to each server, and the load balancer sends the new client requests to the server with the least connections by capacity
- Least response time method
  - The response time is the total time that the server takes to process the incoming requests and send a response
  - The least response time method combines the server response time and the active connections to determine the best server
  - Load balancers use this algorithm to ensure faster service for all users
- Resource-based method
  - In the resource-based method, load balancers distribute traffic by analyzing the current server load
  - Specialized software called an agent runs on each server and calculates usage of server resources, such as its computing capacity and memory
  - Then, the load balancer checks the agent for sufficient free resources before distributing traffic to that server

Load Balancer Implementation

- Application load balancing
  - Complex modern applications have several server farms with multiple servers dedicated to a single application function
  - Application load balancers look at the request content, such as HTTP headers or SSL session IDs, to redirect traffic.
  - For Example
    - An ecommerce application has a product directory, shopping cart, and checkout functions
    - The application load balancer sends requests for browsing products to servers that contain images and videos but do not need to maintain open connections.
    - By comparison, it sends shopping cart requests to servers that can maintain many client connections and save cart data for a long time
- Network load balancing
  - Network load balancers examine IP addresses and other network information to redirect traffic optimally
  - They track the source of the application traffic and can assign a static IP address to several servers
  - Network load balancers use the static and dynamic load balancing algorithms described earlier to balance server load.

Load Balancer Implementation

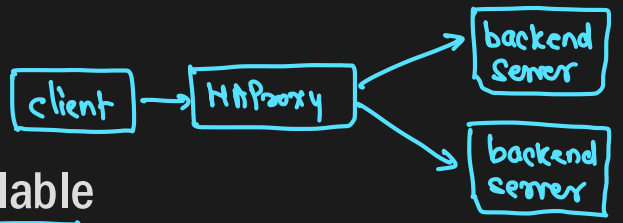
- Global server load balancing
  - Global server load balancing occurs across several geographically distributed servers
  - For example, companies can have servers in multiple data centers, in different countries, and in third-party cloud providers around the globe
  - In this case, local load balancers manage the application load within a region or zone
  - They attempt to redirect traffic to a server destination that is geographically closer to the client
  - They might redirect traffic to servers outside the client's geographic zone only in case of server failure
- DNS load balancing
  - In DNS load balancing, you configure your domain to route network requests across a pool of resources on your domain.
  - A domain can correspond to a website, a mail system, a print server, or another service that is made accessible through the internet
  - DNS load balancing is helpful for maintaining application availability and balancing network traffic across a globally distributed pool of resources

HAProxy

Introduction

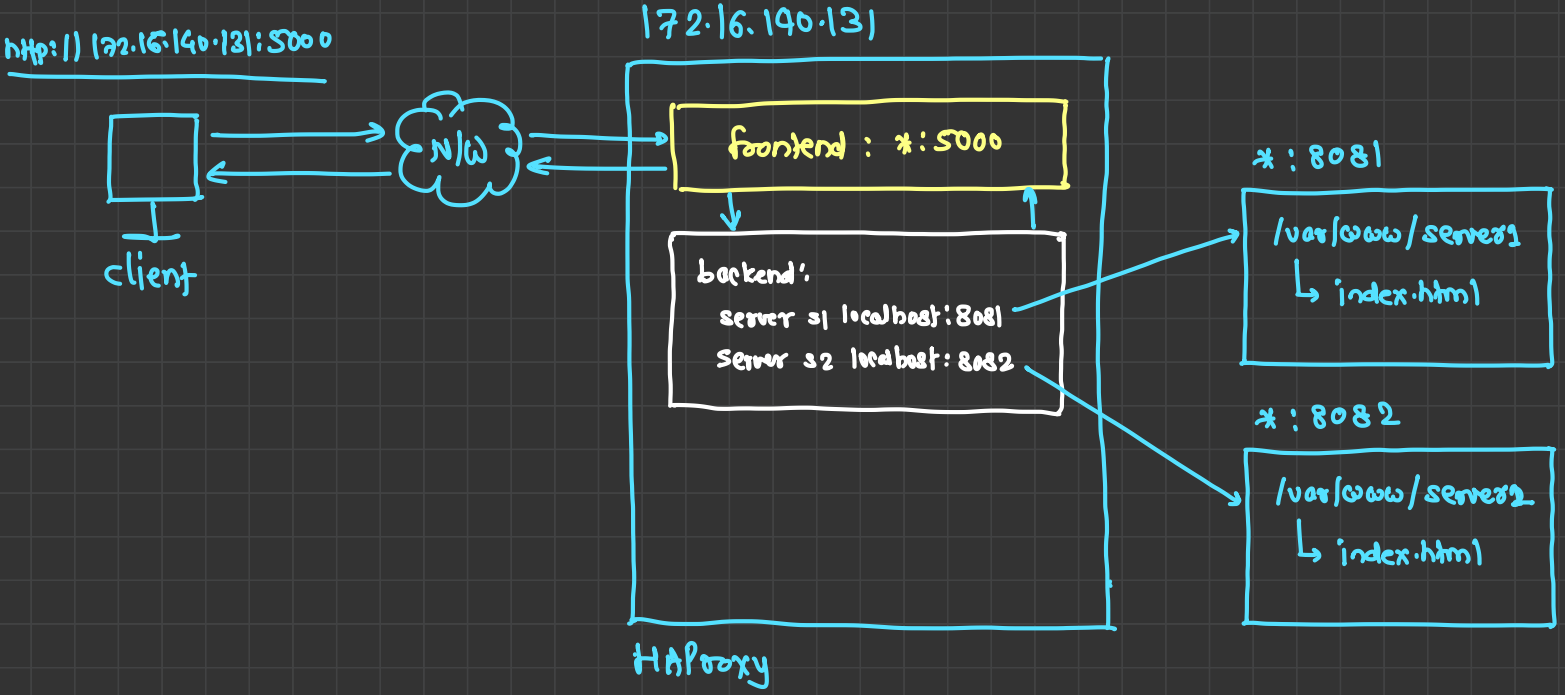
High Availability → application is always available to the user (even if server(s) are down)
↳ can be achieved only by horizontal scaling

- HAProxy (High Availability Proxy) is an open-source software solution used to distribute network traffic across multiple backend servers
- It functions as a load balancer and proxy server, helping applications handle more traffic and remain available when individual backend servers fail
- Key features
  - Load balancing: Distributes incoming requests across multiple backend servers
  - High availability: Helps keep applications accessible when a backend becomes unavailable
  - Reverse proxy: Accepts client requests and forwards them to backend applications
  - Health checks: Monitors backend availability and removes unhealthy servers from service → uses heartbeat signal
  - Performance: Efficiently handles large numbers of concurrent connections
  - Security & traffic management: Supports TLS termination, access controls, connection limits, and traffic-routing rules
- Common use cases
  - Load balancing web applications and REST APIs
  - Distributing requests across application servers → Network load balancing
  - Routing traffic to different services based on hostnames or URL paths → application load balancing
  - Providing a single entry point for a cluster of servers



HAProxy Architecture

- HAProxy sits between clients and backend servers
- Clients send requests to HAProxy, which selects an appropriate backend server and forwards the traffic
- Main components
  - Frontend: Defines the address and port on which HAProxy accepts client connections
  - Backend: Defines a group of application servers and the rules for distributing traffic among them
  - Server: Represents an individual backend destination, identified by its hostname or IP address and port
  - Health check: Determines whether a backend is available to receive traffic
  - ACL (Access Control List): Matches request properties and enables conditional routing decisions → ALB
- HAProxy supports both Layer 4 (TCP) and Layer 7 (HTTP) load balancing
- Layer 4 works with TCP connections, while Layer 7 can make routing decisions using HTTP information such as hostnames, paths, and headers


http://172.16.140.131:5000
172.16.140.131
client
N/W
frontend : *:5000
backend:
server s1 localhost:8081
server s2 localhost:8082
HAProxy
*:8081
/var/www/server1
index.html
*:8082
/var/www/server2
index.html

Load-Balancing Algorithms

- HAProxy supports multiple algorithms for choosing a backend server
- The appropriate choice depends on application behavior, server capacity, connection duration, and whether clients need session persistence
- roundrobin
  - Rotates through available servers, accounting for weights
  - General web applications
- leastconn
  - Selects the server with the fewest active connections
  - Long-lived connections or uneven request durations
- source
  - Uses the client's source IP to select a server consistently
  - Simple IP-based affinity
- first
  - Fills servers in order, subject to capacity and availability
  - Consolidating load onto fewer servers

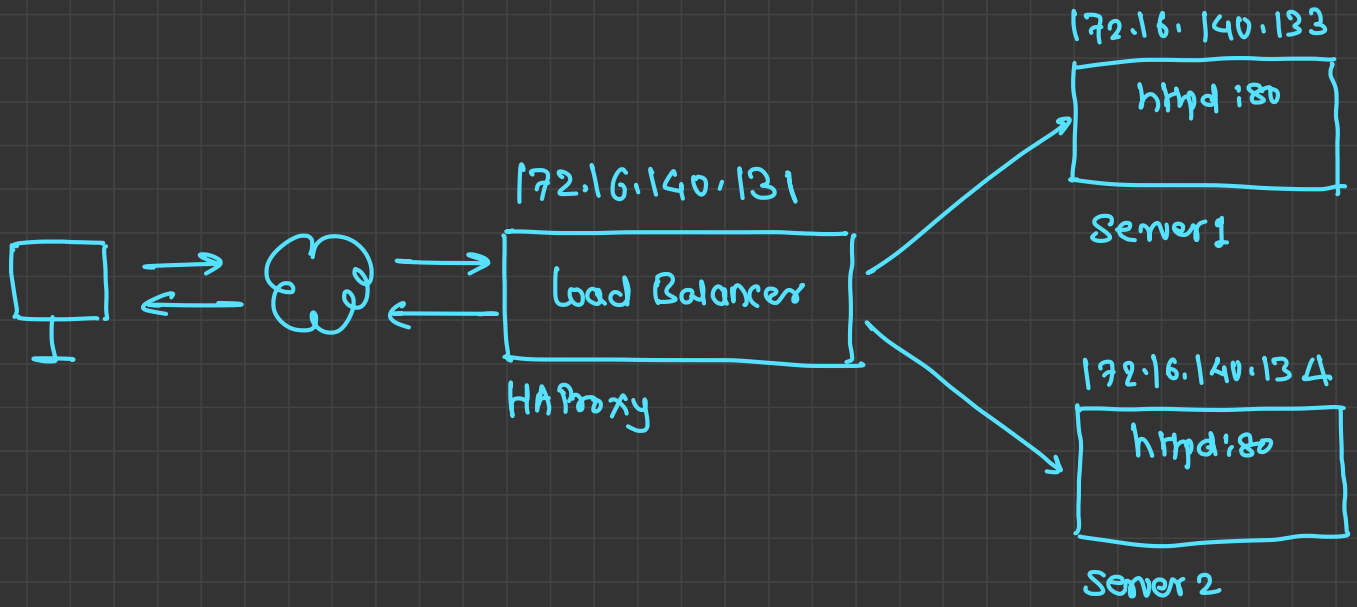
Configuration for Layer 4 Load Balancing

- Install HAProxy package
  - sudo dnf install haproxy -y
- Check HAProxy version
  - haproxy --version
- Check service status
  - sudo systemctl status haproxy
- Configuration file
  - The main configuration file is typically located at /etc/haproxy/haproxy.cfg
- Backend: Distribute to two web servers

```
backend http_backend
  mode http
  balance roundrobin
  server web_server1 192.168.1.10:80 check
  server web_server2 192.168.1.11:80 check
```

Configuration for Layer 7 Load Balancing

```
frontend http_frontend
    bind *:80

    # Define ACLs
    acl is_api path_beg /api/
    acl is_static path_beg /static/

    # Route to backends based on ACL matches
    use_backend api_servers if is_api
    use_backend static_servers if is_static
    default_backend web_servers

backend api_servers
    http-request replace-path ^/api/(.*) /\1
    http-request replace-path ^/api$ /
    server api1 10.0.0.1:8080 check
backend static_servers
    http-request replace-path ^/static/(.*) /\1
    http-request replace-path ^/static$ /
    server static1 10.0.0.10:80 check
backend web_servers
    http-check expect status 200
    server web1 10.0.0.20:8080 check
```

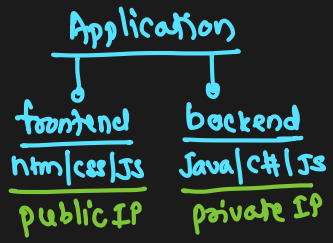

172.16.140.133
httpd :80
Server1
172.16.140.131
Load Balancer
HAProxy
172.16.140.134
httpd:80
Server 2

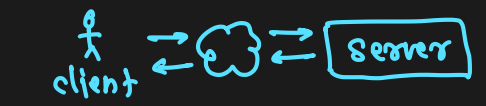
Nginx

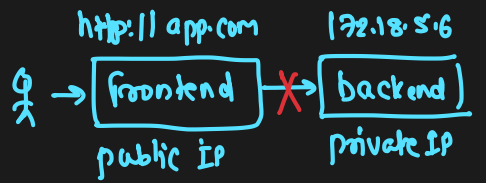
Introduction

- NGINX is a high-performance web server that can also act as a reverse proxy, HTTP load balancer, and TCP/UDP proxy.
- It receives requests from clients and forwards them to backend servers according to configured routing and balancing rules
- A reverse proxy provides a single entry point to backend applications
- A load balancer extends this arrangement by distributing traffic across multiple backend servers
- Key functions
  - Reverse proxy: Forwards requests to one or more backend applications
  - Load balancing: Distributes incoming requests across a pool of backend servers
  - SSL/TLS termination: Handles HTTPS encryption at the proxy, when configured
  - Traffic routing: Sends requests to different applications based on hostnames or URL paths
  - Connection management: Handles client connections independently of backend connections
  - Caching and compression: Can reduce backend workload and improve response performance when configured appropriately

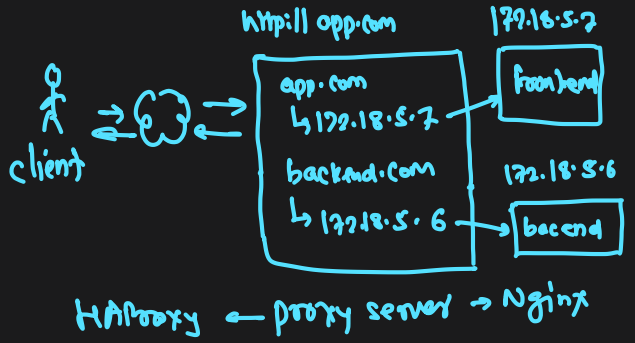
Reverse Proxy







- A reverse proxy sits between clients and backend servers
- Clients connect to the proxy rather than directly to the application servers
- How the request flows
  - A client sends a request to the NGINX server
  - NGINX evaluates its configuration to identify the matching server and location blocks
  - NGINX forwards the request to the configured backend or upstream group
  - The backend processes the request and returns a response
  - NGINX returns the response to the client

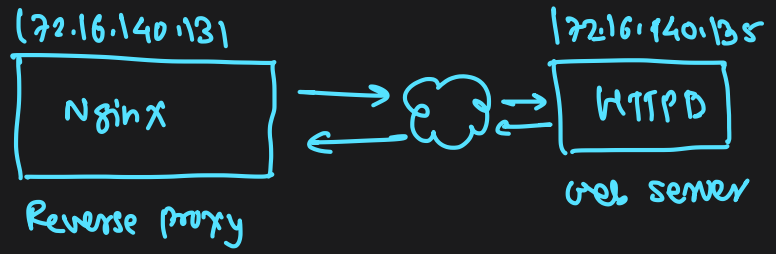


- A reverse proxy can forward requests to a single backend, while a load balancer distributes requests across multiple backends
- NGINX can perform both functions using the same configuration framework

Installing nginx

- Install nginx
  - sudo dnf install nginx –y
- Autostart nginx
  - sudo systemctl enable --now nginx
- Check status
  - sudo systemctl status nginx
- Check version
  - nginx –v
- Aad firewall rules
  - sudo firewall-cmd --permanent --add-service=http
  - sudo firewall-cmd –reload
- Configuration files
  - /etc/nginx/nginx.conf: Main NGINX configuration file
  - /etc/nginx/conf.d/: Directory for additional configuration files
  - /var/log/nginx/access.log: HTTP access log, when configured by the active server
  - /var/log/nginx/error.log: Error log, when configured by the active server

Configuring NGINX as a Reverse Proxy




```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://<server address>;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Configuring NGINX as a Load Balancer

```
upstream servers {
    server 172.16.140.211:8000;
    server 172.16.140.215:8000;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://servers;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Configure SELinux

- Check SELinux before changing security settings
  - getenforce
  - getsebool httpd_can_network_connect
- Allow the NGINX SELinux domain to make outbound network connections
  - sudo setsebool -P httpd_can_network_connect 1
- Check recent errors
  - sudo tail -f /var/log/nginx/error.log
- Inspect recent access records
  - sudo tail -n 50 /var/log/nginx/access.log
- Test configuration syntax
  - sudo nginx -t

Pros and cons

- **Advantages of Nginx Load Balancing**
  - Very fast, lightweight, handles thousands of concurrent connections
  - Works as both load balancer and reverse proxy (SSL termination, caching, compression)
  - Simple configuration syntax
  - Supports HTTP, HTTPS, TCP/UDP (stream module), gRPC
  - Free open-source version available

- **Limitations**
  - **Active health checks** and **sticky sessions (cookie-based)** require **Nginx Plus** (paid)
  - No built-in GUI — configuration is file-based
  - Limited advanced traffic-shaping features compared to HAProxy (e.g., sophisticated queuing)

Nginx vs HAProxy

Feature	Nginx	HAProxy
Primary Role	Web server, reverse proxy, load balancer	Dedicated load balancer and proxy
Architecture	Event-driven, master process with dedicated worker processes	Event-driven, multi-threaded single process design
Configuration	Hierarchical block structure (http, server, location)	Flat, section-based configuration (global, defaults, frontend, backend)
Health Checks	Passive only in open-source version (via max_fails); Active checks require **Nginx Plus**	Active and passive health checks built-in
Load Balancing Scope	Primarily L7 (HTTP/HTTPS); L4 support via stream module	Full L4 (TCP) and L7 (HTTP) capabilities
Observability	Basic logging; advanced stats via third-party modules	Native, detailed stats page and Runtime API
Static Content	**Yes** – Excellent at serving files and caching	**No** – Not designed as a content server

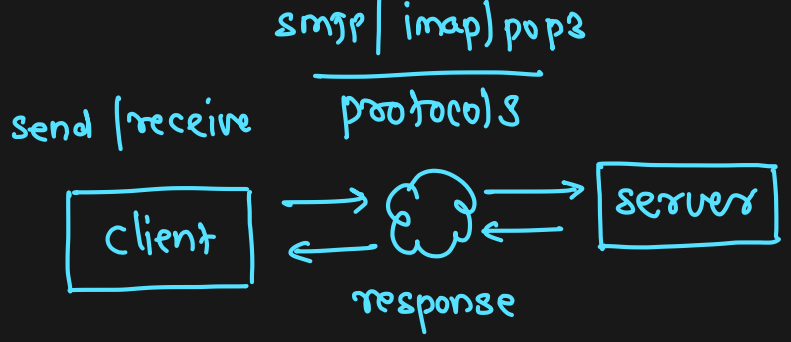
When to Choose Which

- **Choose HAProxy if**
  - Your primary goal is **high-performance load balancing** and proxying, especially for TCP traffic or at massive scale
  - You need **active, reliable health checks** to ensure high availability
  - You require **granular control** over traffic, sophisticated ACLs, and detailed real-time stats
  - You are deploying it as a dedicated component in your infrastructure
- **Choose Nginx if**
  - You need a **versatile web server** that can also serve static content, cache, and terminate SSL.
  - Your load balancing needs are for **HTTP/HTTPS traffic** and you want a simpler, well-known configuration
  - You are already using Nginx for other purposes and want to consolidate your tooling
  - You are working with Kubernetes and want to use the widely adopted **NGINX Ingress Controller**

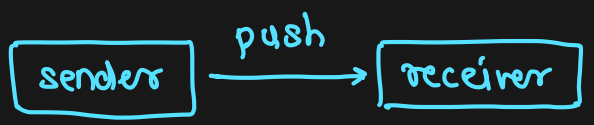
Mail Servers

Mail Services

- A **mail service** is a system that enables **sending**, **receiving**, **storing**, **and** **managing** email messages over a network
- It allows users to communicate electronically across the internet or within an organization
- Mail services typically follow a **client-server model** using **standard protocols** to ensure reliable and interoperable communication between different systems and devices
- A mail service operates using a combination of three key elements:
  - **Protocols**
  - **Servers**
    - **Clients**



Protocols



- **SMTP (Simple Mail Transfer Protocol)**
  - **Purpose:** Sending and routing email
  - **Direction:** Push (sender → server → recipient server)
  - **Ports:** 25 (server-to-server), 587 (submission), 465 (SSL)
  - **Role:** Acts as the "postal service" — moves mail from sender to recipient's server
  - **Example Commands:** HELO, MAIL FROM, RCPT TO, DATA, QUIT
- **POP3 (Post Office Protocol version 3)** → deletes message from server after downloading it
  - **Purpose:** Downloading email from the server to a single device
  - **Direction:** Pull (server → client)
  - **Ports:** 110 (plain), 995 (SSL)
  - **Behavior:** Downloads mail and usually deletes it from the server
  - **Limitation:** Not ideal for multi-device use
- **IMAP (Internet Message Access Protocol)** → leaves a copy of email on server
  - **Purpose:** Accessing and syncing email on the server
  - **Direction:** Pull (server → client)
  - **Ports:** 143 (plain), 993 (SSL)
  - **Behavior:** Keeps mail on the server; syncs across multiple devices
  - **Advantage:** Ideal for modern multi-device users

port	protocol	is secure
25	SMTP	No
465	SMTP	Yes
587	submission	Yes
110	POP3	No
995	POP3	Yes
143	IMAP	No
993	IMAP	Yes

Servers

- Servers are the **backbone** of a mail service
- They store, route, and deliver email messages. Different servers perform different roles in the email delivery chain

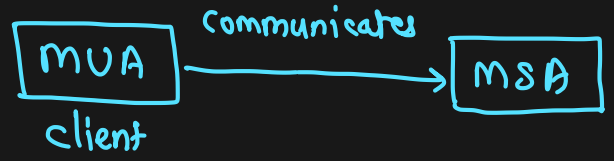
- **Mail Submission Agent (MSA)**
  - Receives outgoing mail from the client (MUA)
  - Validates and formats the message before sending
  - Usually requires authentication
- **Mail Transfer Agent (MTA)**
  - The **core routing engine** of email
  - Transfers messages between servers using SMTP
  - Performs DNS lookups (MX records) to find the recipient's server
  - **Examples:** Postfix, Sendmail, Exim, Microsoft Exchange
- **Mail Delivery Agent (MDA)**
  - Receives mail from the MTA
  - Delivers it to the recipient's **mailbox**
  - Can apply filters (spam, rules, folders)
  - **Examples:** Dovecot, Procmail

- **Mail Storage Server (Mailbox Server)**
  - Stores emails for users
  - Allows retrieval via POP3 or IMAP
  - Manages folders, attachments, and quotas
- **Relay Server**
  - Forwards email from one server to another
  - Used in corporate networks and ISPs
  - Can be **open relay** (insecure) or **closed relay** (secure)
- **Gateway Server**
  - Acts as a bridge between different mail systems
  - Performs spam filtering, antivirus scanning, and policy enforcement

Clients

- Clients are the **applications or interfaces** used by end-users to compose, send, receive, and manage email
- They interact with servers using protocols

- **Mail User Agent (MUA)**
  - The software used by the user to read and write email.
  - Communicates with MSA (for sending) and IMAP/POP3 (for receiving)



- **Types of Clients**
  - Webmail Client: Accessed via browser (Gmail, Yahoo Mail, Outlook.com)
  - Desktop Client: Installed software (Outlook, Thunderbird, Apple Mail)
  - Mobile Client: Smartphone/tablet apps (Gmail App, Outlook Mobile, Spark)
  - Command-Line Client: Text-based interface (Mutt, Pine, Alpine) → mailx
  - API-Based Client: Programmatic access (SendGrid API, Mailgun API)
- **Functions of a Client**
  - Compose and format messages
  - Add recipients (To, Cc, Bcc)
  - Attach files
  - Send via SMTP
  - Retrieve via POP3/IMAP
  - Organize into folders
  - Search and filter messages
  - Manage contacts and calendars

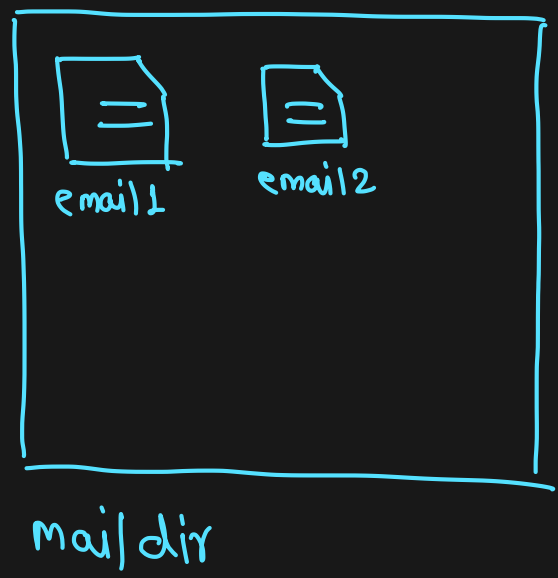
Envelope Information

Part	Example	Purpose
From	Alice(alice@gmail.com)	Sender's name and email
To	Bob(bob@company.com)	Primary recipient(s)
Cc	Carol@company.com	Carbon copy recipients
Bcc	(hidden from recipients)	Blind carbon copy
Subject	"Meeting tomorrow"	Brief summary of content
Date	Mon, Apr 13, 2026 10:30 AM	When it was sent
Attachments	meeting_notes.pdf	Files attached to the email

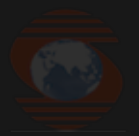
Body

Raw Email Format - RFC 5322

```
[Headers]
From: alice@gmail.com
To: bob@company.com
Subject: Meeting tomorrow
Date: Mon, 13 Apr 2026 10:30:00 -0700
Message-ID: <12345@mail.gmail.com>

[Blank line]

[Body]
Hello Bob,

Let's meet at 2pm.

Thanks,
Alice
```



Mail storage formats



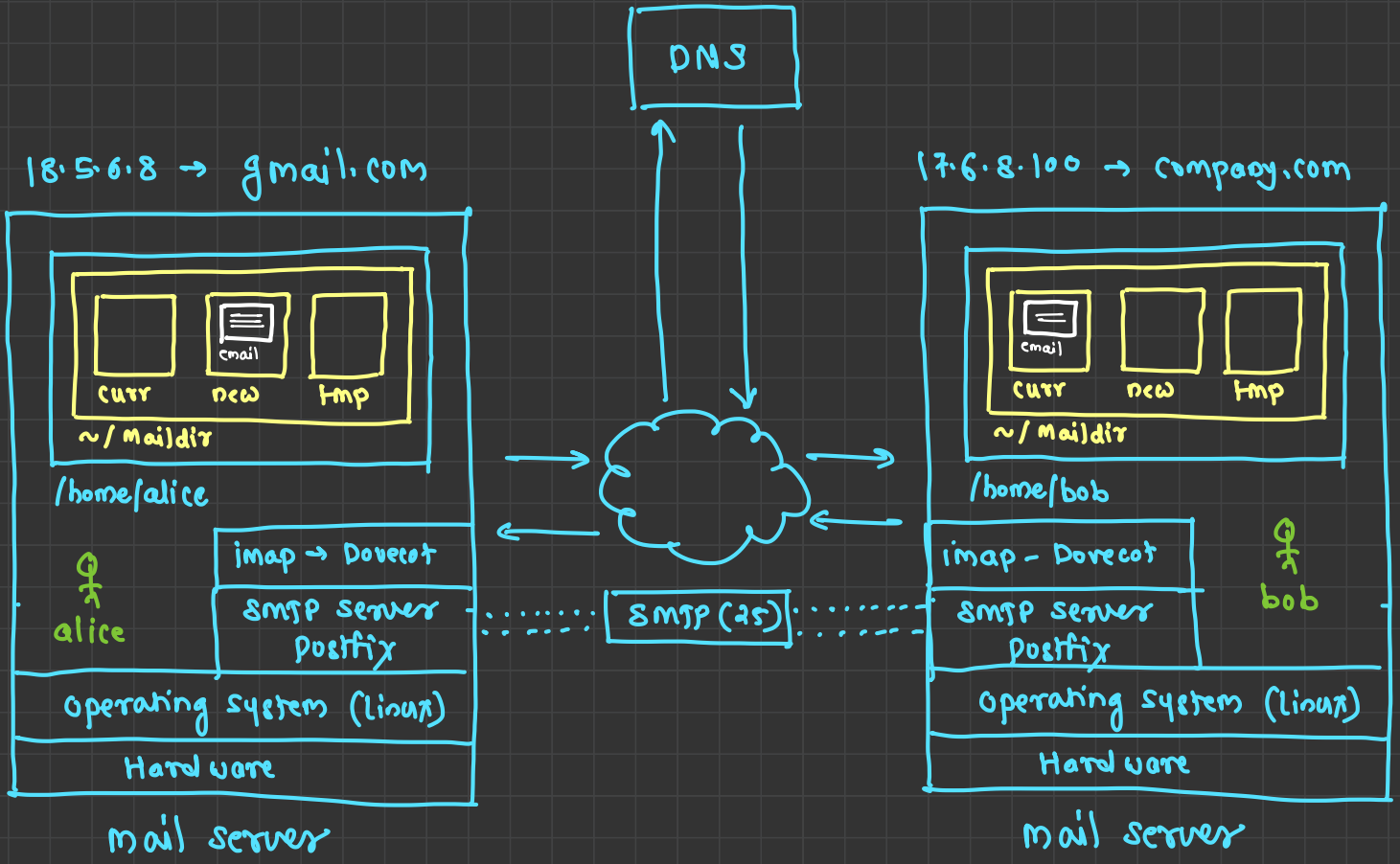
- Mail storage formats define how email messages are stored on disk by the Mail Delivery Agent (MDA) and accessed by the Mail User Agent (MUA) or IMAP/POP3 server
- The choice affects performance, reliability, and compatibility

preferred

Feature	mbox	Maildir
Messages per file	All in one file	One file per message
Locking required	Yes (flock, fcntl, dot-lock)	No (filename-based)
NFS safety	Poor	Good
Backup granularity	Whole mailbox	Per message
Deletion speed	Slow (rewrites entire file)	Fast (unlinks single file)
Corruption risk	High (single file corruption)	Low (only one message lost)

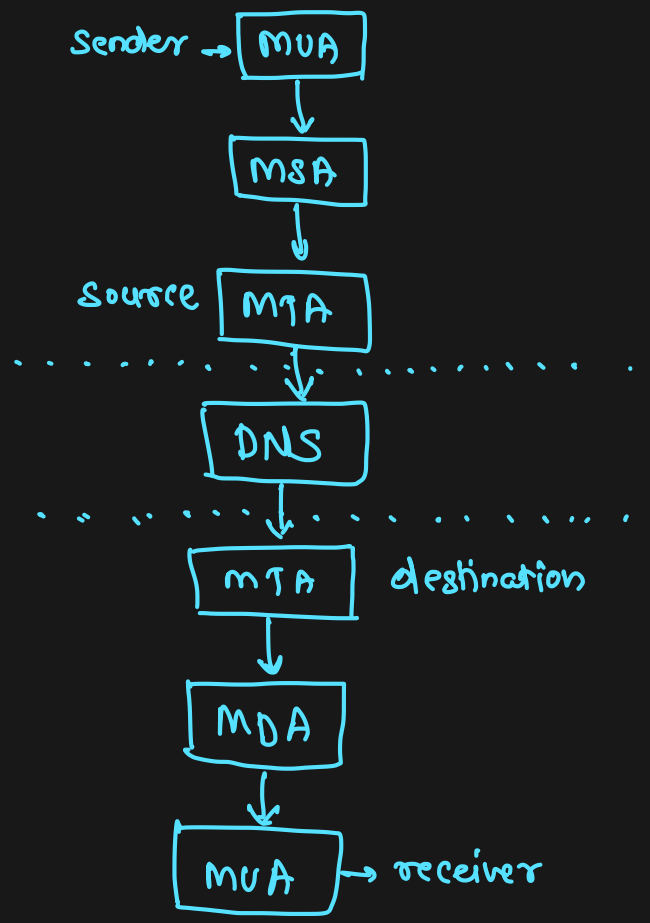
Maildir format

- Standard Maildir (qmail-style)
- Structure:
  - A directory with three subdirectories:
  - ~/Maildir/
  - ├── cur/ # Messages already seen/processed
  - ├── new/ # New messages not yet seen
  - └── tmp/ # Temporary files during delivery
- Filename format:
  - [unique] [info] , [host] : [flags]
  - E.g. 1234567890.12345_0.hostname,S=1024,W=2048:2,S
  - Flags
    - D: Draft
    - F: Flagged - Starred/Important
    - P: Passed/Forwarded
    - R: Replied
    - S: Seen (Read)
    - T: Trashed - In Trash


DNS
18.5.6.8 → gmail.com
17.6.8.100 → company.com
curr
cmail
new
tmp
~/Maildir
/home/alice
/home/bob
imap → Dovecot
imap - Dovecot
SMTP server
postfix
SMTP (25)
alice
bob
Operating system (linux)
Hardware
mail server
mail server

Working of Email

- **Step 1: User Composes and Sends**
  - Alice opens Gmail (MUA) and writes an email to bob@company.com
  - She clicks **Send**
- **Step 2: MUA to MSA (Submission)**
  - Alice's MUA connects to Gmail's **MSA** on port 587 (SMTP with STARTTLS)
  - It authenticates using Alice's credentials
  - The MSA performs initial checks:
    - Is Alice authorized to send?
    - Is the message size within limits?
    - Does it contain obvious spam?
- **Step 3: MSA to MTA (Relay)**
  - Gmail's MSA passes the message to Gmail's **MTA**
  - The MTA adds **headers**:
    - Received: from ... (trace headers)
    - Date: ...
    - Message-ID: <unique@mail.gmail.com>



Working of Email

- **Step 4: DNS Lookup (Finding the Destination)**
  - Gmail's MTA extracts the domain: company.com
  - It performs a DNS MX (Mail Exchange) query
  - MX records show priority (lower = higher priority): 10 mail.company.com
  - Then resolves mail.company.com to an IP address (A record)
- **Step 5: SMTP Conversation (Server to Server)**
  - Gmail's MTA connects to mail.company.com on port **25** (SMTP)
  - They perform an SMTP conversation
- **Step 6: MTA to MDA (Delivery)**
  - Company's MTA (Postfix) receives the message
  - It hands the message to the **MDA** (e.g., Dovecot, Procmail)
  - The MDA:
    - Applies filters (spam, antivirus)
    - Checks recipient exists (bob)
    - Writes the message to Bob's mailbox in **mbox or Maildir format**
- **Step 7: Storage**
  - The message is stored on disk:
    - **mbox format:** /var/mail/bob (single file)
    - **Maildir format:** /home/bob/Maildir/new/ (one file per message)
  - **Step 8: Retrieval by Recipient**
  - Bob opens Outlook (MUA) on his computer
  - Outlook connects to Company's **IMAP server** (Dovecot) on port 993 (IMAPS)
  - Bob authenticates
  - The server sends a list of messages in Bob's INBOX
  - Bob reads the message (server marks it as \Seen)

Common Scenarios

- **Scenario A: Both users on same server**
  - Alice@company.com → Bob@company.com
  - MTA recognizes local domain
  - Bypasses DNS/network hop
  - Directly hands to MDA
- **Scenario B: User offline (no IMAP/POP3 access)**
  - Server stores message in mailbox
  - When Bob finally connects (hours/days later)
  - IMAP server delivers stored messages
- **Scenario C: Greylisting (anti-spam)**
  - Receiving server returns temporary failure (450)
  - "I don't know you, try again later"
  - Legitimate servers retry (spammers often don't)
  - On retry (minutes later), server accepts
- **Scenario D: Bounce handling**
  - Invalid recipient (bob@company.com doesn't exist)
  - Receiving MTA returns 550 No such user
  - Sending MTA generates **DSN (Delivery Status Notification)**
  - Alice receives bounce message

SMTP

SMTP

- SMTP's origins began in 1980, building on concepts implemented on the ARPANET since 1971
- The Simple Mail Transfer Protocol (SMTP) is the de facto standard for transporting mail around the Internet
- Anyone who wants to have a mail server capable of sending and receiving mail must be able to support SMTP
- As a standards-based protocol (see RFC 5321), SMTP is well understood, platform independent, and well supported across a variety of operating systems and devices

Port	Name	Encryption	Typical Use
25	SMTP	None (STARTTLS optional)	MTA to MTA relaying
587	Submission	STARTTLS (mandatory recommended)	MUA to MTA (authenticated)
465	SMTPS	Implicit TLS (deprecated but used)	Legacy MUA submission

SMTP Commands

Command	Syntax	Purpose
HELO	HELO hostname	Initiate session (basic)
EHLO	EHLO hostname	Initiate session with extensions
MAIL FROM	MAIL FROM:<sender>	Specify sender
RCPT TO	RCPT TO:<recipient>	Specify recipient (multiple allowed)
DATA	DATA	Begin message content
RSET	RSET	Abort current transaction
VRFY	VRFY <user>	Verify user exists (often disabled)
EXPN	EXPN <list>	Expand mailing list (often disabled)
NOOP	NOOP	Do nothing (keep-alive)
QUIT	QUIT	End session

SMTP Response

Code	Meaning	Example
2xx	Success	250 OK
3xx	Additional info needed	354 Start mail input
4xx	Temporary failure (retry later)	450 Mailbox busy
5xx	Permanent failure	550 No such user

```
$ telnet mail.example.com 25
220 mail.example.com ESMTP Postfix
EHLO client.example.com
250-mail.example.com
250-PIPELINING
250-SIZE 10240000
250-STARTTLS
250-AUTH PLAIN LOGIN
250-8BITMIME
250 DSN
MAIL FROM:<alice@example.com>
250 2.1.0 Ok
RCPT TO:<bob@remote.com>
250 2.1.5 Ok
DATA
354 End data with <CR><LF>.<CR><LF>
Subject: Test

Hello Bob.
.
250 2.0.0 Ok: queued as 12345
QUIT
221 2.0.0 Bye
```

SMTP Working

- **Step 1: Composition**
  - The user writes an email in the **MUA** (e.g., Gmail, Outlook).
  - Fields filled: **From, To, Subject, Body, Attachments**.
- **Step 2: Submission**
  - The MUA sends the email to the **MSA/MTA** using SMTP.
  - Usually done on **Port 587** with authentication (SMTP-AUTH).
- **Step 3: DNS Lookup**
  - The sender's MTA extracts the recipient's domain (e.g., gmail.com).
  - It queries the **DNS server** for the **MX (Mail Exchange) record**.
  - The MX record returns the IP address of the recipient's mail server.
- **Step 4: Connection Establishment**
  - The sender's MTA opens a **TCP connection** to the recipient's MTA on **Port 25**.
  - A **three-way handshake** occurs (SYN, SYN-ACK, ACK).
  - Once connected, SMTP communication begins.

SMTP Working

- **Step 5: SMTP Handshake & Commands**

Step	Client Command	Server Response	Meaning
1	—	220 mail.server.com	Server ready
2	HELO client.com	250 Hello	Client identifies itself
3	MAIL FROM:<sender@a.com>	250 OK	Sender specified
4	RCPT TO:<receiver@b.com>	250 OK	Recipient specified
5	DATA	354 Start mail input	Ready for message body
6	*(message body)* .	250 Message accepted	Message transferred
7	QUIT	221 Bye	Session closed

- **Step 6: Message Transfer**
  - After DATA, the client sends:
    - **Header** (From, To, Subject, Date)
    - **Body** (the actual message)
    - **Attachments** (Base64 encoded)
  - The message ends with a single dot . on a new line

SMTP Working

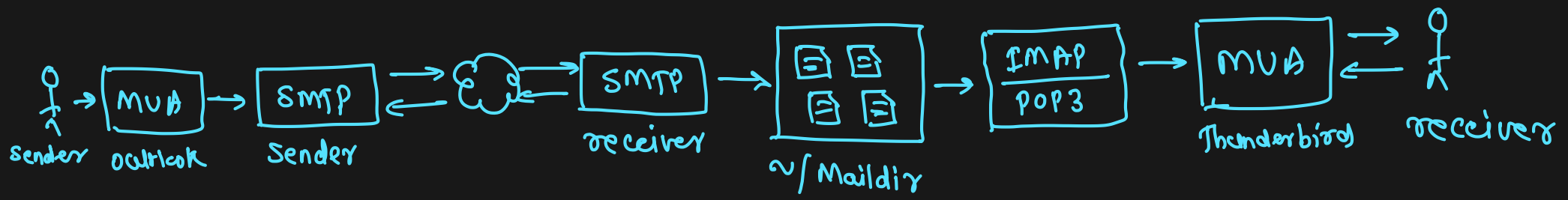
- **Step 7: Routing to Recipient's MTA**
  - If the recipient is on a different domain, the mail may pass through **multiple MTAs** (relays)
  - Each MTA forwards the message closer to the destination
- **Step 8: Delivery by MDA**
  - The recipient's MTA hands the message to the **MDA**
  - The MDA places the email in the recipient's **mailbox**
- **Step 9: Retrieval by Recipient**
  - The recipient uses **POP3** or **IMAP** to fetch the email
  - The MUA displays the message to the user

Postfix

- Install postfix
  - sudo yum install postfix
- Configuration
  - sudo vim /etc/postfix/main.cf

myhostname = <hostname>.example.com
mydomain = example.com
myorigin = $mydomain
inet_interfaces = all
mydestination = $myhostname, localhost.$mydomain, localhost, $mydomain
mynetworks = 192.168.1.0/24, 192.168.96.0/24, 127.0.0.0/8
home_mailbox = Maildir/

- Start server
  - sudo systemctl enable postfix
  - sudo systemctl start postfix
- Firewall Configuration
  - sudo firewall-cmd --add-service=smtp --permanent
  - sudo firewall-cmd --reload



IMAP/POP3

Receiving emails from Maildir

POP3

- **RFC:** 1939
- **Role:** Download emails from server to client (typically deletes from server)
- **Characteristics:** Simple, stateful, limited server-side management

Port	Encryption	Use
110	Plain (STARTTLS optional)	Standard POP3
995	Implicit TLS (POP3S)	Secure POP3

- **Limitations**
  - No folders (only one INBOX)
  - No message flags (read/unread lost after download)
  - No partial fetch (must download entire message)
  - No server-side searching
  - State management primitive

POP3 Commands

Command	Syntax	Purpose
USER	USER username	Specify username
PASS	PASS password	Send password
APOP	APOP name digest	Authenticate with MD5 (deprecated)
STAT	STAT	Mailbox status (count + size)
LIST	LIST [msg]	List message(s) and sizes
RETR	RETR msg	Retrieve message
DELE	DELE msg	Mark message for deletion
RSET	RSET	Unmark all deletions
TOP	TOP msg n	Get headers + first n lines
UIDL	UIDL [msg]	Get unique identifier list
QUIT	QUIT	End session (commits deletions)

```
$ telnet mail.example.com 110
+OK Dovecot POP3 ready
USER alice
+OK
PASS secret
+OK Logged in
STAT
+OK 2 5432
LIST
+OK 2 messages
1 3200
2 2232
.
RETR 1
+OK 3200 octets
Return-Path: <bob@example.com>
From: Bob <bob@example.com>
Subject: Hello

Message body...
.
DELE 1
+OK Marked for deletion
QUIT
+OK Logging out, messages deleted
```

IMAP (Internet Message Access Protocol)

- **RFC:** 3501 (IMAP4rev1), 9051 (current)
- **Role:** Remote mailbox management – messages stay on server
- **Characteristics:** Complex, stateful, rich feature set

Port	Encryption	Use
143	Plain (STARTTLS optional)	Standard IMAP
993	Implicit TLS (IMAPS)	Secure IMAP

POP3 vs IMAP

Feature	SMTP	POP3	IMAP
Direction	Push	Pull (download)	Pull (sync)
Message storage	Temporary (queues)	Local client only	Server + optionally local
Mailbox access	N/A	Single INBOX	Multiple folders
Message flags	N/A	No	Yes (server-side)
Partial fetch	N/A	No	Yes
Server search	N/A	No	Yes
Concurrent access	N/A	No (single client)	Yes
Offline support	N/A	Yes	Limited (requires sync)
Bandwidth	Moderate	High (full downloads)	Low (headers only possible)
Stateful	No	Yes (per session)	Yes (per session)

Protocol Selection Guide

- **Use SMTP (only)**
  - Sending notifications from applications
  - Internal mail relay between servers
- **Use POP3 when**
  - Single device only
  - Limited server storage
  - Always-online not required
  - Simple requirements
- **Use IMAP when**
  - Multiple devices (phone, laptop, webmail)
  - Need server-side folders and flags
  - Collaboration (shared mailboxes)
  - Low bandwidth optimization needed
- **Modern standard stack**
  - **Submission:** SMTP over TLS (port 587)
  - **Relay:** SMTP with STARTTLS (port 25)
  - **Retrieval:** IMAP over TLS (port 993)

Dovecot

- Installation
  - sudo yum install dovecot

- Configuration 1
  - sudo vim /etc/dovecot/dovecot.conf

```
protocols = imap pop3 lmtp submission
listen = *, ::
```

- Configuration 2
  - sudo vim /etc/dovecot/conf.d/10-mail.conf

```
mail_location = maildir:~/Maildir
```

- Configuration 3
  - sudo vim /etc/dovecot/conf.d/10-auth.conf

```
disable_plaintext_auth = no
auth_mechanisms = plain login
```

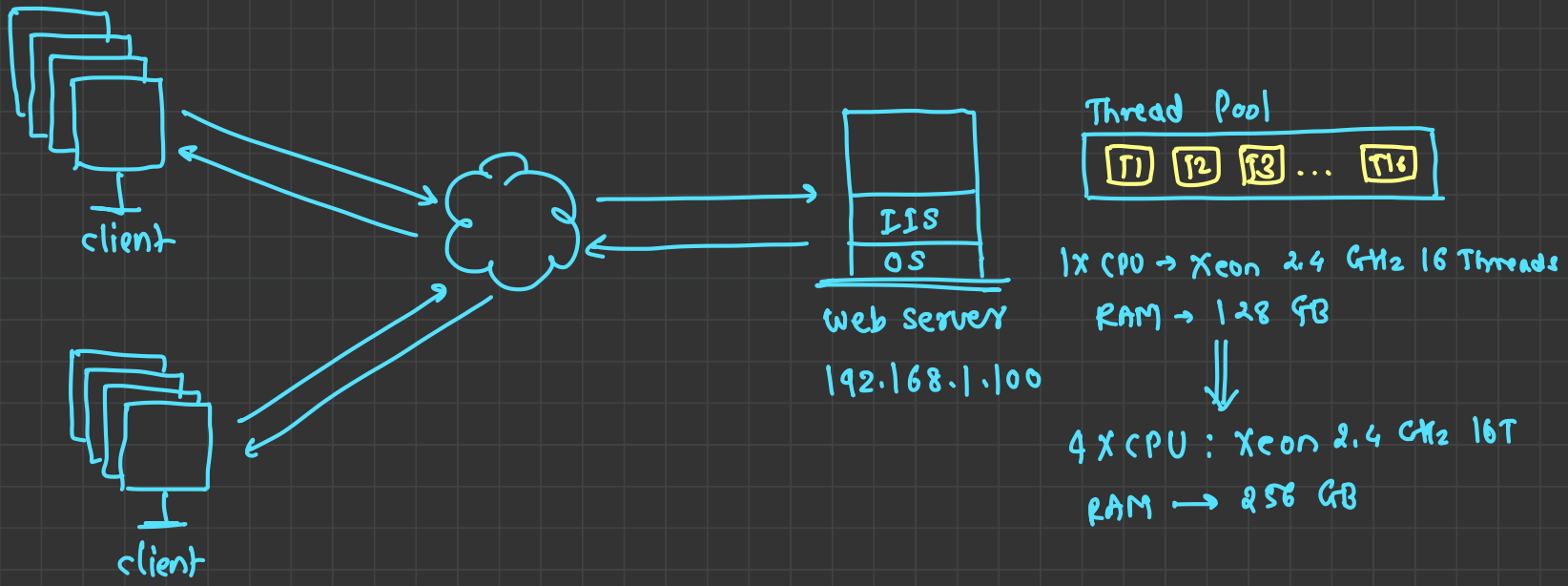
- Configuration 4
  - sudo vim /etc/dovecot/conf.d/10-master.conf

```
unix_listener /var/spool/postfix/private/auth {
  mode = 0666
  user = postfix
  group = postfix
}
```

- Start the server
  - systemctl enable dovecot
  - systemctl start dovecot

- Firewall Configuration
  - sudo firewall-cmd --add-service=imap --permanent
  - sudo firewall-cmd --add-service=pop3 --permanent
  - sudo firewall-cmd –reload

Scalability


client
client
IIS
OS
Web server
192.168.1.100
Thread Pool
T1
T2
T3
...
T16
1X CPU → Xeon 2.4 GHz 16 Threads
RAM → 128 GB
4 X CPU : Xeon 2.4 GHz 16T
RAM → 256 GB

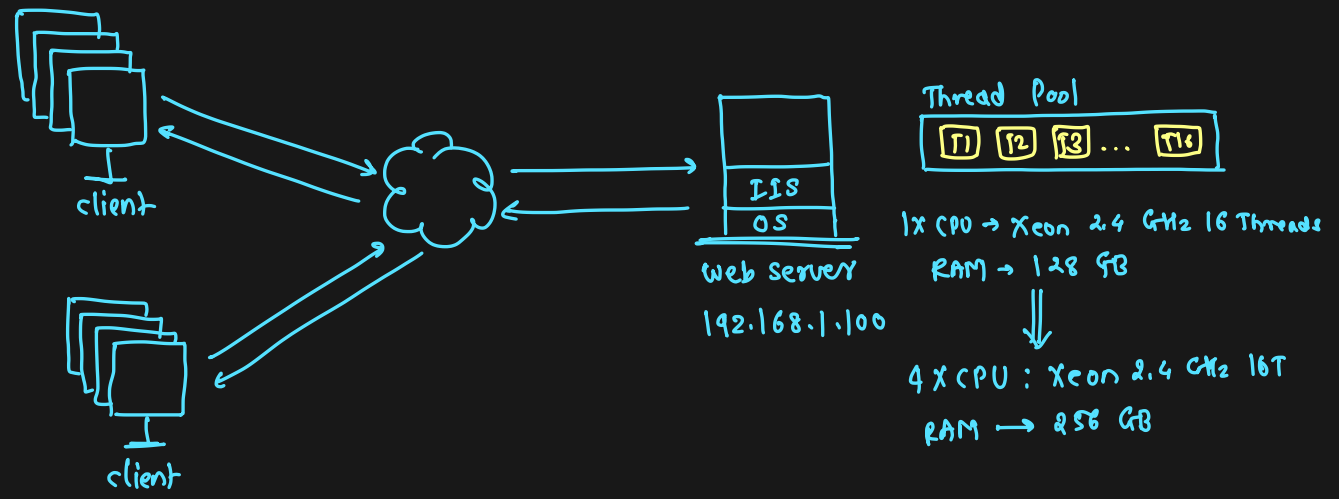
Scalability

- The ability of a computer application or product (hardware or software) to continue to function well when it (or its context) is changed in size or volume in order to meet a user need. Typically, the rescaling is to a larger size or volume
- The rescaling can be of the product itself (for example, a line of computer systems of different sizes in terms of storage, RAM, and so forth) or in the scalable object's movement to a new context (for example, a new operating system)
- It is the ability not only to function well in the rescaled situation, but to actually take full advantage of it
- For example, an application program would be scalable if it could be moved from a smaller to a larger operating system and take full advantage of the larger operating system in terms of performance (user response time and so forth) and the larger number of users that could be handled

types
- vertical
- Horizontal

Vertical Scaling

- Vertical scaling (or "scaling up") refers to adding more hardware to an existing machine so that you run the same workload on better specs
- For example, if a server requires more processing power, vertically scaling the device would mean upgrading its CPU. Other examples of vertical scaling include:
  - Adding more RAM to a machine
  - Upgrading a hard disk drive to an SSD with more storage
  - Improving network speed
- This type of scaling was the go-to option back when we had monolithic apps that had to run on a single server
- Nowadays, vertical scaling is a common choice for small-to-mid-size companies that see a noticeable difference in performance or UX from an upgraded machine

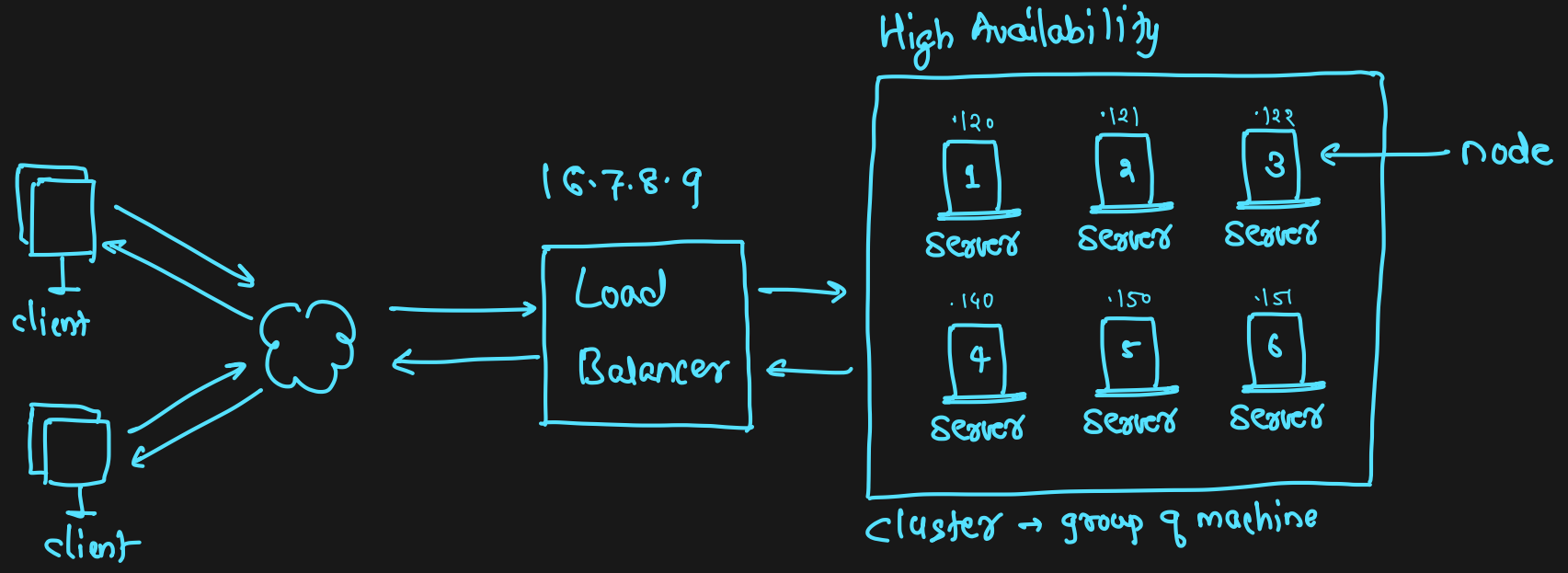


Vertical Scaling

- Advantages
  - Fast response times as there are no synchronized nodes that communicate with each other
  - Upgrading specific components costs less than purchasing a new server
  - You need less space in a server room to scale vertically than horizontally (cooling and power bills are also lower).
  - Everything runs on a single node, which leads to better data consistency and integrity
  - Easy to implement as there is no need for load balancing or virtualization
  - You do not add any extra complexity to day-to-day IT management
- Disadvantages
  - There's a limit to how much a single machine can scale since each device has a threshold for RAM, storage, and processing power
  - Higher possibility for service downtime as you rely on one machine that acts as a single point of failure
  - You must take the machine offline to perform the upgrade → downtime
  - There is always a percentage of computing resources you are not using actively
  - Increased risk of permanent data loss due to hosting everything on a single servers

Horizontal Scaling

- Horizontal scaling (or "scaling out") refers to adding more nodes to a system's resource pool to meet the increased demand.
- You do not improve the specifications of the existing machine—instead, you add more same-size servers to the cluster and share the workload across more devices
- Scaling out enables a sysadmin to combine the power of multiple machines to meet the current load demands
- Nodes do not have to be in the same data center, they do not have to be in the same region even



Horizontal Scaling

- Advantages
  - Systems that scale out have no scalability cap. The only limits to how much you scale horizontally are the budget and data center space
  - Distributing data and operations among nodes increases fault tolerance. If one of the servers goes down, the rest of the system keeps services online and maintains availability
  - The system does not go offline during the scaling process
  - Systems that scale out also enable you to reduce capacity seamlessly, so you're not stuck always paying for peak demand
  - A skilled team knows how to set up a system that scales out automatically (in both ways)
  - Sysadmins get to distribute data backups across several nodes
- Disadvantages
  - You must add software for load balancing and ensure nodes synchronize effectively. This setup is challenging for less experienced teams
  - Day-to-day server management gets more complex the more nodes you add to a system
  - Data consistency is a concern due to cross-server communication
  - Node-to-node communication slows down performance in a poorly optimized cluster
  - Servers still encounter hardware limits if machines are too small
  - Each new server requires more space, cooling, and power, so the costs of scaling out quickly spiral out of control
  - More nodes create a larger attack surface and lead to more complex incident response planning

Load Balancer

Load Balancer

cluster of nodes

- A **load balancer** is a system (hardware, software, or cloud service) that sits between **clients** and a **pool of servers** and distributes incoming network traffic across those servers
- Think of it as a **traffic cop** at a busy intersection
- Instead of every car (client request) piling onto one road (server), the officer directs each car to a different lane (server) so no single road gets overwhelmed

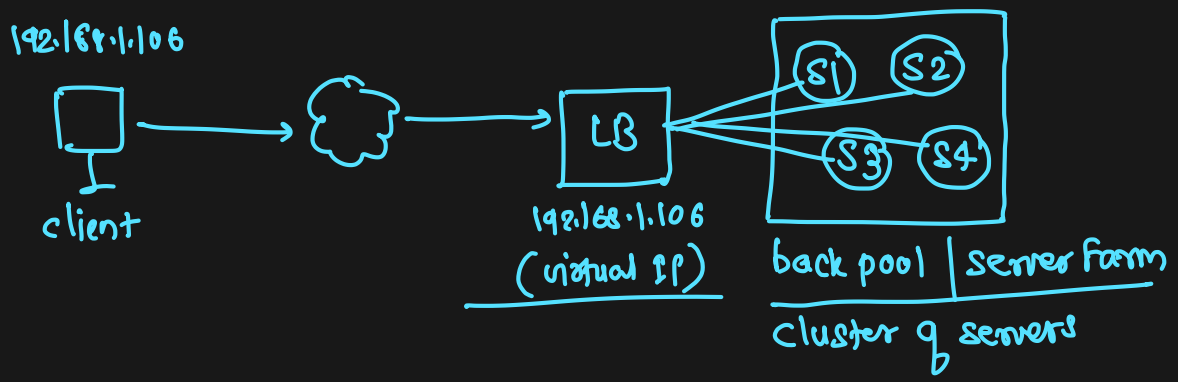
  - load balancer checks health of all the nodes by sending a heart beat signal
    - network packet

Why Load Balancer

- **Scalability (Handling More Traffic)**
  - A single server has finite CPU, RAM, and network capacity
  - When traffic grows, one machine cannot keep up
  - A load balancer lets you add more servers (**scale out**) instead of buying a bigger one (**scale up**), which is cheaper and practically unlimited → horizontal scaling
- **High Availability (Avoiding Downtime)**
  - If one server crashes, the load balancer detects the failure and stops sending traffic to it
  - Users experience no interruption because the remaining servers absorb the load. This eliminates the **single point of failure**
- **Performance & Efficiency**
  - By spreading requests evenly, no server is overworked while others sit idle
  - Response times stay consistent, and hardware is used efficiently

Components of Load Balancer

- **Virtual IP (VIP) / Cluster IP:** The single public-facing IP address clients connect to. Owned by the load balancer, not any individual server
- **Backend Pool / Server Farm:** The group of real servers that actually process the requests
- **Health Probes / Heartbeats:** Periodic checks the load balancer performs to verify each server is alive and responding
- **Load Balancing Algorithm:** The rule that decides which server gets the next request
- **Listener / Port Rules:** Defines which ports and protocols (TCP/UDP) the load balancer accepts and how to handle them
- **Session Persistence (Affinity):** Ensures a client keeps talking to the same server when needed → stateful

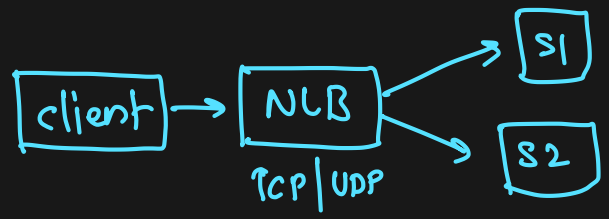


How a Load Balancer Works

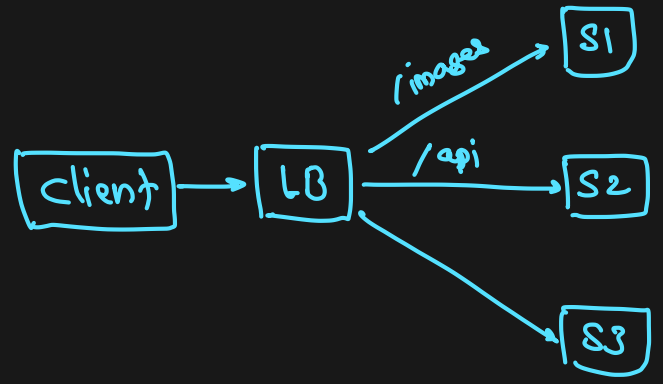
- **Client sends a request** to the VIP (e.g., 192.168.1.100:80)
- **The load balancer receives it** and consults its algorithm
- **It picks a backend server** based on health, load, and affinity rules
- **It forwards the request** to that server (or rewrites the destination)
- **The server processes the request** and returns a response
- **The load balancer relays the response** back to the client
- **Health probes run continuously** so failed servers are removed from rotation

Types of Load Balancers

- **Layer 4 (L4) — Transport Layer**
  - Works with **TCP and UDP** — IP addresses and ports only
  - Does **not** inspect the content of packets (no URLs, no cookies)
  - **Extremely fast** because it just forwards packets
  - Cannot make smart decisions based on application data
  - **Examples:** Windows NLB, AWS NLB, F5 LTM (in L4 mode), HAProxy (TCP mode)



- **Layer 7 (L7) — Application Layer**
  - Works with **HTTP, HTTPS, FTP, DNS**, etc
  - Can **inspect content**: URL paths, headers, cookies, hostnames
  - Can route /images to one server and /api to another
  - Can terminate SSL, rewrite URLs, compress responses
  - **Slower** than L4 due to deeper inspection, but far smarter
  - **Examples:** IIS ARR, NGINX, HAProxy (HTTP mode), AWS ALB, Azure App Gateway

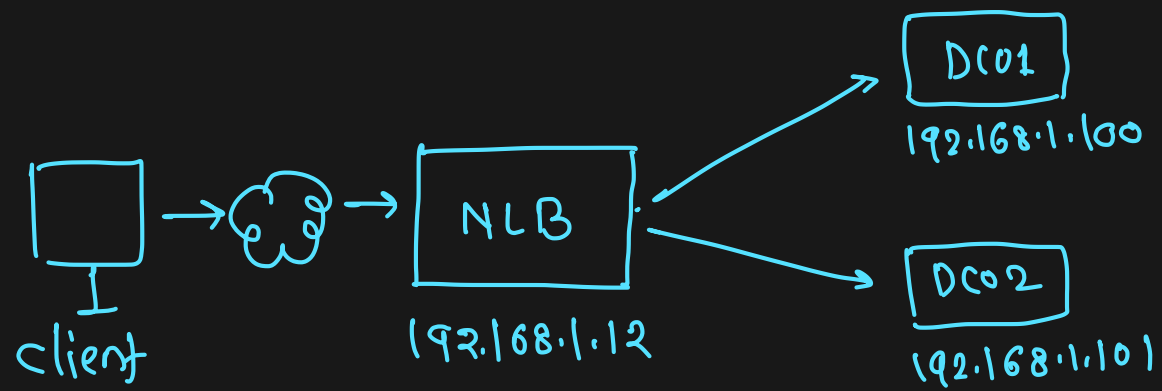


Load Balancing Algorithms

- Round Robin
  - Sends requests to servers in sequential order (1→2→3→1→2→3)
  - Servers with equal capacity; stateless apps
- Least Connections
  - Sends to the server with the fewest active connections
  - Apps with long-lived sessions
- Least Response Time
  - Sends to the fastest-responding server
  - Latency-sensitive apps
- IP Hash
  - Hashes client IP to always pick the same server
  - Session persistence without cookies
- Weighted
  - Servers with more capacity get more requests
  - Mixed hardware in a pool
- Random
  - Picks any server randomly
  - Simple, uniform environments

Session Persistence

- Some applications store user state in memory on the server (e.g., a shopping cart).
- If the next request goes to a different server, the session is lost
- Affinity solves this by ensuring a client always reaches the same server
  - None: Any server can handle any request. Best for stateless apps.
  - Single (Source IP): A client IP always maps to the same server.
  - Network (Class C): A client's entire /24 subnet maps to the same server.
- Trade-off: Affinity reduces even distribution and can create hotspots if one client sends heavy traffic.



Network Load Balancer

- Network Load Balancing (NLB) is a Windows Server software feature that distributes incoming TCP/IP traffic across multiple servers (a cluster) to ensure high availability and scalability
- From the client's perspective, the cluster appears as a single virtual server with one IP address
- NLB operates at **Layer 4 (Transport Layer)** of the OSI model, balancing TCP and UDP traffic
- NLB uses a distributed filtering algorithm on each host, meaning **no central dispatcher** exists
- All hosts in the cluster simultaneously receive incoming packets and independently decide whether to accept or discard them based on the client's IP and port mapping

Features & Concepts

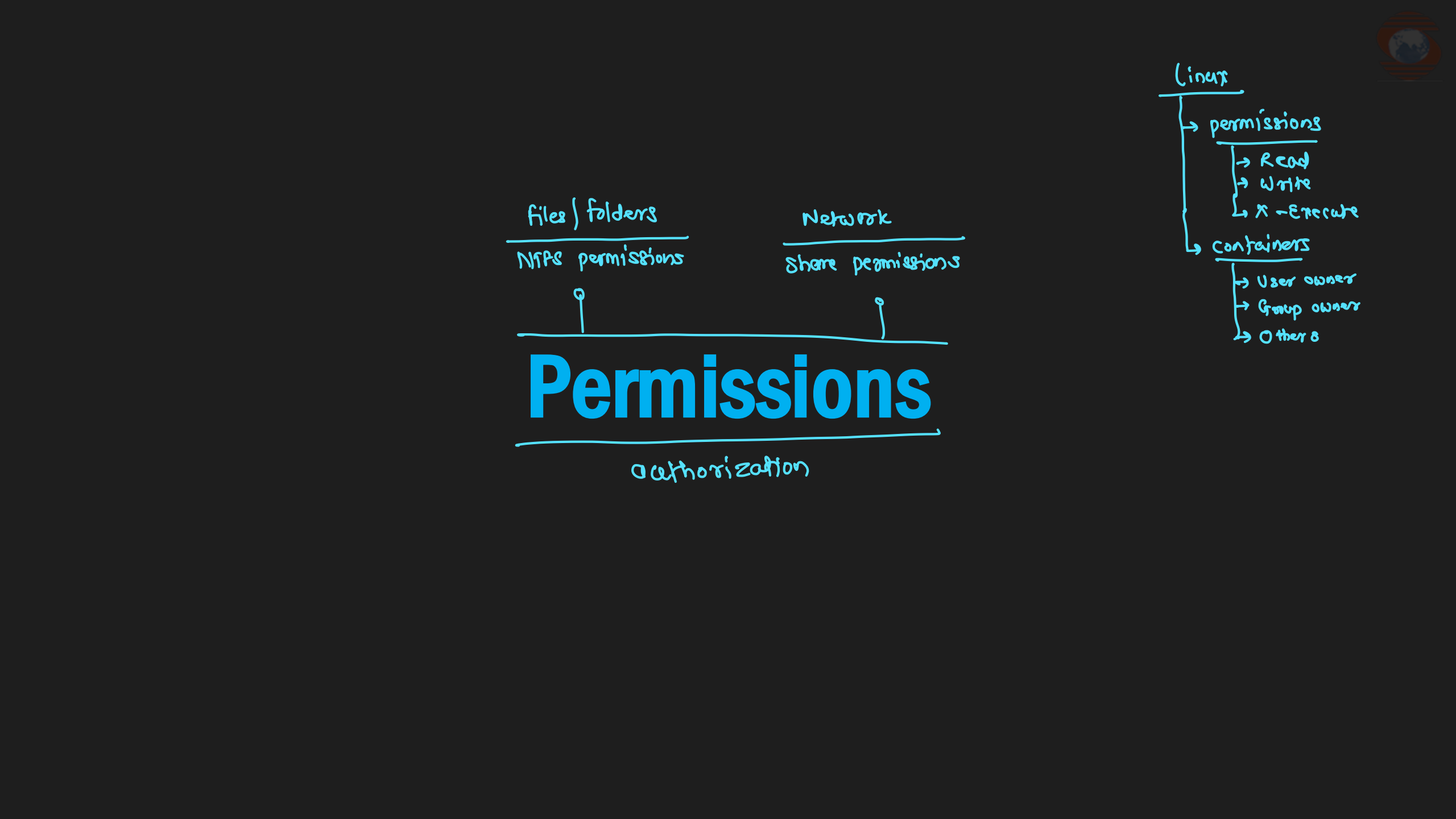
- **Cluster IP (Virtual IP)**
  - The shared IP address that clients use to connect. It is assigned to all hosts in the cluster
- **Convergence**
  - When a host fails or is added, the remaining hosts exchange heartbeat messages (typically within 10 seconds) to determine a new load distribution
  - Traffic is then redistributed
- **Port Rules**
  - Define which TCP/UDP ports are load-balanced, blocked, or handled by a single host (default host)
  - This allows fine-grained control over specific services like HTTP (port 80) or HTTPS (443)
- **Affinity (Session Persistence)**
  - Controls whether a client's requests are always directed to the same host
    - None: No affinity; any host can respond
    - Single: A client IP always maps to the same host
    - Network: A client's /24 subnet maps to the same host
- **Operation Modes**
  - Unicast: All hosts share one MAC address. Simple but can cause network switch flooding
  - Multicast: Each host keeps its unique MAC but also accepts the cluster's multicast MAC. Requires proper switch configuration but scales better

Configuration

- **Step 1: Install the NLB Feature**
  - Open **Server Manager**.
  - Click **Add roles and features**.
  - Click **Next** through the wizard until you reach **Features**.
  - Check the box for **Network Load Balancing**.
  - Click **Install** (no reboot required).
- **Step 2: Create the NLB Cluster**
  - In Server Manager, click **Tools > Network Load Balancing Manager**.
  - Right-click **Network Load Balancing Clusters** and select **New Cluster**.
  - **Host:** Type the name or IP of the **first** server. Click **Connect**.
  - Select the network interface that will handle cluster traffic (e.g., "Ethernet"). Click **Next**.
  - **Host Parameters:** Leave the **Priority** as 1 for the first host. Click **Next**.
  - **Cluster IP Addresses:** Click **Add**. Enter your virtual Cluster IP (e.g., 192.168.1.100) and Subnet Mask. Click OK, then Next.
  - **Cluster Parameters:**
    - **Full Internet Name:** Enter a name (e.g., webfarm.contoso.com).
    - **Operation Mode:** Select **Multicast** (recommended for better switch compatibility) or Unicast. Click **Next**.
  - **Port Rules:** You can use the default rule (all ports) or edit it. For a web server, click **Edit** and set the **Port Range** to 80 (and add another for 443). Set **Affinity** based on your app (usually "Single" for web sessions). Click **Finish**

Configuration

- **Step 3: Add Additional Servers**
  - In the NLB Manager, right-click the newly created cluster (e.g., webfarm.contoso.com) and select **Add Host to Cluster**.
  - Type the name/IP of the **second** server, click **Connect**, and select the interface.
  - **Host Parameters:** This host will automatically get the next available **Priority** (e.g., 2). Click **Next** and **Finish**.
  - **Verify:** The second host will appear. The status icon will turn **green** when it is successfully converged.
- **Step 4: Verification & Monitoring**
  - **Client Test**
    - Open a browser on a client machine and navigate to the **Cluster IP**.
    - You should receive responses
    - To see which server is responding, you can put a distinct identifier (like "Server 1") on each host's default web page
  - **Monitoring**
    - In the NLB Manager, you can see the status of each host (**Converged** or **Converging**)
    - You can also right-click a host to **Start/Stop** it for maintenance, which triggers convergence and redirects traffic to the remaining host


files / folders
NTFS permissions
Network
Share permissions
Permissions
authorization
Linux
permissions
Read
Write
X - Execute
containers
User owner
Group owner
Others

Introduction

final permissions = share + NTFS permission

- Windows Server 2022 uses **two independent permission systems** that combine to control access to shared resources:
  - server message block → protocol to share files over network → built-in feature of windows
- **Share Permissions (SMB-level):** Applied only when a folder is accessed over the network
- **NTFS Permissions (File System-level):** Applied to every file and folder, regardless of whether access is local or remote
- **The Golden Rule**
  - Deny
  - When a resource is accessed remotely, **the most restrictive permission wins**
  - If Share permissions grant "Read" but NTFS permissions grant "Modify," a network user gets **Read** access only
  - A user logged on locally to the server gets **Modify**, because Share permissions do not apply locally

Share Permissions (Network-Level Control)

- Share permissions are simpler and less granular than NTFS permissions
- They apply to the folder and all its contents as a single unit
- **Share Permission Levels**
  - Read: View file and subfolder names, navigate paths, view file data and attributes
  - Change: Everything Read allows, plus: Create files/folders, modify and append data, delete files/folders
  - Full Control: Everything Change allows, plus: Change NTFS permissions, take ownership
- **Best Practices**
  - **Keep Share permissions broad**
  - Set them to **Full Control** for the group that should access the share, and use **NTFS permissions to do the real fine-grained restriction**
  - This avoids confusing "double-restriction" scenarios where Share permissions block legitimate access that NTFS intended to allow
  - For most departmental shares
    - Set Share permission to **Change** for the department group and **Full Control** for Administrators
  - Only use Share-level restrictions when you need a hard network-only ceiling (e.g., a share that must be read-only for everyone over the network, even if local NTFS allows more)

NTFS Permissions (File System-Level Control)

- NTFS permissions are the primary tool for access control → applied on the child items
- They apply locally and remotely, and they support inheritance through folder hierarchies
- Basic NTFS Permission Levels
  - Basic Permission: Effectively Grants ***
  - Read: Read & Execute, List Folder Contents, Read. → Read file contents | get listing of folder
  - Read & Execute: Read, plus the ability to run executables. → execute a file
  - Write: Create files/folders, write data, append data. → update contents of file
  - Modify: Read & Execute, Write, and Delete. → delete a file from a folder
  - Full Control: Everything, plus Change Permissions and Take Ownership. → change of ownership
  - List Folder Contents: View folder contents only (applies only to folders). → view listing of folder

Critical Rules for Permission Evaluation

- **Rule 1: Cumulative Group Membership**
  - User's effective permissions are the **sum of all Allow permissions** from their user account and all groups they belong to
  - If User A has Read and is also a member of a group with Modify, User A gets **Modify**
- **Rule 2: Explicit Deny Wins**
  - An explicit **Deny** permission overrides any Allow permission, whether that Allow comes from a user account, a group, or inheritance
- **Rule 3: Explicit Allow Beats Inherited Deny**
  - An explicit **Allow** on a child object overrides an **inherited Deny** from a parent folder
  - This is a nuance that makes troubleshooting complex
- **Rule 4: Inheritance**
  - Child files and folders inherit permissions from their parent by default
  - Inherited permissions are shown with a faded/greyed checkbox in the GUI
  - You can break inheritance and copy the inherited permissions to become explicit, or remove them entirely

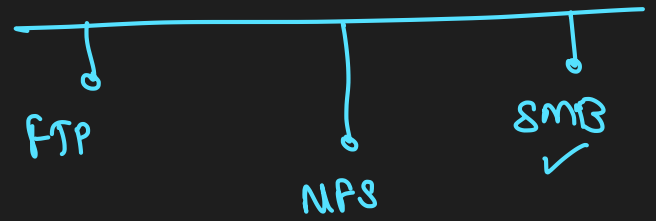
Access-Based Enumeration (ABE)

- ABE hides files and folders a user does not have **Read** access to, they simply do not appear in the directory listing
- **Scope:** Applies only to access via a share (SMB/DFS), not to local file system access
- **Effect:** It governs **visibility**, not access. If a user knows the full path, they can still attempt to open it—Share and NTFS permissions still govern whether they succeed
- **Requirement:** ABE works reliably only when NTFS permissions are set using **Active Directory groups**. Local groups (like local "Administrators") do not carry the per-user distinction ABE needs
- **Enablement:** It is configured per-share, usually when creating the SMB share in Server Manager or via Set-SmbShare -FolderEnumerationMode AccessBased

Quick Setup Checklist

- **Create the folder** on a data drive (not C:)
- **Share it** via Server Manager > File and Storage Services > Shares > New Share > SMB Quick
- **Enable ABE** on the share if users should only see what they can access
- **Set Share Permissions** to Full Control for the access group (let NTFS do the restriction)
- **Disable Inheritance** on the folder (if needed) and convert inherited permissions to explicit
- **Add Domain Local Groups** (e.g., Finance_Modify, Finance_Read) with the appropriate NTFS permissions
- **Verify with Effective Access** tab (Advanced Security Settings) to confirm a sample user's actual access

File Server



File Server

- The File Server role in Windows Server 2025 provides centralized storage and sharing of files across a network
- It encompasses file system management, the SMB (Server Message Block) protocol, security hardening, and resource management tools
- Windows Server 2025 introduces significant security enhancements to file services, particularly around SMB protocol hardening, making it the most secure Windows file server release to date

Core Components

- **File System**
  - Windows Server 2025 supports NTFS and ReFS
  - The choice depends on the intended use case—NTFS for general-purpose shares with advanced permissions, ReFS for large data sets and integrity-focused workloads
- **File Server Resource Manager (FSRM)**
  - FSRM provides quota management, file screening, and storage reporting.
  - It is an optional role service installed alongside the File Server role
- **SMB Protocol**
  - The primary network file-sharing protocol
  - Windows Server 2025 includes major security hardening and new connectivity options for SMB

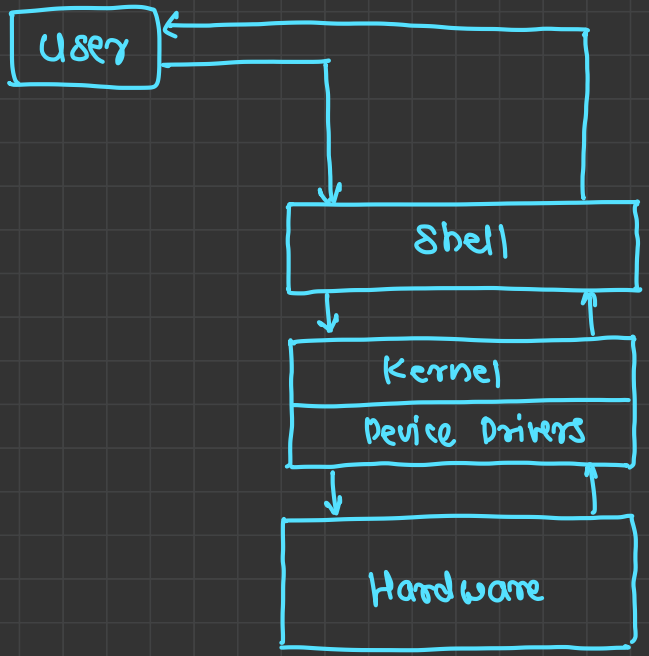
Configuration Workflow

- **Step 1: Install the Role**
  - Use Server Manager → Add Roles and Features → **File and Storage Services** → **File Server**. Optionally add **File Server Resource Manager** for quota and screening capabilities
- **Step 2: Prepare Storage**
  - Create volumes/folders on data drives (not C:). Plan folder structure before sharing
- **Step 3: Configure SMB Sharing**
  - Use Server Manager or PowerShell (New-SmbShare)
  - Set Share Permissions broadly (Full Control or Change for access group)
  - Enable **Access-Based Enumeration (ABE)** if users should only see what they can access
- **Step 4: Apply NTFS Permissions**
  - Use the **AGDLP model**: Accounts → Global groups → Domain Local groups → Permissions. Assign permissions only to Domain Local groups, never individual users
- **Step 5: Configure FSRM (Optional)**
  - **Quotas**: Soft or hard limits per user/folder (e.g., 5 GB per user)
  - **File Screening**: Block media files or executables on data shares
  - **Notifications**: Email alerts on threshold breaches
- **Step 6: Enable VSS**
  - Configure shadow copies for previous versions and backup integration

DFS

- DFS Namespaces provide a unified namespace across multiple servers
- DFS Replication synchronizes data between servers
- **Key PowerShell commands**
  - New-DfsnFolder / New-DfsnFolderTarget — Create namespace folders and targets
  - New-DfsReplicationGroup — Create replication group
  - Add-DfsrConnection — Configure replication topology
  - Set-DfsrMembership — Assign content paths and primary member
- **Considerations**
  - Set staging quota (typically 16 GB per replicated folder)
  - Configure conflict and deleted quota
  - Schedule replication to avoid bandwidth saturation during business hours

Powershell



Desktop manager

Windows → Aero | metro
macos → Acqua user interface
linux → GNOME, Cinamon, xfce, KDE

Shell → parser | interpreter
understands user commands

Windows → powershell

Linux → sh → shell
bash → bourne again shell
csh → c shell
tsh → t shell

macos → zsh → z shell

What is PowerShell?

- **A Task Automation and Configuration Management Framework**
  - Consists of a command-line shell and a scripting language
- **Built on the .NET Framework (.NET Core for cross-platform)**
  - This means it works with **objects**, not just text.
- **Cross-Platform**
  - Originally Windows-only, now available on **Windows, Linux, and macOS**
- **Two Main Components**
  - **The Shell:** An interactive command-line interface (CLI)
  - **The Scripting Language:** A full-fledged language for building complex automation.
-

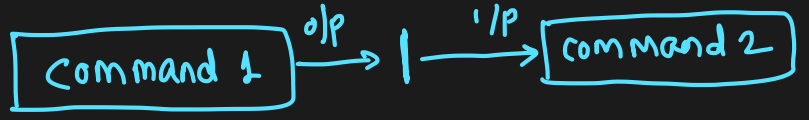
Why Automate with PowerShell?

- **Efficiency:** Automate repetitive, time-consuming tasks (e.g., user onboarding, file cleanup).
- **Consistency:** Ensure tasks are performed the same way every time, reducing human error.
- **Scalability:** Manage one server or ten thousand servers with the same script.
- **Integration:** Seamlessly manage Windows, Azure, Microsoft 365, VMware, AWS, and more.
- **Power:** Access underlying system APIs (WMI, CIM, .NET) for deep system control.

The Cmdlet

- A lightweight command used in the PowerShell environment
- **Naming Convention: Verb-Noun**
  - This makes commands predictable and discoverable
  - Examples:
    - Get-Process (Gets running processes)
    - Set-Service (Modifies a service)
    - New-Item (Creates a new file or directory)
- **Key Commands for Discovery**
  - Get-Command: Find all available cmdlets
  - Get-Help: Learn how to use a cmdlet. (e.g., Get-Help Get-Process -Examples)
  - Get-Member: See what an object can do. (e.g., Get-Process | Get-Member)

The Pipeline



- **The Problem with Older Shells:** They pass text between commands
- **The PowerShell Way:** It passes full **.NET objects** through the pipeline
- **How it Works**
  - The output of one command becomes the input for the next
  - Command-A | Command-B | Command-C
- **Example**
  - Get-Process | Where-Object {$_.CPU -gt 100} | Sort-Object CPU -Descending
  - - Get-Process gets all process objects
  - - Where-Object filters those objects, keeping only those with CPU > 100
  - - Sort-Object sorts the filtered objects by CPU usage

PowerShell Scripting Fundamentals

Adjust the Execution Policy

- Get Execution Policy

```
Get-ExecutionPolicy
```

- Set Execution Policy

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Variable Declaration

- **Basic Declaration:** Prefix with $

```
$name = "Alice"
$age = 30
```

- **Case-Insensitive:** $Name and $name refer to the same variable.
- **Typed Declaration (Strong Typing)**

```
[string]$city = "Mumbai"
[int]$count = 5
```

- **Read-Only Variables**

```
Set-Variable -Name "PI" -Value 3.14 -Option ReadOnly
```

- **Constant**

```
New-Variable -Name "MAX" -Value 100 -Option Constant
```

- **Scope Modifiers:** $global:, $script:, $local:, $private:

```
$global:appName = "MyApp"
```

Data Types

Type	Example	Description
[string]	"Hello"	Text
[int]	42	32-bit integer
[long]	9999999999	64-bit integer
[double]	3.14159	Floating point
[decimal]	10.99	High-precision decimal
[bool]	$true, $false	Boolean
[datetime]	Get-Date	Date/time
[array]	@(1,2,3)	Collection
[hashtable]	@{a=1}	Key-value pairs
[pscustomobject]	[PSCustomObject]@{}	Custom object

Arrays

- **Literal Syntax**

```
$fruits = "Apple", "Banana", "Cherry"
$numbers = 1, 2, 3, 4, 5
```

- **Using @()**

```
$empty = @()
$mixed = @(1, "two", 3.0, $true)
```

- **Strongly Typed Array**

```
[int[]]$scores = 10, 20, 30
[string[]]$names = "Alice", "Bob"
```

- **Range Operator**

```
$range = 1..10                # 1,2,3,...,10
$alpha = 'a'..'e'             # a,b,c,d,e
```

- **From Cmdlet Output:**

```
$processes = Get-Process
```

Array Operations

- **Access by Index (0-based)**

```
$fruits[0]          # Apple
$fruits[-1]         # Cherry (last item)
$fruits[0,2]        # Apple, Cherry
```

- **Slice a Range:**

```
$fruits[0..1]       # Apple, Banana
```

- **Count Items**

```
$fruits.Count       # 3
$fruits.Length      # 3
```

- **Add Items**

```
$fruits += "Date"                # Creates new array
$list = [System.Collections.ArrayList]@()
$list.Add("Mango")               # Efficient add
```

Array Operations

- **Sort**

```
$sorted = $numbers | Sort-Object
$desc   = $numbers | Sort-Object -Descending
```

- **Filter**

```
$even = $numbers | Where-Object { $_ % 2 -eq 0 }
```

- **Select / Project**

```
$names = $users | Select-Object -Property Name, Email
```

- **Search**

```
$fruits -contains "Apple"       # True
$fruits -notcontains "Grape"    # True
$fruits -like "*an*"            # Banana
```

- **Iterate**

```
$numbers | ForEach-Object { $_ * 2 }
```

Loops

- **For loop**

```
for ($i = 0; $i -lt 5; $i++) {
    Write-Host "Iteration $i"
}
```

- **Foreach**

```
$fruits = "Apple", "Banana", "Cherry"
foreach ($fruit in $fruits) {
    Write-Host "Fruit: $fruit"
}
```

- **ForEach-Object Cmdlet (pipeline-friendly)**

```
$fruits | ForEach-Object {
    Write-Host "Fruit: $_"
}
```

- **Key Difference**
  - foreach loads everything into memory first → faster for small sets.
  - ForEach-Object streams → better for large pipelines.

Functions

- **Simple Function**

```powershell
function Say-Hello {
    Write-Host "Hello, World!"
}
Say-Hello
```

- **With Parameters**

```powershell
function Greet-User {
    param(
        [string]$Name,
        [int]$Age
    )
    Write-Host "Hello $Name, you are $Age years old."
}
Greet-User -Name "Alice" -Age 30
```

Snippets

System Information Reporter

```powershell
# Computer name and current user
Write-Host "Computer Name : $env:COMPUTERNAME"
Write-Host "Logged-in User: $env:USERNAME"
Write-Host ""

# Operating System details
$os = Get-CimInstance -ClassName Win32_OperatingSystem
Write-Host "OS Name       : $($os.Caption)"
Write-Host "OS Version    : $($os.Version)"
Write-Host "Last Boot Time: $($os.LastBootUpTime)"
Write-Host ""

# CPU details
$cpu = Get-CimInstance -ClassName Win32_Processor
Write-Host "CPU           : $($cpu.Name)"
Write-Host "Cores         : $($cpu.NumberOfCores)"
Write-Host ""

# Memory details (converted to GB)
$ramGB = [math]::Round($os.TotalVisibleMemorySize / 1MB, 2)
Write-Host "Total RAM (GB): $ramGB"
Write-Host ""
```

Check Disk Space & Alert

```powershell
$threshold = 15  # Warn if free space is below 15%

Write-Host "Checking disk space..." -ForegroundColor Cyan
Write-Host ""

Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3" | ForEach-Object {
    $sizeGB = [math]::Round($_.Size / 1GB, 2)
    $freeGB = [math]::Round($_.FreeSpace / 1GB, 2)
    $percentFree = [math]::Round(($_.FreeSpace / $_.Size) * 100, 2)

    if ($percentFree -lt $threshold) {
        Write-Host "⚠ WARNING: Drive $($_.DeviceID) has only $percentFree% free ($freeGB GB of $sizeGB GB)" -ForegroundColor Red
    }
    else {
        Write-Host "✓ OK: Drive $($_.DeviceID) has $percentFree% free ($freeGB GB of $sizeGB GB)" -ForegroundColor Green
    }
}
```

List Running Services

```powershell
param(
    [string]$Filter = "*"
)

Write-Host "Running services matching '$Filter':" -ForegroundColor Cyan
Write-Host ""

Get-Service -Name $Filter |
    Where-Object { $_.Status -eq "Running" } |
    Select-Object Name, DisplayName, Status |
    Format-Table -AutoSize
```

Bulk Create AD Users from CSV

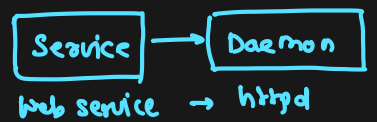
```powershell
Import-Module ActiveDirectory
$pwd = ConvertTo-SecureString "P@ssw0rd!" -AsPlainText -Force

Import-Csv C:\ad_users.csv | ForEach-Object {
    New-ADUser `
        -Name "$($_.FirstName) $($_.LastName)" `
        -SamAccountName $_.Username `
        -UserPrincipalName "$($_.Username)@contoso.com" `
        -GivenName $_.FirstName `
        -Surname $_.LastName `
        -Department $_.Department `
        -Path $_.OU `
        -AccountPassword $pwd `
        -Enabled $true
}
```

```
FirstName,LastName,Username,Department,OU
Alice,Smith,asmith,HR,"OU=HR,DC=contoso,DC=com"
Bob,Jones,bjones,IT,"OU=IT,DC=contoso,DC=com"
Charlie,Brown,cbrown,Finance,"OU=Finance,DC=contoso,DC=com"
```

Service Management

Services and Daemons

- While the terms service and daemon are sometimes used interchangeably, there are specific differences between the two
- Daemons are noninteractive programs running on the system. They do not provide direct communication with users and operate in the background.
- Services are interactive, though they too operate in the background. Users may interact with services. Services are the purpose behind server computers.
- Note that services are usually implemented as daemons and also contain an interactive component. Hence, services such as SSH will have a related daemon, such as sshd.
- Service and daemon management is an essential part of the sysadmin role
- Here are a few critical services that users rely on throughout the business day
  - Web service: Makes web pages available across the network and relies on httpd
  - SSH service: Allows secure remote administration and relies on sshd
  - NFS service: Allows remote access to exported directories and relies on several daemons, including nfsd and mountd
  - NTP service: Synchronizes time among network servers and relies on ntpd



System Initialization

- System initialization is the process that begins when the kernel first loads
- It involves the loading of the operating system and its various components, including the boot process
- System initialization is carried out by an init daemon in Linux—the "parent of all processes."
- The init daemon refers to a configuration file and initiates the processes listed in it
- This prepares the system to run the required software
- The init daemon runs continuously until the system is powered off, and programs on the system will not run without it
- On Linux, there are two main methods that initialize a system: systemd and SysVinit. Which method is active on your Linux system will affect how you manage services on that system
- Most modern Linux distributions use systemd to manage initialization
- However, some distributions, especially older ones, rely on a different initialization mechanism named System V

systemd

- The systemd software suite provides an init method for initializing a system
- It also provides tools for managing services on the system that derive from the init daemon → systemctl
- The systemd suite is now the dominant init method in modern Linux distributions, and was designed as a replacement for other methods like SysVinit
- The systemd suite offers several improvements over older methods
- For example, it supports parallelization (starting programs at the same time for quicker boot) and reduces shell overhead
- In systemd, Control Groups (cgroups) are used to track processes instead of process IDs (PIDs), which provides better isolation and categorization for processes

systemctl command

- One of the most frequent tasks for administrators is managing services and daemons
- Management includes starting, stopping, and checking the status of these applications
- It's critical to remember that Linux services and daemons acquire their settings from configuration files that are read when the system or the applications start
- Any changes to these settings occur by editing the files, which must then be reread to discover the new configurations
- In addition, administrators may choose whether a given service starts when the system boots
- Managing Services with systemctl
  - start: Start the service or daemon (not persistent)
  - stop: Stop the service or daemon (not persistent)
  - restart: Restart the service or daemon
  - enable: Set the service or daemon to start at boot (persistent)
  - disable: Set the service or daemon not to start at boot (persistent)
  - status: Display the current status of the service or daemon

systemd unit files

- Under the systemd initialization process, the term unit refers to a resource that systemd knows how to manage via daemons and configuration files
- This system is much more flexible than earlier initialization methods
- Unit files define how systemd manages the unit
- There are many systemd unit file types, with .service, .timer, .mount, and .target as the primary examples
- Stored in:
  - /etc/systemd/system: custom
  - /usr/lib/systemd/system: default
- Commands
  - Use **systemctl –t help** to get list of service units
  - Use **systemctl list-unit-files** for listing unit files
  - Use **systemctl list-units --type=service** for filtering units with type service

Unit file types

- .service: Background services
- .socket: IPC/network sockets
- .target: Group of units (like runlevels)
- .mount: Filesystem mounts
- .automount: On-demand mounts
- .timer: Scheduled jobs
- .path: File-based triggers
- .device: Hardware devices
- .swap: Swap partitions

System Targets

- A target in systemd is a group of units (services, mounts, etc.) that represents a system state
- A mode of operation (CLI mode, GUI mode, rescue mode)
- A replacement for traditional runlevels

Run level	Target	Description
0	poweroff.target	Shutdown the system
1	rescue.target	Single user mode
2	multi-user.target	Multi-user without networking
3	multi-user.target	CLI (without GUI) with networking
4	multi-user.target	Reserved
5	graphical.target	With GUI mode
6	reboot.target	Reboot the system

- Check current target: sudo systemctl get-default
- Set a target: sudo systemctl set-default <target>

Modifying systemd Service configurations

- Default system provided unit files are located in /usr/lib/systemd/system
- Custom unit files are located in /etc/systemd/system
- Run-time automatically generated unit files are located in /run/systemd
- While modifying a unit file, do NOT edit the file in /usr/lib/systemd/system, instead create a custom file in /etc/system/system that is used as an overlay file
- Better option is to use **systemctl edit unit.service** command to edit unit files
- Use **systemctl show** command to show the available parameters
- Use after modifying the parameters

Service Unit File Structure

- Example: /etc/systemd/system/myapp.service

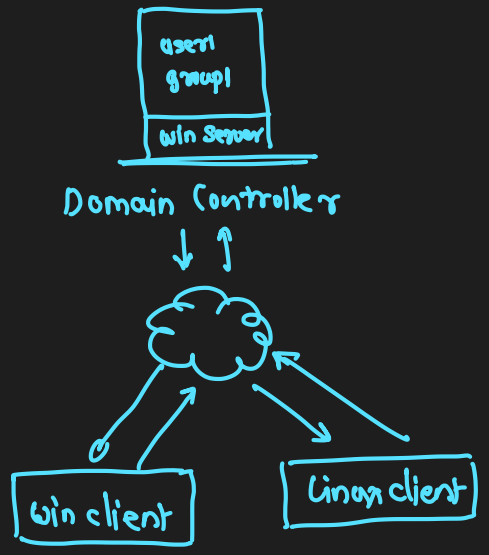
```
[Unit]
description=My application
After=network.target

[Service]
ExecStart=/usr/bin/python3 /home/vagrant/myapp.py
Restart=always
User=vagrant
```

- Reload the daemon
  - sudo systemctl daemon-reload
- Debug the service
  - sudo journalctl –u myapp

Mount Unit File Structure

- Example: /etc/systemd/system/mydisk.mount

```
[Unit]
Description=auto mount mydisk
After=local-fs.target

[Mount]
What=/dev/nvme0n2p1
Where=/mydisk
Type=ext4
Options=defaults

[Install]
WantedBy=multi-user.target
```

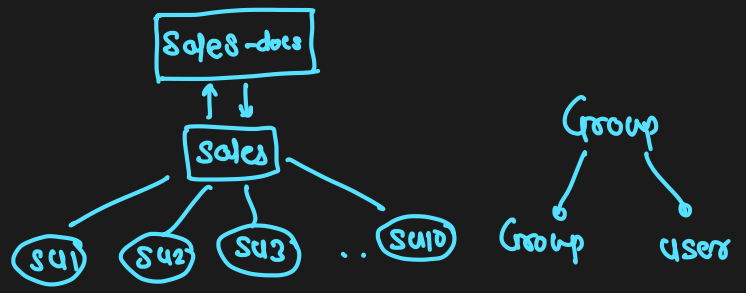


Users and Group Management

Introduction

- User and group management is the day-to-day operational core of Active Directory administration
- Every person who logs into the network, every computer that joins the domain, and every service that authenticates requires a user or computer account
- Groups then tie these accounts to permissions and policies
- **Core Objectives**
  - **Identity Management:** Create, modify, and delete user identities → authentication
  - **Access Control:** Determine who can access what resources → authorization → permissions | Group policy
  - **Administrative Efficiency:** Use groups to assign permissions at scale
  - **Security:** Enforce least privilege, monitor privileged accounts
  - **Lifecycle Management:** Handle onboarding, role changes, and offboarding

default security principle → implicit deny



Users

User object

- A user object represents a **security principal** that can:
  - Authenticate to the domain (logon) → authentication
  - Be assigned permissions to resources → authorization
  - Be a member of groups → group member
  - Own files and objects → owner
  - Have a mailbox (if Exchange is deployed)
- Types of users

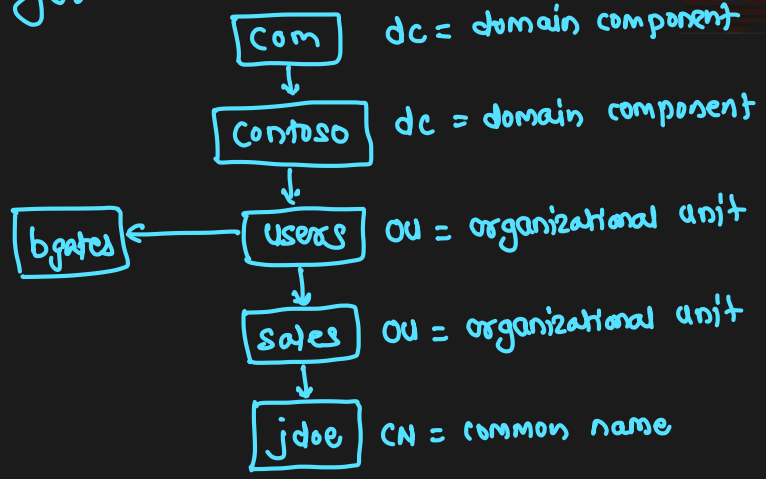
Type	Purpose	Characteristics
✓ **Regular User** → human user	Standard employee/end-user	Limited permissions; interactive logon
✓ **Administrative User** → superuser (root)	IT staff with elevated rights	Member of admin groups; separate admin account
✓ **Service Account** → system user	Used by applications/services	Non-interactive; often no password expiry
gMSA (Group Managed Service Account)	Modern service account	Automatic password management; ideal for services
Managed Service Account (MSA)	Single-server service account	Automatic password; limited to one server
Shared Account → disabled	Multiple users (discouraged)	Hard to audit; security risk
Guest Account → disable	Temporary/limited access	Disabled by default; high risk if enabled

User Account Attributes

- **General Attributes**
  - First name, Last name, Display name
  - Description, Office, Telephone, Email
  - Web page, Notes
- **Account Attributes**
  - User logon name (UPN): user@domain.com
  - User logon name (pre-Windows 2000): DOMAIN\user
  - Logon hours
  - Logon workstations
  - Account options (password never expires, etc.)
  - Account expiry date
- **Profile Attributes**
  - Profile path
  - Logon script
  - Home folder (local or network)
  - Remote Desktop Services profile
- **Organization Attributes**
  - Title, Department, Company
  - Manager
  - Direct reports
- **Address Attributes**
  - Street, City, State, ZIP
  - Country/region
- **Telephones Attributes**
  - Home, Pager, Mobile, Fax, IP phone
- **Member Of** **
  - Group memberships
- **Dial-in**
  - Remote access permission
  - Callback options
  - Static IP address

Creating User Accounts

SAM → Security Account Manager




```powershell
# Basic user creation
New-ADUser -Name "John Doe" `
    -GivenName "John" `
    -Surname "Doe" `
    -SamAccountName "jdoe" `
    -UserPrincipalName "jdoe@contoso.com" `
    -Path "OU=Sales,OU=Users,DC=contoso,DC=com" `
    -AccountPassword (ConvertTo-SecureString "P@ssw0rd123!" -AsPlainText -Force) `
    -Enabled $true `
    -ChangePasswordAtLogon $true `
    -Department "Sales" `
    -Title "Sales Representative" `
    -Manager "cn=Jane Smith,ou=Managers,dc=contoso,dc=com"
```

→ first name

→ last name

→ unique name used to login

Managing User Accounts

Comparison operators
- -eq → ==
- -ne → !=
- -gt → >
- -lt → <
- -ge → >=
- -le → <=
- -like → like

```powershell
# Find user
Get-ADUser -Identity "jdoe" -Properties *

# Find users by filter
Get-ADUser -Filter {Department -eq "Sales"} -Properties Department, Title

# Search by name
Get-ADUser -Filter {Name -like "*John*"}

# Reset password
Set-ADAccountPassword -Identity "jdoe" `
    -Reset `
    -NewPassword (ConvertTo-SecureString "NewP@ssw0rd!" -AsPlainText -Force)

# Force password change at next logon
Set-ADUser -Identity "jdoe" -ChangePasswordAtLogon $true

# Unlock account
Unlock-ADAccount -Identity "jdoe"
```

SamAccountName | SID | objectGUID

Managing User Accounts

```powershell
# Disable account
Disable-ADAccount -Identity "jdoe"

# Enable account
Enable-ADAccount -Identity "jdoe"

# Move user to different OU
Move-ADObject -Identity "CN=John Doe,OU=Sales,DC=contoso,DC=com" `
    -TargetPath "OU=Marketing,DC=contoso,DC=com"

# Modify attributes
Set-ADUser -Identity "jdoe" -Department "Marketing" -Title "Marketing Specialist"

# Add to group
Add-ADGroupMember -Identity "Marketing Team" -Members "jdoe"

# Remove from group
Remove-ADGroupMember -Identity "Sales Team" -Members "jdoe" -Confirm:$false

# Delete user
Remove-ADUser -Identity "jdoe" -Confirm:$false
```

Bulk User Creation

- CSV format
  - FirstName,LastName,SamAccountName,UPN,Department,Title,OU,Password
  - John,Doe,jdoe,jdoe@contoso.com,Sales,Representative,"OU=Sales,DC=contoso,DC=com",P@ssw0rd123!
  - Jane,Smith,jsmith,jsmith@contoso.com,Marketing,Specialist,"OU=Marketing,DC=contoso,DC=com",P@ssw0rd123!

```powershell
Import-Csv "C:\NewUsers.csv" | ForEach-Object {
    $params = @{
        Name = "$($_.FirstName) $($_.LastName)"
        GivenName = $_.FirstName
        Surname = $_.LastName
        SamAccountName = $_.SamAccountName
        UserPrincipalName = $_.UPN
        Path = $_.OU
        Department = $_.Department
        Title = $_.Title
        AccountPassword = (ConvertTo-SecureString $_.Password -AsPlainText -Force)
        Enabled = $true
        ChangePasswordAtLogon = $true
    }
    New-ADUser @params
    Write-Host "Created user: $($_.SamAccountName)"
}
```

User Account Lifecycle Management

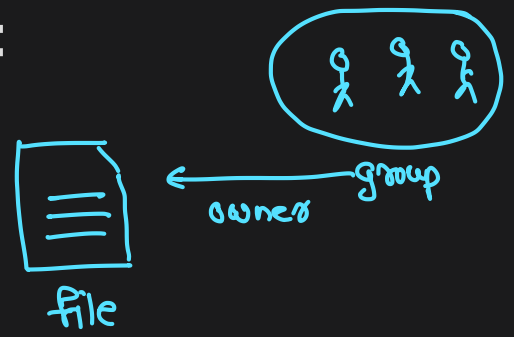
- **Onboarding**
  - Create user account (HR provides details)
  - Assign to appropriate groups
  - Set initial password; force change at first logon
  - Configure mailbox (if Exchange)
  - Set up home folder/profile
  - Provide credentials to user securely
- **Offboarding**
  - Disable account immediately (don't delete)
  - Remove from all groups (except Domain Users)
  - Reset password to random value
  - Hide from address lists (Exchange)
  - Set out-of-office (Exchange)
  - Forward email (if needed)
  - Document and archive
  - Delete after retention period (e.g., 30-90 days)
- **Role Changes**
  - Update group memberships (remove old, add new)
  - Update attributes (department, title, manager)
  - Review permissions
  - Update mailbox/delegation if needed
- **Best Practices**
  - **Never delete immediately:** Disable first; delete after retention period
  - **Use AD Recycle Bin:** Enable for object recovery
  - **Document everything:** Who, what, when, why
  - **Automate where possible:** PowerShell scripts, HR system integration
  - **Review regularly:** Quarterly access reviews

Groups

Group of security principals

What is a Group?

- A group is a **collection of security principals** (users, computers, groups) used to:
  - Simplify permission assignment
  - Simplify policy application
  - Enable email distribution (distribution groups)
  - Organize objects for management



- **Group Types**

→ files | folders | computers

Type	Purpose	Can be Used for Permissions?
Security Group	Assign permissions to resources	Yes
Distribution Group	Email distribution lists	No (only email)

Group Scopes

Scope	Can Contain	Can Be Used In
Domain Local	Users, computers, global groups, universal groups from any domain	Permissions on resources in the same domain
Global	Users, computers from the same domain	Permissions on resources in any domain
Universal	Users, computers, global groups from any domain	Permissions on resources in any domain

- **Group Scope Best Practices**
  - **AGDLP (Accounts, Global, Domain Local, Permissions)**
    - **Accounts:** Add users to **G**lobal groups (by role/department)
    - **Global groups:** Add to **D**omain **L**ocal groups (by resource)
    - **Domain Local groups:** Assign **P**ermissions to resources
  - **Example**
    - **User:** John Doe (Sales)
    - **Global Group:** Sales Team (contains all sales users)
    - **Domain Local Group:** Sales Shared Folder Access (contains Sales Team global group)
    - **Permission:** Sales Shared Folder Access has Modify on \\server\sales

Built-in Domain Local Groups

Group	Purpose
Administrators	Full control over the domain
Account Operators	Create/modify user and group accounts
Backup Operators	Backup/restore files and registry
Guests	Limited access (disabled by default)
Print Operators	Manage printers
Server Operators	Manage servers
Users	Standard user access
Performance Monitor Users	Access performance data
Performance Log Users	Manage performance logs
Remote Desktop Users	Remote Desktop access
Network Configuration Operators	Manage network settings
Incoming Forest Trust Builders	Create incoming forest trusts
Terminal Server License Servers	Manage TS licensing
Windows Authorization Access Group	Access token information
Pre-Windows 2000 Compatible Access	Legacy access

Built-in Global Groups

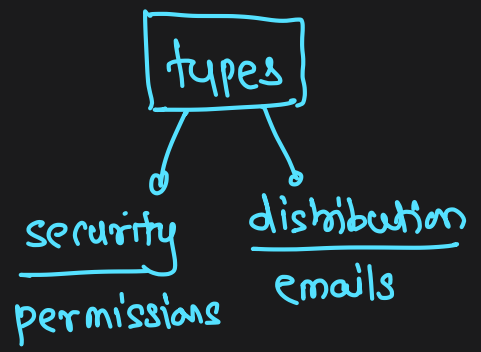
Group	Purpose
Domain Admins	Full control over the domain
Domain Users	All users in the domain
Domain Guests	Guest access
Domain Computers	All computers in the domain
Domain Controllers	All DCs in the domain
Schema Admins	Modify AD schema (forest root only)
Enterprise Admins	Full control over forest (forest root only)
Group Policy Creator Owners	Create/modify GPOs
RAS and IAS Servers	Access remote access properties
Protected Users	Extra security for privileged accounts
Enterprise Key Admins	Manage key objects (forest root)
Key Admins	Manage key objects (domain)

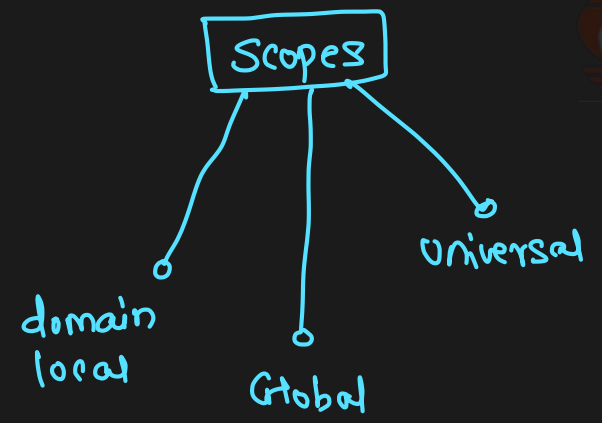
Built-in Universal Groups

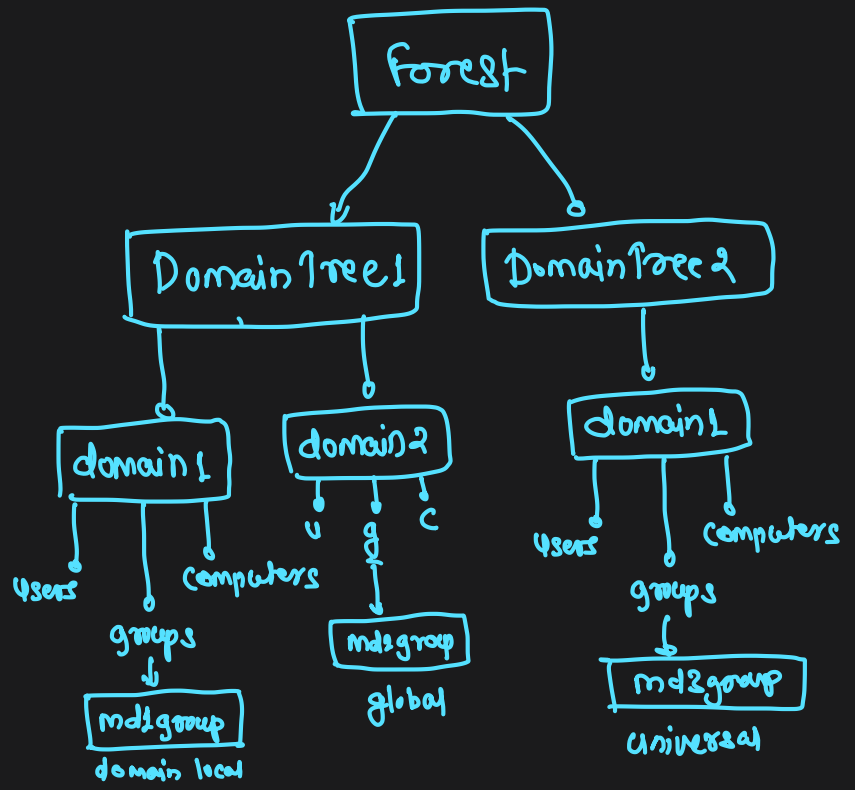
Group	Purpose
Enterprise Admins	Full control over forest (forest root only)
Schema Admins	Modify AD schema (forest root only)
Enterprise Key Admins	Manage key objects (forest root)

Creating Groups

```powershell
# Create a global security group
New-ADGroup -Name "Sales Team" `
    -SamAccountName "Sales Team" `
    -GroupCategory Security `
    -GroupScope Global `
    -Path "OU=Groups,DC=contoso,DC=com" `
    -Description "All sales department users"

# Create a domain local security group
New-ADGroup -Name "Sales Folder Access" `
    -SamAccountName "Sales Folder Access" `
    -GroupCategory Security `
    -GroupScope DomainLocal `
    -Path "OU=Groups,DC=contoso,DC=com" `
    -Description "Access to Sales shared folder"

# Create a distribution group
New-ADGroup -Name "All Company" `
    -SamAccountName "All Company" `
    -GroupCategory Distribution `
    -GroupScope Universal `
    -Path "OU=Distribution Groups,DC=contoso,DC=com"
```







Managing Groups

```powershell
# Get group info
Get-ADGroup -Identity "Sales Team" -Properties *

# Get group members
Get-ADGroupMember -Identity "Sales Team"

# Get user's groups
Get-ADUser -Identity "jdoe" -Properties MemberOf | Select-Object -ExpandProperty MemberOf

# Add member
Add-ADGroupMember -Identity "Sales Team" -Members "jdoe"

# Add multiple members
Add-ADGroupMember -Identity "Sales Team" -Members "jdoe","jsmith","bwilliams"

# Remove member
Remove-ADGroupMember -Identity "Sales Team" -Members "jdoe" -Confirm:$false

# Remove all members
Get-ADGroupMember -Identity "Sales Team" | Remove-ADGroupMember -Identity "Sales Team" -Confirm:$false
```

Managing Groups

```powershell
# Change group scope
Set-ADGroup -Identity "Sales Team" -GroupScope Universal

# Change group type
Set-ADGroup -Identity "Sales Team" -GroupCategory Distribution

# Delete group
Remove-ADGroup -Identity "Sales Team" -Confirm:$false

# Find empty groups
Get-ADGroup -Filter * -Properties Members | Where-Object {$_.Members.Count -eq 0}

# Find groups with no members (alternative)
Get-ADGroup -Filter * | ForEach-Object {
    $members = Get-ADGroupMember -Identity $_ -ErrorAction SilentlyContinue
    if ($members.Count -eq 0) {
        $_.Name
    }
}

# Export group members to CSV
Get-ADGroupMember -Identity "Sales Team" |
    Select-Object Name, SamAccountName, ObjectClass |
    Export-Csv "C:\SalesTeam.csv" -NoTypeInformation
```

Bulk Group Membership

- CSV Format
  - SamAccountName,Group
  - jdoe,Sales Team
  - jsmith,Sales Team
  - bwilliams,Marketing Team

```
Import-Csv "C:\GroupMembership.csv" | ForEach-Object {
    Add-ADGroupMember -Identity $_.Group -Members $_.SamAccountName
    Write-Host "Added $($_.SamAccountName) to $($_.Group)"
}
```

Group Nesting

- **What is Nesting?**
  - Adding a group as a member of another group
  - Allows hierarchical permission assignment
- **Rules**
  - Global groups can be members of: Domain Local, Universal, Global (same domain)
  - Domain Local groups can be members of: Domain Local (same domain)
  - Universal groups can be members of: Domain Local, Universal
- **Best Practices**
  - **Avoid deep nesting:** Can impact performance and troubleshooting
  - **Document nesting:** Keep a map of group relationships
  - **Use AGDLP:** Standard nesting pattern
  - **Avoid circular nesting:** Can cause issues

User Group Management

- Add user to a group

```
Add-ADGroupMember -Identity "SalesTeam" -Members "jdoe"
```

- Add multiple users at once

```
Add-ADGroupMember -Identity "SalesTeam" -Members "jdoe", "asmith", "bwayne"
```

- Add a user to multiple groups at once

```
Add-ADGroupMember -Identity "SalesTeam", "VPN-Users", "M365-E3" -Members "jdoe"
```

- Remove a user from a group

```
Remove-ADGroupMember -Identity "SalesTeam" -Members "jdoe" -Confirm:$false
```

- Remove multiple users at once

```
Remove-ADGroupMember -Identity "SalesTeam" -Members "jdoe", "asmith" -Confirm:$false
```

Computers

Computer Object

- A computer object represents a **computer joined to the domain**
- It Authenticates at startup (machine account)
- It has a password (managed automatically)
- It can be a member of groups
- It can have GPOs applied
- It has a trust relationship with the domain

Creating Computer Objects

- **Via Domain Join**
  - Computer joins domain → computer object created automatically
  - Default location: CN=Computers container
  - Can redirect to specific OU via redircmp command
- **Via ADUC (Pre-staging)**
  - Right-click OU → New → Computer
  - Enter computer name
  - Select "Assign this computer account as a pre-Windows 2000 computer" (optional)
  - Click OK

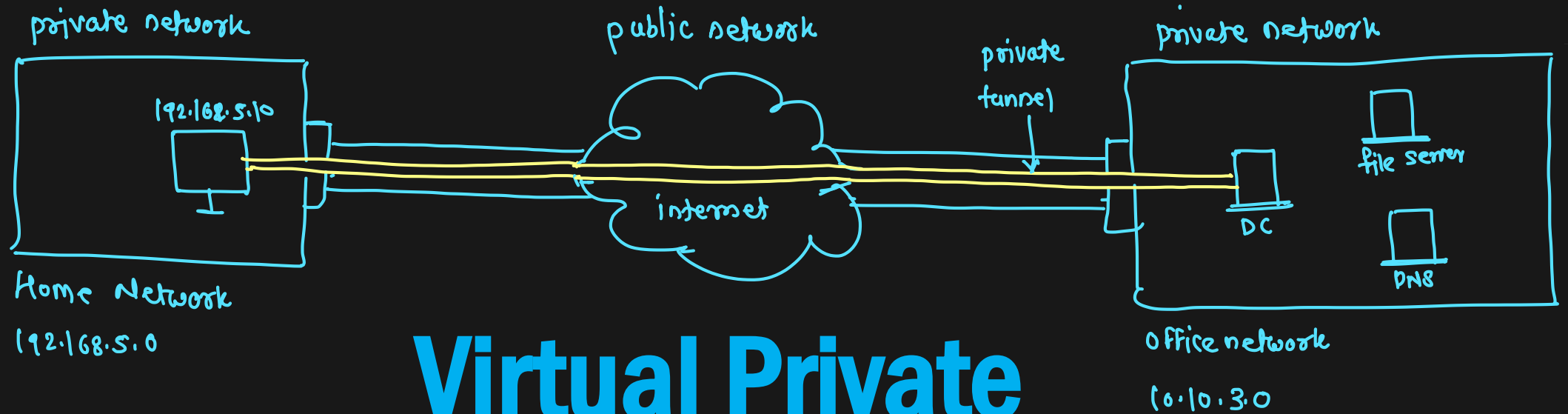
```powershell
# Create computer object
New-ADComputer -Name "WS-001" `
    -SamAccountName "WS-001$" `
    -Path "OU=Workstations,DC=contoso,DC=com" `
    -Description "Sales workstation"

# Pre-stage computer with specific GUID
New-ADComputer -Name "WS-002" `
    -SamAccountName "WS-002$" `
    -Path "OU=Workstations,DC=contoso,DC=com" `
    -Description "Marketing workstation"
```

Managing Computer Objects

```powershell
# Get computer info
Get-ADComputer -Identity "WS-001" -Properties *

# Find computers by OS
Get-ADComputer -Filter {OperatingSystem -like "*Windows 11*"} -Properties OperatingSystem

# Find stale computers (not logged in for 90 days)
$90DaysAgo = (Get-Date).AddDays(-90)
Get-ADComputer -Filter {LastLogonTimeStamp -lt $90DaysAgo} -Properties LastLogonTimeStamp

# Disable computer
Disable-ADAccount -Identity "WS-001$"

# Move computer
Move-ADObject -Identity "CN=WS-001,OU=Workstations,DC=contoso,DC=com" `
    -TargetPath "OU=Stale Computers,DC=contoso,DC=com"

# Reset computer account
Reset-ADAccountPassword -Identity "WS-001$" -Reset

# Delete computer
Remove-ADComputer -Identity "WS-001" -Confirm:$false
```

Delegation of Administration

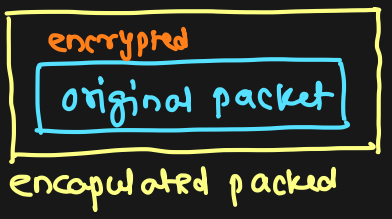
- It allows you to grant **specific administrative permissions** to users or groups without making them Domain Admins
- This follows the **principle of least privilege**
- **Delegation Options**
  - **Built-in Groups**
    - **Account Operators:** Manage user/group accounts
    - **Server Operators:** Manage servers
    - **Backup Operators:** Backup/restore
    - **Print Operators:** Manage printers
  - **Delegation of Control Wizard**
    - Right-click OU → Delegate Control
    - Add users/groups to delegate to
    - Select tasks to delegate
      - Create, delete, and manage user accounts
      - Reset user passwords and force password change
    - Or create custom task to delegate
    - Finish



Virtual Private Network

Introduction

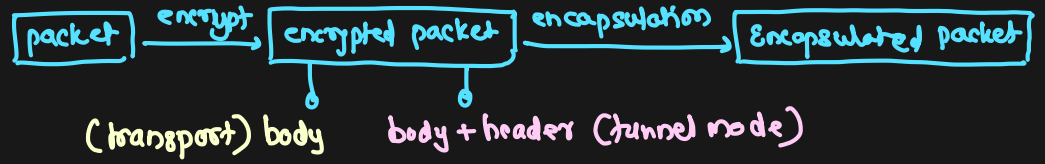
- A Virtual Private Network (VPN) is a technology that creates an **encrypted tunnel** over a public network (such as the internet), allowing remote users or branch offices to access resources as if they were directly connected to the internal network
- The core mechanism is **tunneling**: original data packets are encrypted and encapsulated inside a new packet, which travels across the public network
- Only the devices at both ends of the tunnel can decrypt and read the contents
- How doe it work?
  - VPN achieves secure communication through **encapsulation**
  - The original IP packet (including its header) is encrypted and wrapped inside a new packet
  - The outer packet uses normal routing to travel across the internet
  - The tunneling protocol determines how encapsulation and de-encapsulation are performed
  - Data confidentiality relies on symmetric encryption (such as AES-256-GCM), while key exchange typically uses Diffie-Hellman or its elliptic-curve variants



Types of VPN

- Gateway-to-Gateway
  - Purpose: Connect two networks
  - Scenario: Branch office interconnection
- Host-to-Gateway (Remote Access)
  - Purpose: Remote user accesses corporate network
  - Scenario: Remote work
- Host-to-Host
  - Purpose: Connect specific host to specific resource
  - Scenario: Point-to-point secure communication
- Third-Party Privacy VPN
  - Purpose: Protect connection on public hotspots
  - Scenario: Airport/Hotel Wi-Fi

VPN Protocol Comparison



- **IPsec (Internet Protocol Security)**
  - Recommended as the preferred protocol for enterprise VPNs; it is an open standard, so different implementations can interoperate
  - Two modes: **Transport mode** (encrypts only the payload, keeps the original IP header) and **Tunnel mode** (encrypts the entire original packet and adds a new IP header)
  - Tunnel mode is the common mode for business VPNs
- **SSTP (Secure Socket Tunneling Protocol)**
  - Based on HTTPS, **firewall-friendly**, uses TCP port 443
  - **Still fully supported** in Windows Server 2025

private key → used for decrypted
public key → used for encryption

- **IKEv2 (Internet Key Exchange version 2)**
  - Modern and stable, **mobile-device friendly**, supports keeping the connection alive when switching networks
  - Uses UDP ports 500 and 4500
- **L2TP/IPsec** ✗
  - Older protocol; **no longer accepts connections by default in new Windows Server 2025 installations**
  - Can still be enabled manually, but its security profile is insufficient — **not recommended**
- **PPTP** ✗
  - **Obsolete and insecure**; disabled by default in new Windows Server 2025 installations
  - Strongly discouraged

Network Policy Server

- **Network Policy Server (NPS)** is Microsoft's implementation of a **RADIUS server** and **RADIUS proxy**
- It is installed as part of the **Network Policy and Access Services (NPAS)** role on Windows Server
- **RADIUS** = Remote Authentication Dial-In User Service
- NPS centralizes **authentication, authorization, and accounting (AAA)** for network access
- It decides **who can connect**, **what they can access**, and **logs what they did**
- **Core Functions**
  - Authentication: Verifies the user's identity (username/password, certificate, etc.)
  - Authorization: Decides if the user is allowed to connect and under what conditions
  - Accounting: Logs connection events (who connected, when, how long, data used)

NPS Components

- **RADIUS Clients**
  - Devices that send authentication requests to NPS (e.g., RRAS, Wi-Fi APs, VPN gateways)
  - Each client must be registered in NPS with a **shared secret** or use certificate
    - → private key
    - → public key
  - In your lab: RRAS itself is the RADIUS client (localhost)
- **Network Policies**
  - Rules that determine whether a connection is allowed
  - Each policy has:
    - Conditions: Who/what must match (user group, time, IP, NAS type)
    - Constraints: Limits (idle timeout, session timeout, encryption strength)
    - Settings: What happens if matched (Grant/Deny, auth method, VLAN)
- **Connection Request Policies**
  - Decide **which NPS server** processes the request
  - Used mainly in **RADIUS proxy** scenarios (forwarding to another NPS)
  - In a single-server lab, these are rarely modified
- **Remote RADIUS Server Groups**
  - Used when NPS forwards requests to another RADIUS server
  - Not needed in your single-server lab

Authentication Methods Supported by NPS

- **PAP:** Basic, unencrypted, Avoid — sends password in clear text
- **CHAP:** Legacy Rarely used
- **MS-CHAP v2:** VPN with username/password
- **EAP-MSCHAPv2:** Needs certificate on server
- **EAP-TLS:** Certificate-based .Needs PKI (CA + client certs)
- **PEAP-TLS:** Certificate + tunnel. Enterprise Wi-Fi/VPN

NPS Policy Processing Order

- **Connection Request Policy:** picks the target NPS server
- **Network Policy:** first matching policy wins
- **If no policy matches:** connection is **rejected**
- **If Grant Access:** RRAS completes the tunnel
- **If Deny Access:** error returned to client (e.g., Error 812)

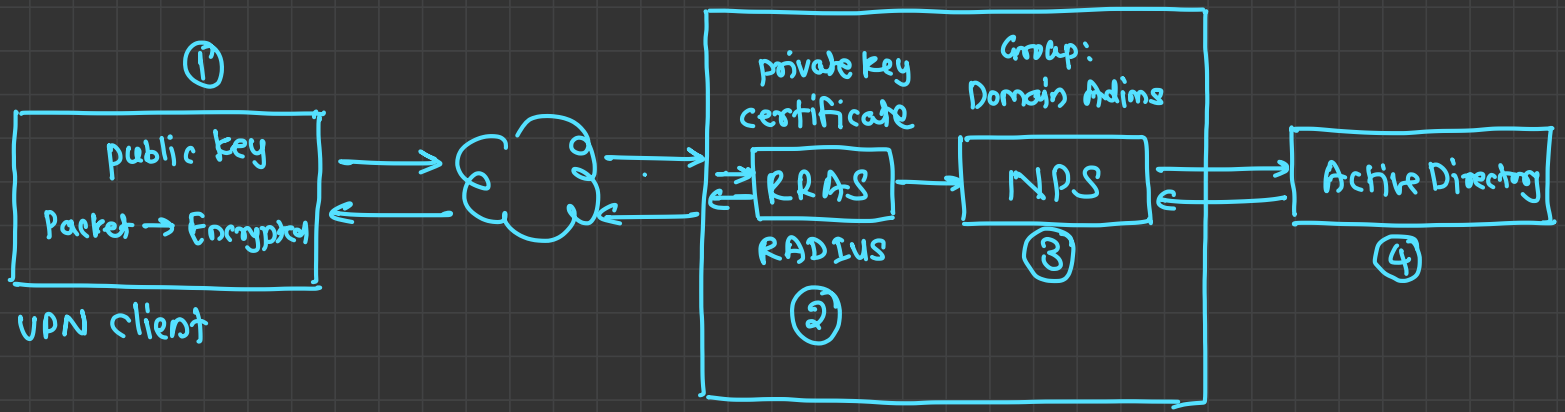
Certificate Creation

- Create Certificate

```
New-SelfSignedCertificate `
-Type Custom `
-Subject "CN=VPNLabRoot" `
-KeySpec Signature `
-KeyExportPolicy Exportable `
-KeyUsage CertSign `
-KeyUsageProperty Sign `
-KeyLength 2048 `
-HashAlgorithm sha256 `
-NotAfter (Get-Date).AddYears(5) `
-CertStoreLocation "Cert:\LocalMachine\My"
```

- Export Certificate

```
Export-Certificate `
-Cert (Get-ChildItem Cert:\LocalMachine\My\<THUMBPRINT>) `
-FilePath C:\VPNLabRoot.cer
```


①
public key
Packet → Encrypted
VPN client
private key
certificate
RRAS
RADIUS
②
Group:
Domain Admins
NPS
③
Active Directory
④

Configuring VPN

- **Prerequisites**
  - **Windows Server 2025** installed with Desktop Experience
  - Membership in the **Administrators group** or equivalent; domain-joined RAS servers require domain admin rights
  - A planned **static IP address pool** for VPN clients
- **Step 1: Install the Remote Access Role**
  - Open **Server Manager** → **Manage** → **Add Roles and Features**
  - Select **Role-based or feature-based installation**
  - Check **Remote Access** in the roles list
  - On the role services page, check **DirectAccess and VPN (RAS)**
  - Complete the installation and reboot if prompted
- **Step 2: Configure RRAS as a VPN Server**
  - Open the **Routing and Remote Access MMC** (run rrasmgmt.msc)
  - Right-click the server name and choose **Configure and Enable Routing and Remote Access**
  - Select **Custom Configuration** in the wizard
  - Check **VPN access** and finish the wizard
  - Click **OK** to close the dialog after the wizard completes

Configuring VPN

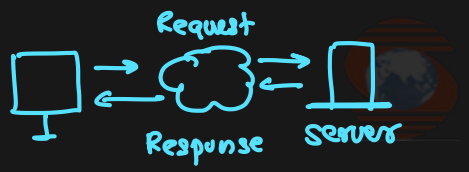
- **Step 3: Configure the IP Address Pool**
  - VPN clients need IP addresses from the server. A **static address pool** is recommended:
  - In the RRAS console, right-click the server → **Properties**
  - Select the **IPv4** tab
  - Choose **Static address pool** → **Add**
  - Enter the start and end IP addresses (ensure these are **not already assigned by DHCP**)
  - Click **OK** to save
- **Step 4: Configure VPN Protocols**
  - New RRAS installations on Windows Server 2025 **no longer accept PPTP and L2TP connections by default**
  - It is recommended to **keep only SSTP and IKEv2** and disable insecure protocols
  - In the RRAS console, right-click **Ports** → **Properties**
  - Locate **WAN Miniport (SSTP)** and click **Configure**
  - Ensure **Remote access connections (inbound only)** is checked and **Demand-dial routing connections** is cleared
  - Do the same for **WAN Miniport (IKEv2)**
  - If not needed, uncheck inbound connections for **WAN Miniport (PPTP)** and **WAN Miniport (L2TP)**
  - Restart the Routing and Remote Access service for changes to take effect

Configuring VPN

- **Step 5: Configure the Firewall**
  - Open the appropriate ports based on your chosen protocols
  - **IKEv2**: UDP **500** and **4500** inbound
  - **SSTP**: TCP **443** inbound
- **Step 6: Grant Users VPN Access**
  - Open **Active Directory Users and Computers** (domain) or **Local Users and Groups** (non-domain)
  - Edit the target user's properties
  - Switch to the **Dial-in** tab
  - Under **Network Access Permission**, select **Allow access**
  - Click **OK** to save
- **Step 7: Test the Client Connection**
  - On the client device:
    - Go to **Settings** → **Network & Internet** → **VPN** → **Add a VPN connection**
    - **VPN provider: Windows (built-in)**
    - **Connection name**: choose any name
    - **Server name or address**: enter the server's **public IP or domain name**
    - **VPN type**: select **IKEv2** or **SSTP**
    - **Type of sign-in info**: select **User name and password** and enter credentials
    - Save and click **Connect**
    - After a successful connection, verify access to internal network resources.

Apache

Web Server



- It is software that receives requests from clients, such as web browsers, and returns responses over HTTP or HTTPS
- For example, when a user enters https://www.example.com into a browser, the browser contacts the server hosting that website. The server processes the request and returns the required web page
- A web server can provide
  - Static content: HTML pages, CSS stylesheets, JavaScript files, images, and downloadable documents. → client side language
  - Dynamic content: Content generated by applications, such as login pages, shopping carts, and dashboards. server side languages - Java, JS
  - Application integration: Requests can be forwarded to backend applications or processed using supported server-side technologies.
  - Secure communication: HTTPS encrypts traffic between the browser and the server using TLS.

    ↳ REST | GraphQL
- Apache HTTP Server, commonly called Apache, is open-source web server software maintained by the Apache Software Foundation
- Apache HTTP Server is one of the most widely used web server software package
- Linux administrators use Apache to host websites, serve web pages, configure multiple websites on one server, manage HTTP and HTTPS traffic, and troubleshoot web services

  ↳ virtual hosting

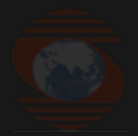
How Apache Processes a Web Request

URL → Uniform Resource Locator
↳ scheme | domain name or ip address | server port | path | query string | hash component

- **URL entry:** The user enters a website address in a browser
- **DNS resolution:** The hostname is translated into an IP address
- **Network connection:** The browser establishes a TCP connection to the server, typically using port 80 for HTTP or port 443 for HTTPS
- **Request transmission:** The browser sends an HTTP request containing information such as the requested path and the Host header
- **Request processing:** Apache accepts the request and selects the appropriate virtual host and configuration rules.
- **Resource lookup:** Apache locates the requested file or forwards the request to a backend application → DocumentRoot
- **Response generation:** Apache returns an HTTP status code, response headers, and, when appropriate, the requested content
- **Logging:** Apache records request information in its access log and records relevant problems in its error log

- **The overall request flow is**
  - Browser → DNS → Server IP address → TCP port → Apache → Website file or backend application → HTTP response

Apache Architecture



- HTTP daemon → httpd
  - The daemon is the running server process that accepts and processes web requests
  - Its service is commonly named apache2 on Ubuntu/Debian and httpd on RHEL-family systems
- Configuration files
  - Configuration files define listening ports, virtual hosts, access permissions, modules, logging, and other server behavior
  - Apache reads these settings when starting and when the service successfully reloads
- Modules SSL module
  - Modules extend Apache functionality. Examples include TLS support, URL rewriting, request headers, authentication, and reverse proxying
- Document root
  - The document root is the filesystem directory from which a virtual host serves files by default
  - A common default is /var/www/html
- Virtual host
  - A virtual host allows one Apache instance to serve multiple websites, each with its own hostname, document root, and potentially its own logs and security settings
- Logs
  - Access logs record HTTP requests
  - Error logs help administrators investigate configuration errors, permission problems, missing files, and service failures

- These components are managed through configuration files, Linux commands, filesystem permissions, and systemd

Apache on Different Linux Distributions

- Apache is available on many Linux distributions, but package names, service names, and configuration layouts differ
- Ubuntu/Debian — sudo apt install apache2
  - Package name: apache2
  - Service name: apache2
  - Main configuration: /etc/apache2/apache2.conf
  - Additional site configurations: sites-available/ and sites-enabled/
  - Common default web root: /var/www/html
  - Common log directory: /var/log/apache2/
- RHEL-family → sudo dnf install httpd
  - Package name: httpd
  - Service name: httpd
  - Main configuration: /etc/httpd/conf/httpd.conf
  - Additional site configurations: Commonly conf.d/
  - Common default web root: /var/www/html
  - Common log directory: /var/log/httpd/

Installing Apache HTTP Server

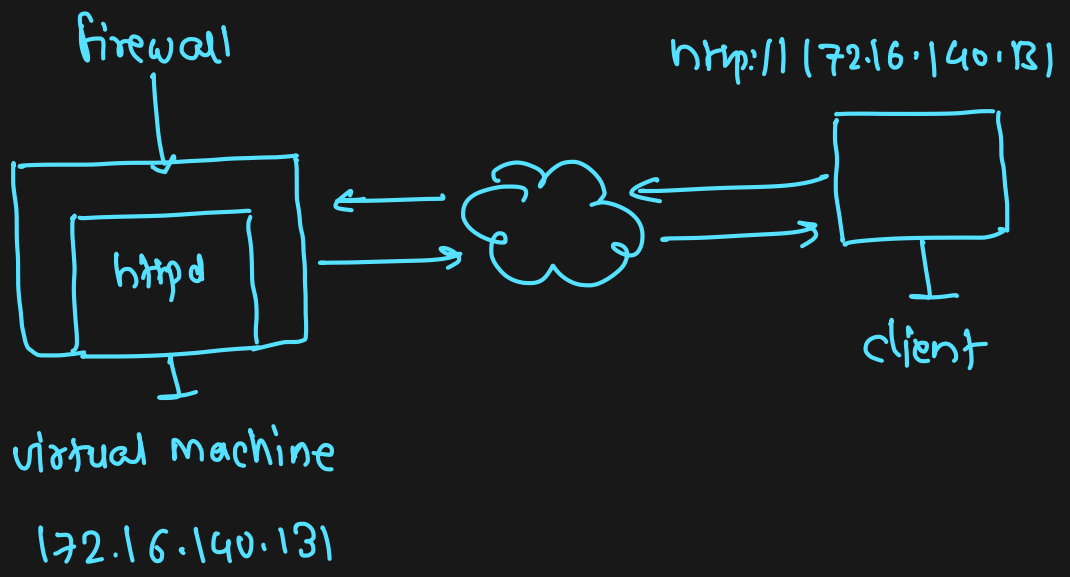
- Apache should generally be installed using the operating system's package manager
- This ensures that dependencies, service integration, and security updates are handled through the distribution's package-management system

- **Installation on Ubuntu/Debian**
  - sudo apt install apache2

- **Installation on RHEL-family distributions**
  - sudo dnf install httpd

Starting, Stopping, and Managing Apache

- Ubuntu/Debian
  - sudo systemctl enable --now apache2
  - sudo systemctl start apache2
  - sudo systemctl stop apache2
  - sudo systemctl restart apache2
  - sudo systemctl statusapache2
- RHEL-family
  - sudo systemctl enable --now httpd
  - sudo systemctl start httpd
  - sudo systemctl stop httpd
  - sudo systemctl restart httpd
  - sudo systemctl status httpd



- Check listening ports
  - sudo ss -ltnp | grep -E ':80|:443'
- Checking connectivity
  - curl -I http://127.0.0.1/

Deploying Default Site

Document Root

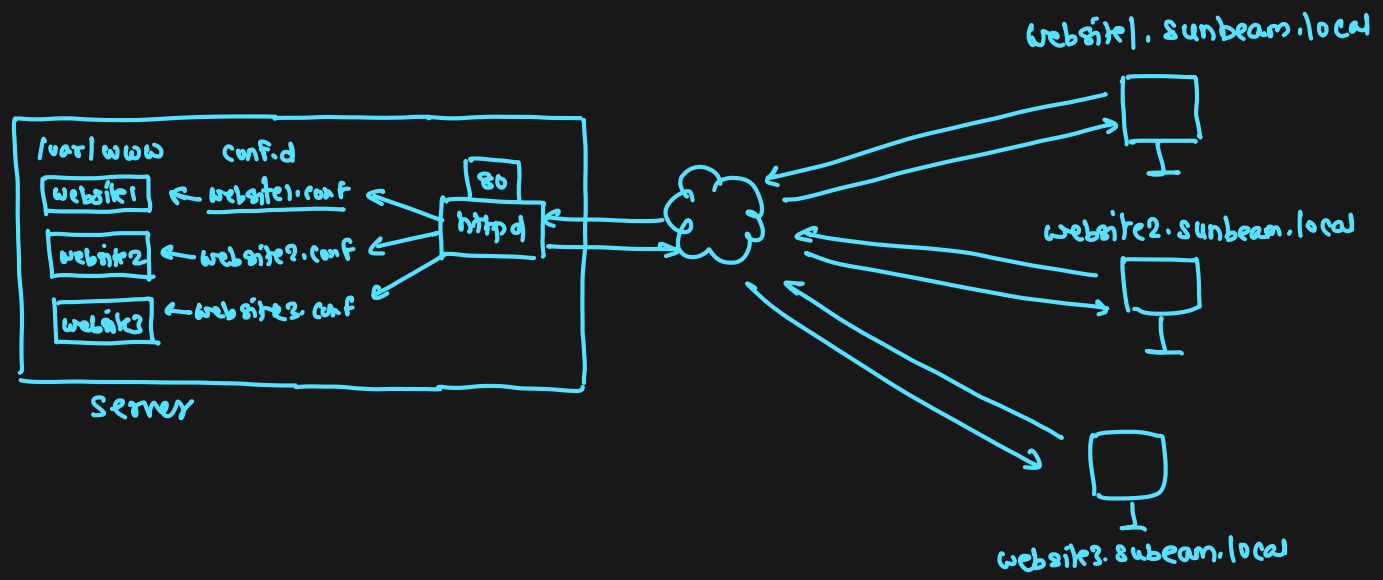
- Place website files into `/var/www/html/`
- Default index filenames are index.html, then index.htm
- Ensure correct file permissions: directories 755, files 644, owner set to apache:apache
- Test configuration syntax:
  - sudo httpd –t
- Restart the service
  - sudo systemctl restart httpd

Directory Structure

- Main configuration file: /etc/httpd/conf/httpd.conf
- Additional configuration directory: /etc/httpd/conf.d/ (files ending in .conf are loaded automatically)
- Default website root directory: /var/www/html/
- Log directory: /var/log/httpd/
- Module configuration directory: /etc/httpd/conf.modules.d/

Virtual Hosting

- **Virtual hosts** allow hosting multiple websites on a single server, routing requests to different sites based on the requested domain (the Host header) → ServerName
- **Name-based virtual hosting**
  - It is the most common method
  - Multiple domains resolve to the same IP, and Apache distinguishes them by ServerName
- **Configuration file location**
  - on Rocky Linux, recommended to place them in /etc/httpd/conf.d/, one .conf file per site



Virtual Hosting

- Create a configuration file
  - sudo vi /etc/httpd/conf.d/site1.conf
- Core directive content:

```
<VirtualHost *:80>
    ServerName site1.example.com
    DocumentRoot /var/www/site1
    ErrorLog /var/log/httpd/site1_error.log
    CustomLog /var/log/httpd/site1_access.log combined
    <Directory /var/www/site1>
        Require all granted
        AllowOverride All
    </Directory>
</VirtualHost>
```

- ServerName specifies the domain, DocumentRoot specifies the website file directory — both must match
- Create the directory and set ownership:
  - sudo mkdir -p /var/www/site1 && sudo chown -R apache:apache /var/www/site1

Changing the Listening Port

- Edit /etc/httpd/conf/httpd.conf
  - Find Listen 80, change it to Listen 8080
- Install SELinux utilities
  - sudo dnf install policycoreutils-python-utils -y
- Check SELinux-allowed HTTP ports
  - sudo semanage port -l | grep http_port_t
- **SELinux must be configured accordingly**
  - sudo semanage port -a -t http_port_t -p tcp 8080
- Open the new port in the firewall
  - sudo firewall-cmd --permanent --add-port=8080/tcp && sudo firewall-cmd --reload
- Test configuration and restart
  - sudo httpd -t && sudo systemctl restart httpd

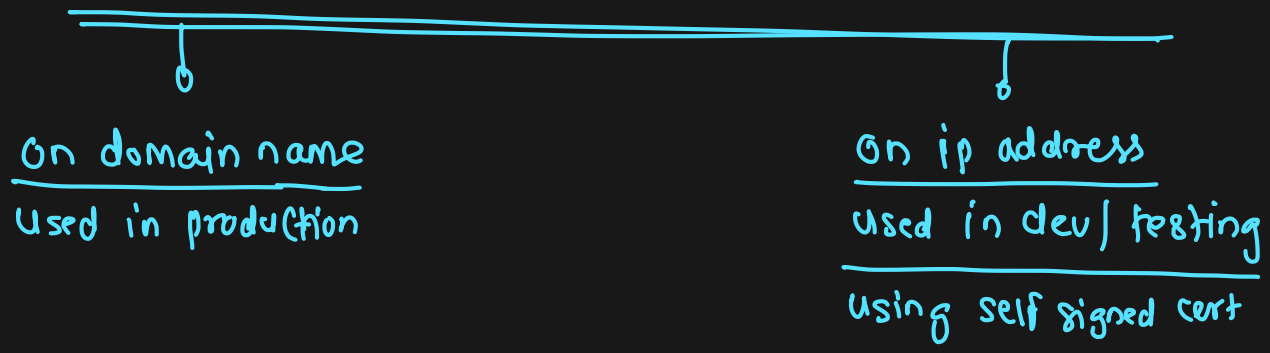
Hardening Points

- **Running user:** Apache runs as the apache user on Rocky Linux. Ensure website directory ownership is correct
- **Hide version information:** add ServerTokens Prod and ServerSignature Off to the configuration
- **Disable directory listing:** set Options -Indexes inside the <Directory> block
- **Remove sample scripts:** delete sample CGI scripts shipped with the distribution (such as PHF), which have known vulnerabilities
- **Enable SSL/TLS:** install mod_ssl, configure a VirtualHost on port 443, and disable TLSv1.0/1.1

Common Management Commands

- Start/stop/restart: sudo systemctl start|stop|restart httpd
- Reload configuration (without interrupting service): sudo systemctl reload httpd
- Syntax check: sudo httpd -t or sudo apachectl configtest
- View loaded modules: sudo httpd -M
- View virtual host mapping: sudo apachectl -S
- Test local access: curl -s http://localhost/ | head -1

Configuring HTTPS



Prerequisites

- Apache installed and running:
  - sudo dnf install httpd –y
  - sudo systemctl enable --now httpd
- Root or sudo access to the server
- For Let's Encrypt only ↪
  - A registered domain name with an A record pointing to your server's public IP
- Firewall (firewalld) installed and active

Install the SSL Module

- Apache requires the mod_ssl package for HTTPS support
- **Install command**
  - sudo dnf install mod_ssl -y
  - This installs the module and creates the default SSL configuration file at /etc/httpd/conf.d/ssl.conf
- **Verify installation**
  - rpm -qi mod_ssl or httpd -M | grep ssl_module
- Without this package, Apache cannot listen on port 443
- **Open Firewall Ports**

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
```

Self-Signed Certificate

- **When to use:** Internal testing, labs, private services. Browsers will show a trust warning
- **Create a directory** for certificates
  - sudo mkdir -p /etc/ssl/mysite
- **Generate certificate**

```
sudo openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/ssl/mysite/apache.key \
  -out /etc/ssl/mysite/apache.crt
```

- **Set permissions**

```
sudo chmod 600 /etc/ssl/mysite/apache.key
sudo chmod 644 /etc/ssl/mysite/apache.crt
```

- **SELinux note**
  - Files outside /etc/pki/ may need context fix
  - sudo restorecon -RvF /etc/ssl/mysite

Configure Apache for HTTPS

- **Edit the SSL configuration file**
  - sudo vi /etc/httpd/conf.d/ssl.conf

- **Key directives to set**

```
<VirtualHost *:443>
    ServerName example.com
    DocumentRoot /var/www/html
    SSLEngine on
    SSLCertificateFile /etc/ssl/mysite/apache.crt
    SSLCertificateKeyFile /etc/ssl/mysite/apache.key
    SSLProtocol -all +TLSv1.2 +TLSv1.3
</VirtualHost>
```

- **Ensure** Listen 443 is present (usually already in ssl.conf)
- **Test syntax:** sudo httpd -t or sudo apachectl configtest .
- **Restart Apache:** sudo systemctl restart httpd

Nginx

What is Nginx?

- **Nginx** is a high-performance open-source web server, reverse proxy, and load balancer
- Created by Igor Sysoev, first released in 2004. Now maintained by F5 Networks
- Core design philosophy: **event-driven, asynchronous, non-blocking architecture** — handles thousands of simultaneous connections with minimal memory overhead
- **Primary use cases**: serving static content, reverse proxying to application servers, load balancing, SSL/TLS termination, and HTTP caching
- **LEMP stack**: Linux + Nginx + MySQL/MariaDB + PHP — a modern alternative to LAMP

Nginx Architecture

- Nginx uses a **master process** and one or more **worker processes**
- **Master process**: reads and validates configuration, manages worker processes, handles signals
- **Worker processes**: do the actual request processing. Nginx uses OS-dependent mechanisms (epoll on Linux, kqueue on BSD) to efficiently distribute requests among workers
- The number of worker processes is defined by the worker_processes directive — typically set to the number of CPU cores
- **Cache loader and cache manager** processes also run if caching is enabled
- This architecture enables Nginx to handle high concurrency with low resource usage

Configuration File Structure

- Main configuration file: /etc/nginx/nginx.conf (verify exact path with nginx -V)
- The configuration consists of **directives** and their parameters. Simple directives end with a semicolon (;)
- Container directives (blocks) are enclosed in curly braces ({})
- **Top-level contexts**
  - events — connection processing settings
  - http — HTTP traffic configuration
  - mail — mail proxy traffic
  - stream — TCP/UDP traffic
- Directives outside these contexts are in the **main context**

Contexts and Virtual Servers

- Within the http context, you define one or more server **blocks** (virtual servers)
- Each server block controls request processing for specific domains, IPs, or ports
- Inside a server block, one or more location **contexts** define how to process specific URI patterns
- **Inheritance**: Child contexts inherit directives from parent contexts. Some directives can be overridden at deeper levels

Server Blocks (Virtual Hosts)

- **Server blocks** allow hosting multiple websites on a single Nginx instance
- Nginx selects the matching server block using listen (port/IP) and server_name (domain/hostname)
- **Matching order**: exact name → wildcard starting with * → wildcard ending with * → regular expression
- The **default server** is the first server block for a given listen port, or explicitly marked with default_server
- Best practice: define a specific default server that returns status 444 (connection closed) to reject unmatched requests

Server Block Configuration

- **listen:** port and address to monitor
- **server_name:** domain names this block responds to
- **root :** filesystem path serving site content
- **index:** default files for directory requests
- **location /:** root path matching block
- **try_files:** try files/directories in order, fallback on miss

```
server {
    listen 80;
    server_name example.com www.example.com;

    root /var/www/example.com/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/example.access.log;
    error_log /var/log/nginx/example.error.log;
}
```

Controlling Nginx Processes

- Nginx can be controlled via systemctl or by sending signals to the master process
- **Signal-based control**
  - nginx -s <SIGNAL>
    - quit — graceful shutdown (wait for workers to finish)
    - reload — reload configuration without dropping connections
    - reopen — reopen log files
    - stop — fast shutdown
- **Systemctl commands**
  - Start: sudo systemctl start nginx
  - Stop: sudo systemctl stop nginx
  - Reload: sudo systemctl reload nginx
  - Restart: sudo systemctl restart nginx
- **Always test configuration before reloading**
  - sudo nginx -t

Testing and Verifying Configuration

- **Syntax check**: sudo nginx -t — validates configuration files.
- **Verify exact config path and compile options**: sudo nginx -V
- **Check service status**: sudo systemctl status nginx
- **Test HTTP response locally**: curl http://localhost
- **Check listening ports**: sudo ss -tlnp | grep nginx
- **View logs**: ls -l /var/log/nginx/
- Always run nginx -t before reloading to catch syntax errors

Internet Information Service (IIS)

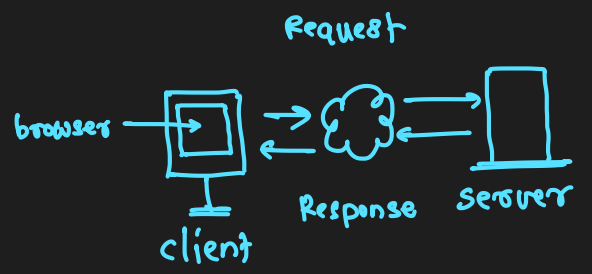
Web Server

Used to host websites

Introduction

- A **web server** is a software application (and the underlying hardware it runs on) that stores, processes, and delivers web content (web pages, files, images, videos, APIs) to clients over the internet or an intranet using the **HTTP/HTTPS** protocol
  ↳ resources
- It operates on a **client-server model**:
  - Client (usually a web browser) sends a request, and the server responds with the requested resource or an error message
- **Popular web servers** Redhat → HTTPD , Debian → apache2
  - Apache HTTP Server: Open-source, modular, .htaccess, widely used
  - Nginx: High performance, event-driven, reverse proxy, load balancing
  - Microsoft IIS: Windows-based, integrates with ASP.NET
  - LiteSpeed: Fast, Apache-compatible, low resource usage
  - Apache Tomcat: Java Servlet/JSP container → application server
  - Caddy: Automatic HTTPS, modern, simple config
  - Node.js (Express): JavaScript runtime used as a lightweight server

Web architecture



Functionalities

- **Serving static content:** Delivers HTML, CSS, JS, images, PDFs directly from disk
- **Serving dynamic content:** Executes scripts (PHP, Python, Node.js) and returns generated output
- **Request handling:** Parses HTTP requests, methods (GET, POST, PUT, DELETE)
- **Response generation:** Sends status codes (200, 404, 500) + headers + body
- **Logging:** Records access logs and error logs
- **Security:** TLS/SSL encryption, authentication, access control
- **Virtual hosting:** Hosts multiple domains on one server
- **Load balancing / proxying:** Distributes traffic across backend servers

How a Web Server Works

- **DNS Resolution** – Browser resolves domain name → IP address.
- **TCP Connection** – Client establishes a connection (usually port 80 for HTTP, 443 for HTTPS).
- **HTTP Request** – Browser sends a request:
  - GET /index.html HTTP/1.1
  - Host: www.example.com
- **Server Processing** – Server locates resource, runs scripts if needed.
- **HTTP Response** – Server returns:
  - HTTP/1.1 200 OK
  - Content-Type: text/html
- **Rendering** – Browser renders the page.
- **Connection Close/Keep-alive** – Depending on headers.

Types of Web Servers

Static website = website developed in html + css + JS

Dynamic website = website developed in server sided programming languages
= eg. Java, python, javascript

- **Static Web Server**
  - Serves pre-built files as-is. Fast and simple. Example: serving a plain HTML site
- **Dynamic Web Server**
  - Combines a web server with an application server to generate content on the fly (e.g., database-driven sites)
- **Proxy / Reverse Proxy Server**
  - **Forward proxy** – Acts on behalf of clients
  - **Reverse proxy** – Acts on behalf of servers (e.g., Nginx, HAProxy) for load balancing, caching, SSL termination

Core Components

- Listener/Socket – Accepts incoming connections
- Request Parser – Interprets HTTP request
- Handler/Module – Processes request (static, CGI, proxy, etc.)
- Thread/Process Pool – Concurrency model (threaded, event-driven, process-based)
- Document Root – Base directory for files
- Configuration Files – httpd.conf, nginx.conf, .htaccess
- MIME Types – Maps file extensions to content types
- Cache – Stores frequent responses for speed

IIS

IIS

- **Internet Information Services (IIS)** is Microsoft's built-in web server role for Windows Server and Windows Client operating systems
- It provides a secure, manageable, and scalable platform for hosting websites, web applications, and services. IIS 10.0 is the current version included with supported Windows Server releases, including Windows Server 2025
- IIS is **not installed by default** on a fresh Windows Server installation., it must be added via Server Manager or PowerShell
- **Installation**
  - IIS is installed as a Web Server (IIS) role via
    - Server Manager → Add Roles and Features → Web Server (IIS)
    - PowerShell
      - Install-WindowsFeature -Name Web-Server
  - Default Installation
    - Creates a "Default Web Site" listening on HTTP port 80, serving content from %SystemDrive%\inetpub\wwwroot

Core Architecture

Application Pool → Website

- **Web Sites**
  - Top-level containers representing individual websites
  - Each site is identified by a unique combination of IP address, port, and host name (binding)
- **Applications**
  - A subset of a site that runs as a distinct unit
  - Applications can have their own application pool, configuration settings, and even a separate .NET runtime
- **Virtual Directories**
  - Logical directory names that map to physical folders on disk (can be outside the site root)
  - They appear as normal subfolders to users
- **Application Pools**
  - Worker process environments where sites/applications run
  - Each pool runs in its own sandboxed process (w3wp.exe), providing isolation

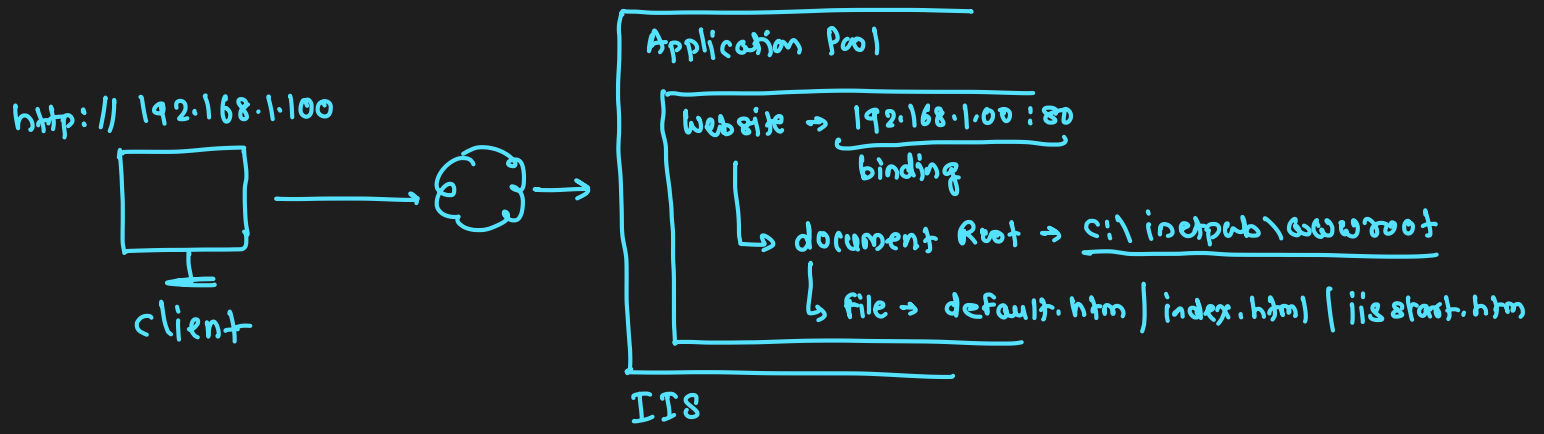
WWW - Worker process

- **Modules**
  - Components that handle specific request-processing tasks (authentication, compression, URL rewriting).
  - Modules can be added or removed as needed

Request Processing Flow

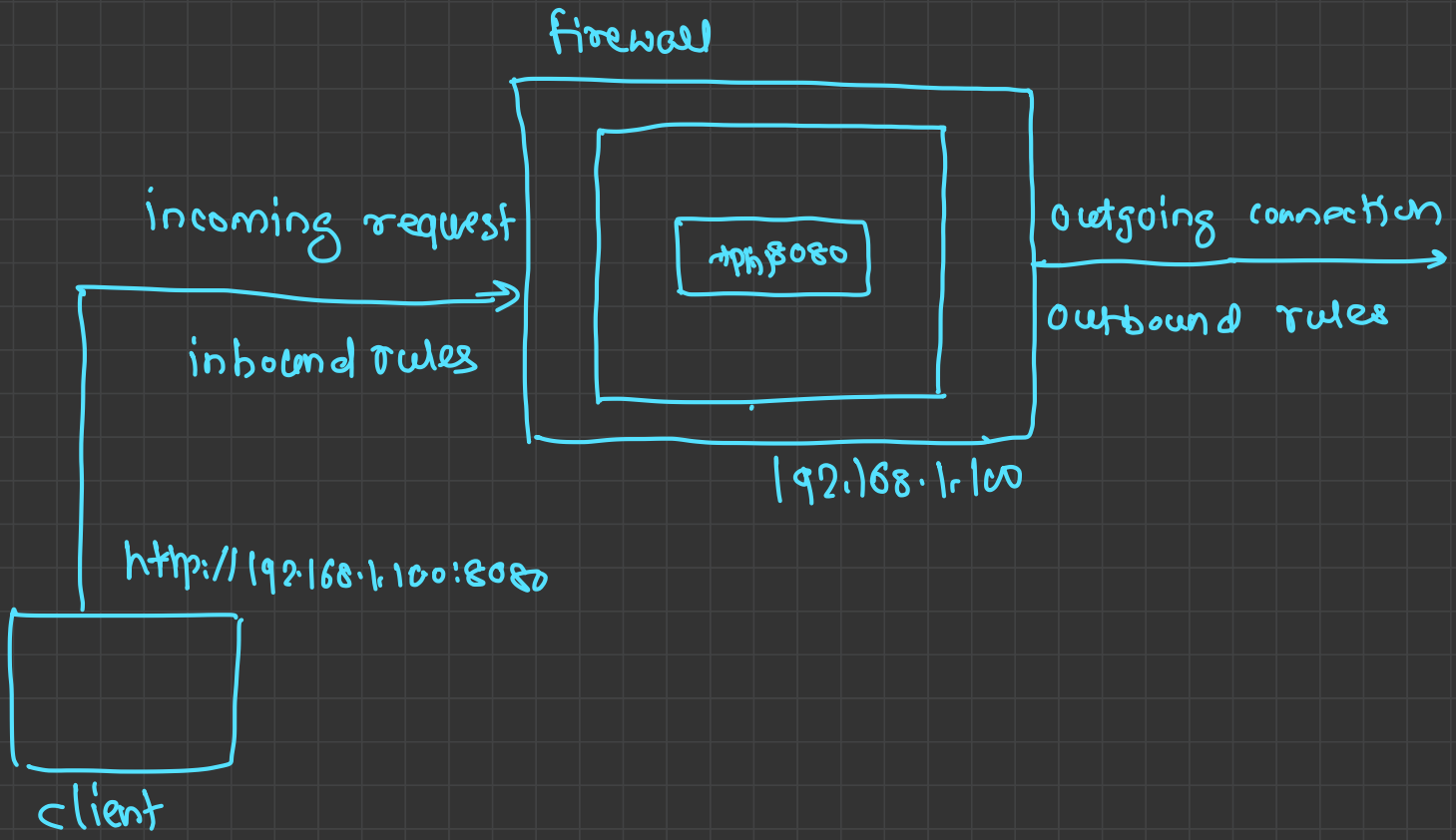
- **Site Identification**
  - Incoming request matched to a site via IP, port, and Host header against site bindings
- **Path Resolution**
  - URL path examined to determine if it maps to an application or virtual directory with distinct settings
- **Execution**
  - Request executed using the assigned application pool, which loads necessary modules for authentication, authorization, content serving, or code execution

Binding ⇒ mapping IP address & port



Creating and Configuring Websites

- Open **IIS Manager** (inetmgr.exe)
- In Connections pane, expand server node → right-click **Sites** → **Add Website**
- Configure
  - **Site name:** Descriptive name
  - **Application pool:** Default creates new pool with same name
  - **Physical path:** Content directory (e.g., C:\inetpub\contoso)
  - **Binding type:** HTTP or HTTPS
  - **IP address:** Usually "All Unassigned"
  - **Port:** 80 (HTTP) or 443 (HTTPS)
  - **Host name:** FQDN (required when multiple sites share the same port/IP)


firewall
incoming request
inbound rules
app:8080
outgoing connection
outbound rules
192.168.1.100
http://192.168.1.100:8080
client

Windows Deployment Service

Windows Configuration Manager

Pxe_Booting eXecution Environment

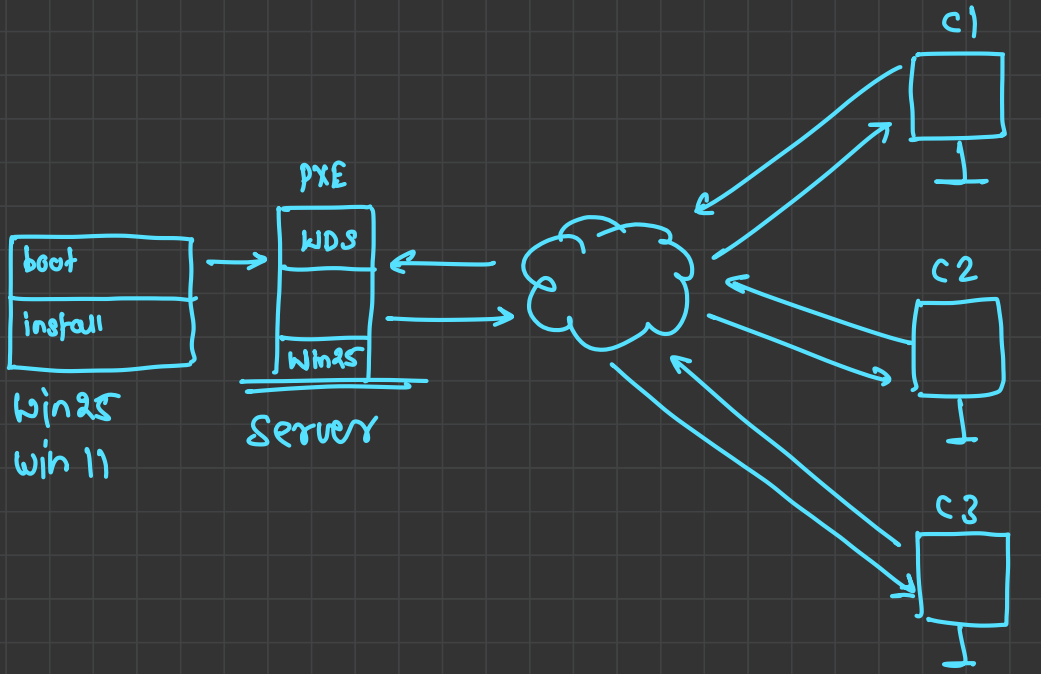
↳ network booting

Bios

↳ Boot order

↳ Network

↳ Disk



Introduction

- Windows Deployment Services (WDS) is a server role included with Windows Server that enables network-based installation of Windows operating systems
- It allows administrators to deploy Windows to computers that have no installed operating system by leveraging Preboot Execution Environment (PXE) boot technology
- Despite the deprecation announcements, the following WDS capabilities remain available on Windows Server 2025:
  - **PXE boot services:** WDS still functions as a PXE server, responding to DHCP requests from network boot clients
  - **Custom boot image deployment:** WDS can boot devices using custom Windows PE images (not the standard boot.wim from installation media)
  - **Image storage and management:** The WDS image store continues to manage boot and install images
  - **Multicast transmission:** The Transport Server component's multicast engine remains functional
  - **Deployment of older operating systems:** Windows 10, Windows Server 2019, and earlier versions can still be deployed using standard workflows

WDS Architecture

- **Transport Server**
  - The Transport Server provides the core networking functionality for WDS, specifically for multicast transmission
  - Its primary purpose is to distribute images efficiently to multiple computers simultaneously, conserving network bandwidth
- **Deployment Server**
  - The Deployment Server requires the Transport Server and provides the actual deployment orchestration
  - It includes the following sub-components :
    - **PXE Server:** Listens for DHCP requests and responds with PXE offers to network boot clients
    - **Image Store:** Manages boot images and install images uploaded to the WDS server
    - **WDS Client:** The component within Windows PE that connects to WDS, joins sessions, and applies install images
    - **TFTP Server:** Functions as a Trivial FTP server for boot file transfer
- **Image Types**
  - **Boot Image:**
    - Provides the Windows PE environment for deployment
    - Scaled-down Windows OS (Windows PE); typically one for 32-bit and one for 64-bit architectures
  - **Install Image:**
    - The actual operating system to be deployed
    - Mirror of a hard drive containing OS, applications, updates, roles, and settings

Installation

- **Prerequisites**
  - **DHCP Server:** Required to assign IP addresses to PXE clients (can be on the same or different server)
  - **Active Directory (for Integrated mode):** WDS can be configured in either AD-integrated or standalone mode
  - **DNS:** Proper name resolution for network services
  - **Dedicated storage:** A separate partition or disk for WDS image storage is strongly recommended
- **Installation Steps**
  - Open **Server Manager** and select **Add Roles and Features**
  - Choose **Role-based or feature-based installation**
  - Select the target server
  - Check **Windows Deployment Services**
  - When prompted, add the required features (management tools)
  - In the role services selection, both **Deployment Server** and **Transport Server** are selected by default—leave both enabled
  - Complete the wizard and install

Windows Server Backup

Introduction

- Windows Server Backup (WSB) is a **built-in, free backup and recovery feature** in Windows Server
- It was introduced in Windows Server 2008 and replaced the older NTBACKUP tool.
- WSB provides **wizards and command-line tools** for performing basic backup and recovery tasks
- **Installation**
  - **GUI Method**
    - Open **Server Manager** → **Add Roles and Features**
    - Select **Features** (not Roles)
    - Check **Windows Server Backup**
    - Click **Install**
  - **PowerShell Method**

```
Install-WindowsFeature -Name Windows-Server-Backup
```

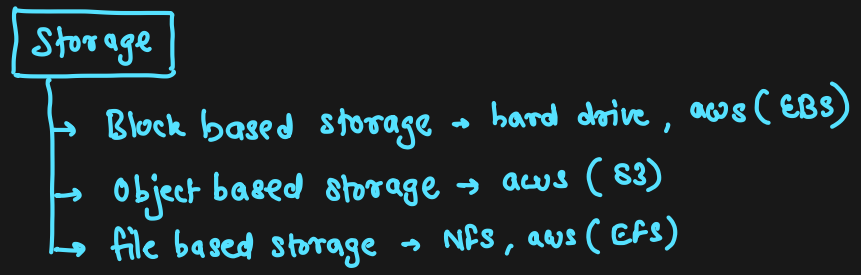
Backup Types

- **Full Server:** All volumes, including OS, system state, and all data
- **Selected Volumes:** Only specified volumes
- **System State:** Critical system components (see Section 4)
- **Specific Files/Folders:** Fine-grained file-level backup
- **Bare Metal Recovery (BMR):** Entire system, for disaster recovery to new hardware

Technical Features

↳ copy the volume block by block

- **Block-based Backup:** WSB integrates with Volume Shadow Copy Service (VSS) and works at the block level, enabling snapshot-style backups. It can back up any application with a VSS Writer
- **Incremental and Full Backups:** Supports incremental backups (prioritizes backup speed) and full backups (prioritizes post-recovery I/O performance, does not retain shadow copies on the volume)
- **Automatic Disk Usage Management:** Configurable deletion policies automatically remove older backups to optimize disk space
- **Large Volume Support:** Supports volumes larger than 2 TB and 4K sector sizes
- **Cluster Shared Volume (CSV) Support:** Limited CSV backup support (VMs cannot be added as part of the backup configuration; volume recovery is not supported)
- **Hyper-V Support:** Full support for backing up and recovering Hyper-V hosts and virtual machines.



System State

- System State is a collection of critical components
- You cannot choose individual components because of dependencies among them
  - Boot files (including system files protected by Windows File Protection)
  - COM+ Class Registration Database
  - Active Directory (on Domain Controllers only)
  - SYSVOL (on Domain Controllers only)
  - Certificate Services (on Certificate Authority only)
  - Cluster database (on cluster nodes only)
  - IIS metabase (on IIS servers only)
  - Performance counter configuration
  - Registry
- **System State Limitations**
  - Cannot back up to a critical volume
  - Cannot be used for BMR alone (requires full server backup)
  - Size varies by server role

Backup Destinations

- Dedicated Local Disk
  - A disk dedicated exclusively to backups
  - Format is handled automatically by WSB
  - Recommended for scheduleackups
- Local Volume (Non-critical)
  - Can be a separate volume on the same disk
  - Must not contain the OS or system state
- Removable Media (USB/eSATA)
  - External drives can be used
  - Must be formatted appropriately
  - Suitable for offsite rotation
- Network Share (Remote SMB Share)
  - Backups can be written to a remote SMB share
  - Requires proper credentials
  - Limitations apply for system state / BMR backups
- DVD/CD (Legacy)
  - Not supported for scheduled backups
  - Only for manual, one-time backups (deprecated in newer versions)

How to Back Up System State

- Open **Windows Server Backup** (via Server Manager → Tools)
- Select **Local Backup** → **Backup Once**
- Choose **Different Options** → **Custom**
- Select **Add Items** → **System State** → OK
- Choose destination (Local drive or Remote shared folder)
- Complete the wizard

Recovery Capabilities

- Volume-level recovery
- File/folder-level recovery
- Application recovery (e.g., Exchange, SQL Server)
- System State recovery
- **Bare Metal Recovery (BMR)**: Restores a complete system to the original or new hardware

Recovery Operations

- **Recovery Types**
  - Files and Folders: Restore individual items
  - Volumes: Restore entire volumes
  - Applications: Restore app data (VSS-aware apps)
  - System State: Restore system components
  - Bare Metal Recovery: Full system to bare metal (machine)
- **Bare Metal Recovery (BMR) Process**
  - Boot from Windows Server installation media
  - Select Repair your computer
  - Choose System Image Recovery
  - Select the backup to restore
  - Start recovery

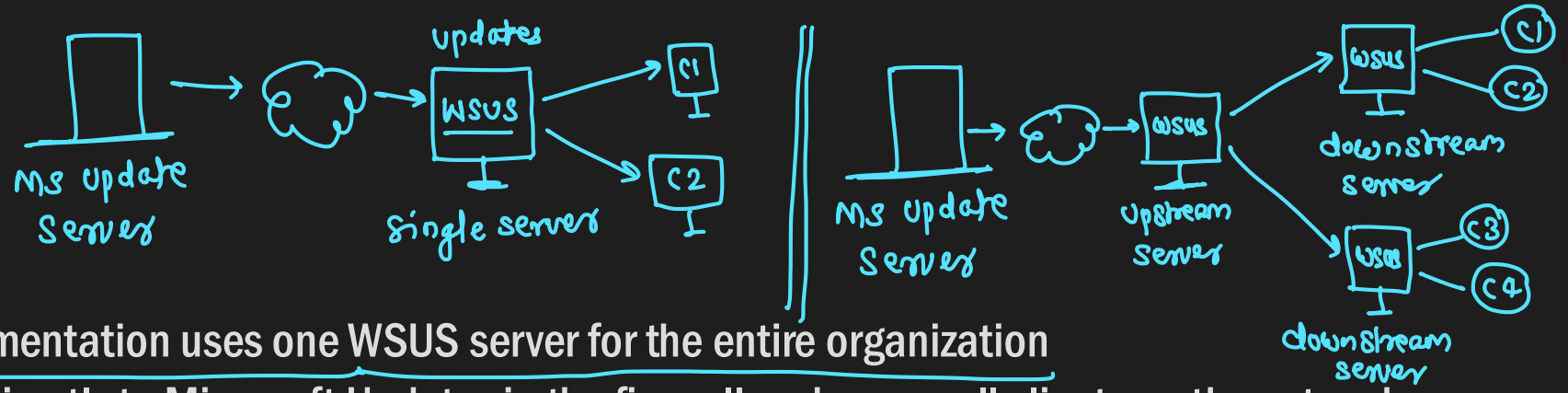
WSUS

Update Services

Introduction

- Windows Server Update Services (WSUS) is a server role that enables administrators to centrally manage and distribute Microsoft updates to computers across their network
- It acts as a middleman between Microsoft Update and your client machines, allowing you to test, approve, and deploy patches on your own schedule
- **Hardware Requirements**
  - **Processor:** 1.4 GHz x64 (2 GHz+ recommended)
  - **Memory:** Additional 2 GB RAM for WSUS (beyond the OS needs)
  - **Disk Space:** 40 GB+ recommended
- **Software Prerequisites**
  - **Updates:**
    - Install the February 2023 cumulative update (or newer) on WSUS servers running Server 2016/2019/2022
  - **Database:**
    - WSUS requires a database
    - It installs **Windows Internal Database (WID)** by default, or you can use SQL Server (Standard, Enterprise, or Express)

Deployment Models



- **Core Deployment Models**
  - **Single Server**
    - The most basic implementation uses one WSUS server for the entire organization
    - This server connects directly to Microsoft Update via the firewall and serves all clients on the network
    - This model is typically used in small networks with a single physical location and is the simplest to manage
  - **Multiple Independent Servers**
    - If your organization has multiple isolated physical locations, you might deploy a WSUS server at each site.
    - Each server maintains its own Internet connection to download updates directly from Microsoft Update
    - This model requires more administrative effort because each server must be managed independently—downloading, approving, and managing client targeting separately for each location
- **Hierarchical Deployment Models**
  - For larger organizations, WSUS servers can be organized in a hierarchy where downstream servers synchronize from an upstream server rather than directly from Microsoft Update
  - This approach reduces Internet bandwidth consumption by downloading updates once and then distributing them internally
  - **Important constraint**: Microsoft recommends a maximum hierarchy depth of **three levels** for WSUS servers

WID vs SQL Server

Feature	Windows Internal Database (WID)	SQL Server (Full/Express)
License Cost	Included with Windows Server	Requires separate license (Full edition)
Performance (Single Server)	Same as SQL Server	Same as WID
NLB / High Availability	**Not supported**	**Required** for NLB clusters
Database Size Limit	No built-in limit	Express edition: **10 GB** limit
Remote Management	Not possible remotely	Full remote management via SSMS
Authentication	Windows Authentication only	Windows Authentication only (WSUS requirement)

When to Choose Which

- **Choose WID when**
  - You are deploying a **single WSUS server** (or multiple independent servers)
  - You have **no existing SQL Server infrastructure** and want to avoid licensing costs
  - You do **not** need Network Load Balancing (NLB)
  - You are deploying WSUS in **branch offices** with separate databases per server
- **Choose SQL Server when**
  - You need an **NLB WSUS solution** for high availability or large-scale client load
  - You **already have SQL Server installed** and want to consolidate database management
  - You need **remote database management** capabilities

Configuration

- **Step 1: Install the WSUS Role**
  - In **Server Manager**, use "Add Roles and Features" to install the **Windows Server Update Services** role
  - During the post-installation configuration wizard, Choose to synchronize from **Microsoft Update**
  - Select the **Languages**, **Products** (e.g., Windows, Office), and **Classifications** (e.g., Critical, Security) you want to download
- **Step 2: Configure Client Computers via GPO**
  - This is the critical step to point your clients to the WSUS server
  - Create a GPO linked to the OU containing your computers, and configure these settings under
    - Computer Configuration > Policies > Administrative Templates > Windows Components > Windows Update
  - **Specify intranet Microsoft update service location**
    - Set both URLs to your WSUS server (http://wsus-server:8530)
  - **Configure Automatic Updates**: Set to **Enabled**
- **Step 3: Approve and Deploy Updates**
  - In the **WSUS Administration Console**, you approve updates for specific computer groups:
  - Go to **Updates** and filter by "Updates needed by computers".
  - Select the updates for your test group, right-click, and select **Approve**.
  - Set the approval to **"Approved for Install"**

Post Installation

- **Configure the Firewall**
  - Ensure client computers can reach the WSUS server.
  - The default ports are **8530 (HTTP)** and **8531 (HTTPS)**
  - The WSUS server itself must also be allowed outbound access to Microsoft domains on ports **80 and 443**
- **Set Up Computer Groups**
  - Create groups in the WSUS console to stage update deployments
  - You can assign clients to these groups manually or use **Client-Side Targeting** via Group Policy
- **Configure Client Machines**
  - Use Group Policy to point your domain computers to the WSUS server
  - Create a GPO with the policy **"Specify intranet Microsoft update service location"** under
    - Computer Configuration > Administrative Templates > Windows Components > Windows Update
- **Approve Updates**
  - After the first synchronization, updates will be listed in the console but not yet approved for installation
  - You must right-click the updates you want to deploy, select **Approve**, and choose the target computer group

Operational Best Practices

- Capacity and Architecture: Avoid the Single Server Trap
- Essential IIS Tuning to Prevent Crashes
- Security: Protect the Connection
- The Golden Rule of Maintenance: Decline Superseded Updates
- Client Targeting: Use Group Policy for Scale
- Sync Only What You Need
- Use SQL Server for large organization
- Monitor reports regularly
- Backup of your update database regularly